國立臺灣海洋大學108年度大學生暑期學習實務體驗計畫
執行成果報告

計畫名稱：結構分析APP製作

學生姓名：白謹瑜、王奕杰、李承曄、林昆毅

就讀系所：系統工程暨造船學系

學生學號：00651029、00651026、00651010、00651025

指導教授：關百宸 教授

研究期間： 108 年 7 月 1 日至 8 月 31 日底止，共計2個月。

指導教授簽名：關百宸

中華民國 108 年 9 月 20 日

致謝

在這次的大暑計畫中，我們團隊要最先感謝關百宸老師，除了給予我們執行大暑計畫的這個機會以外，還教授給我們非常詳盡的力學知識和計畫中會使用到的計算理論。給予我們程式設計的教學書籍，提供給程式設計新手的我們一個切入的管道，以及老師的有限元素法簡報教材，不管是在計畫執行中的計算程式設計，或者是結案報告的編寫方面上，都提供了莫大的幫助。在我們進度停滯不前時，老師也提供給我們設計思路以及即時修正我們的開發方向，讓我們能順利找到突破口使開發進度得以順利推進。在結案報告的編寫方面，老師的高水準要求讓所有成員都知道我們的報告缺乏了那些要素，讓我們深刻的體認到自身的不足，是一次相當刻骨銘心，但是讓我們受益匪淺的教訓。我們相當感謝老師給予的這次機會以及種種幫助，在這裡致上最深的謝意。

再來要感謝結構研究室的所有研究生學長，大家在暑假期間相當的關照我們，尤其是何信彥學長還有官世倫學長，他們是執行過大暑計畫的經驗者，分享給我們一些過來人的計畫執行經驗給我們參考。還有在視察我們工作進度的時候，提供給我們從旁人角度對我們設計的 APP 一些看法，像是使用者想要什麼需求，使用起來的心得等等，讓 APP 可以往更好使用的方向進行修改。在計畫接近尾聲的時候，告訴我們結案報告的文章架構應該如何設計,提供給我們初步的方向。感謝研究室的學長們對我們伸出的援手。

最後要特別感謝在 GitHub 論壇上編寫 Android Studio 自定義 View 教學系列的作者 GcsSloop，這系列的教學文章讓完全不懂 Android Studio 的我們透過其系統化、淺顯易懂的說明讓我們快速的了解 Android Studio 在視覺設計方面的運作機制，完善的程式碼封包介紹介紹使我們了解如何設計出客製化的 Android 自定義視覺介面，這些教學對我們的計畫有相當龐大的貢獻。

摘要

本次計畫目標為設計一個可以進行結構分析的計算用 APP。此 APP 針對的使用對象為有研修材料力學以及結構學課程的學生，目的在於提供學生另一個求出作業習題答案的管道，使其學習效率提升。此類型的 APP 含有的功能要有基本的建立點座標、用點座標形成 Truss、在桿件上施加外力，並且額外加上支承提供限制條件，形成可以完整計算的桿件，最後將以上數種元素組合完成的自由體圖進行計算並將結果輸出到螢幕上，將答案提供給使用者。

此結構分析 APP 開發於 Android 平台上，使用的開發環境為 Google 釋出的免費開發環境 Android Studio 3.5，使用的程式語言是 Java 語言。開發過程中，介面開發的部分大量的參考 GcsSloop 編寫的自定義教學系列[6]　GcsSloop，自定義 View 教學系列的方法以及使用了 GcsSloop 開發的繪製坐標軸輔助工具包 CanvasAidUtils[14] GcsSloop，繪製座標系輔助工具-CanvasAidUtils。

程式計算部分則採用結構學之直接徑度法。在數學運算上，我們使用到數值分析的運算技巧，像是 LU 運算解線性系統，或是在之後會使用的牛頓法求極值等等。而我們自己也設計許多特殊邏輯來使用，例如在直接徑度法的矩陣的填入與刪除、Truss 桿件內力的計算以及 Beam 中剪力彎矩圖的呈現等等，確保能計算出完全正確的數據，提高 APP 的可信度。

目錄

第一章 緒論.....8

1.1 研究動機.....8

1.2 選擇的工具.....8

第二章 基本理論.....10

2.1 矩陣的基本性質.....10

2.2 直接勁度法(DIRECT STIFFNESS METHOD).....12

2.3 LU 矩陣分解.....26

第三章 JAVA 介面開發.....30

3.1 自定義 VIEW.....30

3.2 ONTOUCHLISTENER 監聽介面.....32

3.3 繪圖方法&建立數據庫.....33

1. 節點(Node).....38

2. 桿件(Truss & Beam).....39

3. 外力(Force).....39

4. 力矩(Moment).....40

5. 支承(Support).....41

6. 剪力、彎矩、變形、扭轉角圖表.....41

3.4 鎖點.....42

3.5 偵測 & 選擇物件之邏輯.....44

3.6 清除畫面&重繪.....45

3.7 其他介面物件.....46

(1) 工具列 (ActionBar).....46

(2) 彈出視窗 (Dialog).....46

(3) 滾動視窗 (ScrollView).....47

3.8 切換 ACTITIVY.....47

第四章 JAVA 計算程式開發.....49

4.1 直接徑度法.....49

(1) Truss.....49

(2) Beam.....50

4.2 LU 逆矩陣運算 52
第五章 APP 使用說明 55
第六章 範例演示 66
第七章 結論 72
第八章 成果自評 73
第九章 指導教授評語 75
參考文獻 76

圖目錄

圖 1　力(Force)應力(stress)應變(strain)形變(deformation)位移(displacement)關係式 13
圖 2　一維和二維狀況下衍架的節點位移量 13
圖 3　局部向量矩陣 $\boldsymbol{u}$、全域向量矩陣 $\boldsymbol{U}$ 與轉換矩陣的關係 14
圖 4　局部力和全域力關係圖與關係式 14
圖 5　局部向量矩陣 f 、全域向量矩陣 F 與轉換矩陣之間的關係 14
圖 6　局部向量 $\boldsymbol{f}$ 和局部向量 $\boldsymbol{u}$ 的關係圖與關係式 15
圖 7　Truss 計算範例、圖 8 矩陣表示各自由度所受之力 16
圖 9　各桿件之直接勁度法 16
圖 10　K^1 矩陣之展開和矩陣間元素對照 17
圖 11　K 矩陣之填入方式 17
圖 12　K 矩陣之線性疊加 18
圖 13 集中力、圖 14　分佈力、圖 15　彎矩 18
圖 16 樑之微小變形放大圖 19
圖 17 解釋剪力微分後為什麼等於分布力 20
圖 18 樑受力模型 21
圖 19 樑端點之位移變形與角度變形 22
圖 20 樑剪力與彎矩之預設方向 24
圖 21 自定義 View 的程式調用鍊 30
圖 22 構造函數範例 31
圖 23　layout 資料夾中放置的 xml 格局檔 31
圖 24 常用的自定義 View 構造函數 31
圖 25 調用設有單一參數構造函數的自定義 View 31
圖 26 調用設有兩個參數構造函數的自定義 View 32
圖 27 簡化後的物件責任鍊 33
圖 28　Canvas(畫布)和 Bitmap(位圖)的建立 34
圖 29 設置畫筆之例子 34
圖 30 手機螢幕預設座標系、圖 31 繪圖介面座標系 34
圖 32　Canvas 座標移動 35
圖 33 繪製 xy 軸之副程式 35
圖 34 將 View 設定為 onTouchListener 監聽器之方法 36
圖 35 繪製節點副程式 36
圖 36 建立 Arraylist 之範例 37
圖 37　Arraylist 多重讀取參數 37
圖 38 碰觸座標之座標轉換 38
圖 39 繪製桿件之副程式 39
圖 40 繪製圓弧之方法 40

圖 41 繪製力矩畫筆之屬性 41
圖 42 支承示意圖 41
圖 43 繪製折線圖之 View 的座標偏移量 42
圖 44 剪力折線圖 42
圖 45 x 方向之鎖點判斷 43
圖 46 y 方向之鎖點判斷 43
圖 47 防重複建立節點之邏輯 44
圖 48 搜尋物件目錄值的邏輯 44
圖 49 選取物件使其變色的方法 45
圖 50 重新繪製流程 46
圖 51 Intent(意圖)使用方法 48
圖 52 取得節點數據 49
圖 53 依照自由度的編號填寫 K 矩陣 50
圖 54 彎矩函數之程式碼 51
圖 55 剪力函數之程式碼 51
圖 56 旋轉徑度函數之程式碼 52
圖 57 垂直位移之程式碼 52
圖 58 矩陣相乘之方程式 52
圖 59 反向迭帶(BackwardSupstitution)之程式碼 53
圖 60 正向迭帶(FordwardSupstitution)之程式碼 53
圖 61 將 A 矩陣分解之程式碼 54
圖 62 演示進入繪製 Truss 介面 55
圖 63 演示建立節點 56
圖 64 演示建立桿件 56
圖 65 演示建立支承和外力 57
圖 66 演示更改外力屬性演示 58
圖 67 演示 Truss 計算結果演示 58
圖 68 演示進入繪製 Beam 介面 59
圖 69 演示在 Beam 繪圖介面建立節點 59
圖 70 演示在 Beam 介面建立桿件 60
圖 71 演示建立 Support 中的 Pin 61
圖 72 演示建立 Support 中的 Roller 62
圖 73 演示編輯外力屬性(1) 63
圖 74 演示編輯外力屬性(2) 64
圖 75 演示 Beam 顯示答案 65
圖 76 Truss 範例題目(1) 66
圖 77 Truss 範例題目(2) 67
圖 78 程式計算之答案顯示 68

圖 79 滾動視窗中的答案數據 68
圖 80 Beam 範例題目(1) 69
圖 82 Beam 程式計算後的答案介面(1) 70
圖 83 程式計算後的答案介面(2) 71
圖 84 結構分析 APP 載點 74

表目錄

表格 1 可處理之單點碰觸事件 32
表格 2 在觸摸事件中可使用之方法 33
表格 3 Truss 例題和程式計算之答案比較 68
表格 4 Beam 例題和程式計算答案之比較 71

第一章 緒論

1.1 研究動機

在上大二上學期的必修，材料力學這門課程時，不外乎會使用課本上的習題進行練習、計算，才能夠真正把學到的理論應用到求解上，而書內大部分的習題都可以在網路上找到其他人所提供的解答。然而，在尋找答案的時候，部分題目會發生自己算出的答案與網路上提供的解答不同的情形，甚至發現同一個題目在網路上出現的解答會有好幾個的情形。就算多次計算、和同儕進行答案對比，仍和原本求出的答案相同。最後詢問了指導教授，才發現其實是網路上解答有誤，可能是答案提供者的觀念有些許錯誤、用的方法少了某個步驟，或者是單純的計算錯誤。雖然最後還是得到正確解答，卻因此浪費了不少時間在研究錯誤的答案上。所以我們打算開發出一個可以進行結構分析的 APP。這個 APP 可以運用在結構學相關的習題上，也等於提供修習材料力學與結構學的學生另一個求出確認答案的管道，學生可以使用它進行計算，降低為了尋求答案而耗費的時間成本，提升學習的效率。

1.2 選擇的工具

在計畫開始前，我們找了數種開發軟體，打算從中選擇符合我們計畫需求的作為此計畫使用的開發工具，選項有：G suite Google APP maker、eclipse、Android Studio、Xcode 這四樣開發軟體。我們的需求有：發布在 Android 平台上、使用 Java 語言開發、入手門檻較低、開發成本最小。我們選擇發布在 Android 平台上的原因是我們觀察了現今手機市場環境和周遭同學使用的大多是 Android 系統，因此在程式初步完成後能發布給較多用戶，可以得到較多的測試心得，讓日後的我們能往正確的方向進行改善。使用 Java 語言的原因是因為它是現今 Android APP 開發所使用的主流程式語言，再加上小組成員裡大部份都學習過 Java 語言，選擇 Java 會讓開發難度降低不少。此外，因為此程式都是小組成員的處女作，為了減少失敗成本，我們會盡量從選項中優先選擇門檻低、花費少的開發軟體作為最終結果。

Xcode 開發平台是用來開發運作在 IOS 平台上的 APP，使用的程式語言是 Ruby，這兩點並不符合我們的需求。不過，等程式開發完成以後，我們打算使用 Xcode 將開發完成的 APP 移植到 IOS 平台上，因此雖然 Xcode 並非現階段的首選，但在日後仍有機會使用到。

G suite Google App maker 是 Google 提供給企業使用的簡化版開發軟體，特色是開發端能雲端共用、使用者數據統整以及簡單的開發介面。但是經過調查後發現 APP maker 似乎無法開發出我們想要呈現的使用者繪圖效果，加上在開發過程中還需要定期繳交租借網域費用和雲端使用費，需耗費的金錢成本高於其他選項許多，故不選擇此方案。

Eclipse 是我們的最初選擇，是前幾年前開發 Android APP 的主流，也是我們在學習 Java 語言時使用的開發軟體，是著名的非營利跨平台開源整合式開發環境。Eclipse 支援許多的程式語言以及外掛程式，擁有較佳的靈活性，小組成員大部分都有使用 Eclipse 的經驗，減少了部分學習門檻。但後來發現，如果要在 Eclipse 上開發圖形介面，需要下載許多外掛工具套件，相當麻煩。另外建構畫面的程式碼要使用 Java 以外的其他語言進行更改，過於困難也不直觀，要進行 debug 會耗費相當多的時間成本，顯示的介面結果也較為粗糙，外加 Android 的開發商 Google 已經暫停更新能在 eclipse 中開發 android APP 的軟體開發套件（**S**oftware **D**evelopment **K**it，SDK）。故我們放棄使用 eclipse 作為本計畫的開發工具。

Android Studio，是 Android 的開發商 Google 於 2013 年釋出的免費整合式開發環境，到現今仍以半年一次的頻率持續進行更新，目前的版本為 3.5，是 Google 官方指定的 Android 整合開發環境（**I**ntegrated **D**evelopment **E**nvironment，IDE），從釋出至今已經成為開發 Android APP 的主流開發工具。Android Studio 是以常見的 Java 整合開發環境 IntelliJ IDEA 作為基礎研發出來，是為了開發 Android APP 這個目的而特製化的開發軟體。Android Studio 對此計畫帶來便利的功能有：視覺化布局、內建布局編輯器、智能程式碼編輯、效能分析工具、內建 Android SDK 和 Android 模擬器（**A**ndroid **V**irtual **D**evice，AVD）管理器。

視覺化布局和內建布局編輯器為負責圖形介面的人帶來相當大的方便，可以靠拖拉編輯清單中預設的物件以及在屬性視窗更改參數就完成物件配置。布局編輯器使用的呈現方式是所見即所得，所以就算使用程式碼進行布局修改，也可以在布局編輯器上面馬上看到更動。智能程式碼編輯對程式開發人員是一大福音，只要輸入片段程式碼，就會列出具有該關鍵字的可用函數，新手可以透過該功能在開發過程中逐步瞭解函式庫使用時機和功能，老手也可以縮短編寫程式碼的時間，對所有程度開發者都是相當便利的功能。內建效能分析工具、Android SDK & ADB 讓開發者不必去加載額外套件，就可以在程式編寫到一定階段後馬上在手機模擬器 ADB 上進行試運行，使用這兩項工具可以輕鬆地觀察出執行那些動作會帶來什麼樣的結果，例如：建立物件的時候程式崩潰、繪製的圖形不如預期，ADB 能帶來這些明顯的回饋，給開發者一個清楚的方向去進行修正。又或者是畫面呈現沒問題，但透過效能分析工具發現內存或是 CPU 負載過大，這時候就知道必須去優化內部程式進行調整。上述特色大幅減少了關於 APP 開發的學習成本和時間成本，增進計畫的開發效率。經由比較過後，Android Studio 除了符合我們的需求以外，還明顯優於其他選項，因此我們最終選擇 Android Studio 作為此計畫使用的開發工具。

第二章 基本理論

2.1 矩陣的基本性質

由於此計畫的核心計算方式是使用矩陣運算，在這裡介紹矩陣的使用方法和特性。

(1) 使用矩陣表示向量

假設一向量 $\vec{F} = a\hat{i} + bj + ck$ ，則我們可以使用矩陣表示成：$\vec{F} = \begin{bmatrix} \frac{\partial F}{\partial x} \\ \frac{\partial F}{\partial y} \\ \frac{\partial F}{\partial z} \end{bmatrix} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$

(2) 使用矩陣表示方程式

假設一方程式 $\begin{cases} a_1x + b_1y + c_1z = d_1 \\ a_2x + b_2y + c_2z = d_2 \\ a_3x + b_3y + c_3z = d_3 \end{cases}$ ，則可以用矩陣寫成 $\begin{bmatrix} a_1 & b_1 & c_1 & d_1 \\ a_2 & b_2 & c_2 & d_2 \\ a_3 & b_3 & c_3 & d_3 \end{bmatrix}$ ，並使用高斯消去法求解x、y、z。

(3) 高斯消去法(Gauss Elimination Method)

假設有一個矩陣如下：

$$\begin{matrix} \text{line1} \\ \text{line2} \\ \text{line3} \end{matrix} \begin{bmatrix} 2 & 1 & 1 \\ 3 & 2 & 4 \\ 5 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 13 \\ 32 \\ 17 \end{bmatrix}$$

$$\text{(Step.1)} \quad \frac{\text{line1}}{a_{11}} \Rightarrow \begin{bmatrix} 1 & 0.5 & 0.5 \\ 3 & 2 & 4 \\ 5 & -1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.5 \\ 32 \\ 17 \end{bmatrix}$$

$$\text{(Step.2)} \quad \begin{matrix} \text{line2} - \text{line1} \times a_{21} \\ \text{line3} - \text{line1} \times a_{31} \end{matrix} \Rightarrow \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0 & 0.5 & 2.5 \\ 0 & -3.5 & 0.5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.5 \\ 12.5 \\ -15.5 \end{bmatrix}$$

$$\text{(Step.3)} \quad \text{line2} / a_{22} \Rightarrow \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0 & 1 & 5 \\ 0 & -3.5 & 0.5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.5 \\ 25 \\ -15.5 \end{bmatrix}$$

(step.4) $\text{line3} - \text{line2} \times a_{32} \Rightarrow \begin{bmatrix} 1 & 0.5 & 0.5 \\ 0 & 1 & 5 \\ 0 & 0 & 18 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6.5 \\ 25 \\ 72 \end{bmatrix}$

$x_3 = 72/18 = 4$

最後，可得：$x_2 = 25 - 5x_3 = 5$

$x_1 = 6.5 - 0.5x_2 - 0.5x_3 = 2$

(4) 轉置矩陣(Transformation matrix)

假設一矩陣 A，欲得到 A 的轉置矩陣 A^T，則需要把 A 的橫行寫為 A^T 的縱列，把 A 的縱列寫為 A^{T} 的橫行。舉例來說：$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$。

形式上，$m \times n$ 矩陣 A 的轉置是 $n \times m$ 矩陣，而在剛度法中，所使用的為下列的矩陣：

$$\boldsymbol{T} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & \sin\theta & \cos\theta \end{bmatrix} \tag{1}$$

$\boldsymbol{T}$ 矩陣的名稱為旋轉矩陣，它具有能夠改變向量的方向但是不改變向量大小的效果，此處用途是將局部座標轉換成全域座標，是進行座標轉換的工具。旋轉矩陣有一個特殊性質，它的逆矩陣 T^{-1} 是它本身的轉置矩陣 $\boldsymbol{T^T}$，$\boldsymbol{T^{-1} = T^T}$。透過逆矩陣的性質 $\boldsymbol{AA^{-1}=A^{-1}A=I}$ 我們最後可以得出 $\boldsymbol{TT^{-1}=TT^T=I}$ (單位矩陣)的結論。

(5) 單位矩陣(Identity Matrix / Unit Matrix)

$$\boldsymbol{I} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

單位矩陣它並不會影響計算結果，因此方程式中出現 $\boldsymbol{I}$ 的時候可以將其省略。

(6) 對角矩陣(Diagonal Matrix)

主對角線之外的元素皆為 0 的矩陣，稱之為對角矩陣。對角線上的元素可以為 0 或其他值。

$$\boldsymbol{A}=\begin{bmatrix} a_1 & 0 & 0 & 0 \\ 0 & a_2 & 0 & 0 \\ 0 & 0 & a_3 & 0 \\ 0 & 0 & 0 & a_4 \end{bmatrix}$$

(7) 帶狀矩陣(Banded Matrix)

$$\boldsymbol{B}=\begin{bmatrix} b_{11} & b_{12} & 0 & 0 \\ b_{21} & b_{22} & b_{23} & 0 \\ 0 & b_{32} & b_{33} & b_{34} \\ 0 & 0 & b_{43} & b_{44} \end{bmatrix}$$

(8) 矩陣乘法的規則(Rule for matrix multiplication)

無交換率(Not Commutative)： $\boldsymbol{A}\cdot\boldsymbol{B}\neq\boldsymbol{B}\cdot\boldsymbol{A}$

結合率(Associative)： $\boldsymbol{A}\cdot\left(\boldsymbol{B}\cdot\boldsymbol{C}\right)=\left(\boldsymbol{A}\cdot\boldsymbol{B}\right)\cdot\boldsymbol{C}$

分配率(Distributive law)： $\left(\boldsymbol{A}+\boldsymbol{B}\right)\cdot\boldsymbol{C}=\boldsymbol{A}\cdot\boldsymbol{C}+\boldsymbol{B}\cdot\boldsymbol{C}$

對於一個 $n\times n$ 的矩陣： $\boldsymbol{A}^n=\underbrace{\boldsymbol{A}\cdot\boldsymbol{A}\cdot\boldsymbol{A}\cdot\boldsymbol{A}...\cdot\boldsymbol{A}}_{n}$

(9) 矩陣的加減法

$$\boldsymbol{A}\pm\boldsymbol{B}=\begin{bmatrix} \boldsymbol{A}_{11}\pm\boldsymbol{B}_{11} & \boldsymbol{A}_{12}\pm\boldsymbol{B}_{12} \\ \boldsymbol{A}_{21}\pm\boldsymbol{B}_{21} & \boldsymbol{A}_{22}\pm\boldsymbol{B}_{22} \end{bmatrix}$$

(10) 矩陣的乘法

$$\boldsymbol{A}\cdot\boldsymbol{B}=\begin{bmatrix} \boldsymbol{A}_{11} & \boldsymbol{A}_{12} \\ \boldsymbol{A}_{21} & \boldsymbol{A}_{22} \end{bmatrix}\cdot\begin{bmatrix} \boldsymbol{B}_{11} & \boldsymbol{B}_{12} \\ \boldsymbol{B}_{21} & \boldsymbol{B}_{22} \end{bmatrix}=\begin{bmatrix} \boldsymbol{A}_{11}\cdot\boldsymbol{B}_{11}+\boldsymbol{A}_{12}\cdot\boldsymbol{B}_{21} & \boldsymbol{A}_{11}\cdot\boldsymbol{B}_{12}+\boldsymbol{A}_{12}\cdot\boldsymbol{B}_{22} \\ \boldsymbol{A}_{21}\cdot\boldsymbol{B}_{11}+\boldsymbol{A}_{22}\cdot\boldsymbol{B}_{21} & \boldsymbol{A}_{21}\cdot\boldsymbol{B}_{12}+\boldsymbol{A}_{22}\cdot\boldsymbol{B}_{22} \end{bmatrix}$$

2.2 直接勁度法(Direct Stiffness Method)

直接勁度法是藉由描述單一桿件的力平衡行為，並將力平衡方程式轉換成線性系統，以將複雜結構轉換為聯立方程式求解的一種基本方法。進行結構分析時，可以對一個結構體可承受的外力種類將其分類成 Truss(衍架)、Beam(樑)、Frame(構架)這三種結構，再透過不同結構具有的特性去進行受力分析。在直接勁度法中，我們會使用到名為自由度(Degree of Freedom)的基本參數。自由度在結構力學的定義為，在分析結構系統時，有效的結構節點上之未知節點變位，自由度可再細分為旋轉自由度和移動自由度。當節點可以對相對於參考座標系的某方向具有獨立運動參數，該節點即是擁有該方向的自由度，最後去計算節點的自由度數量或整個桿件的自由度數量得知矩陣大小。

Truss(衍架)是個只有在兩端點有受力，沒有承受彎矩的二力構件，因此在 Truss 的節點上不會有旋轉的現象發生，所以 Truss 中的每個節點只能在 X 方向和 Y 方向進行位移，也就是每一個節點都只有兩個自由度。由於直接勁度法是以一個桿件為單位進行分析，而一根 Truss 構件中只有起點和終點這兩個節點，所以 Truss 構件總共有 4 個自由度。圖 1 為結構中各個參數之間的關係式，A 為桿件截面面積、σ 為應力(單位面積之受力)、ε 為應變(長度變化率)、E 為楊氏模數(Young's Modulus，又稱彈性模數，關係式如(3))、L 為桿件原長、$\triangle L$ 為桿件長度變化量。而從圖 1 的關係式可以推導出施加外力的公式(4)，公式中的 $\frac{AE}{L}$ 我們使用 k (stiffness，剛度)將其取代。

$$E = \frac{\sigma}{\varepsilon} \tag{3}$$

$$F = \frac{AE}{L}\triangle L = k\triangle L \tag{4}$$

Force-stress $\sigma = \frac{F}{A}$

Strain-deformation $\varepsilon = \frac{\Delta L}{L}$

Stress-strain $\sigma = E\varepsilon$

圖 1　力(Force)-應力(stress)-應變(strain)-形變(deformation)-位移(displacement)關係式

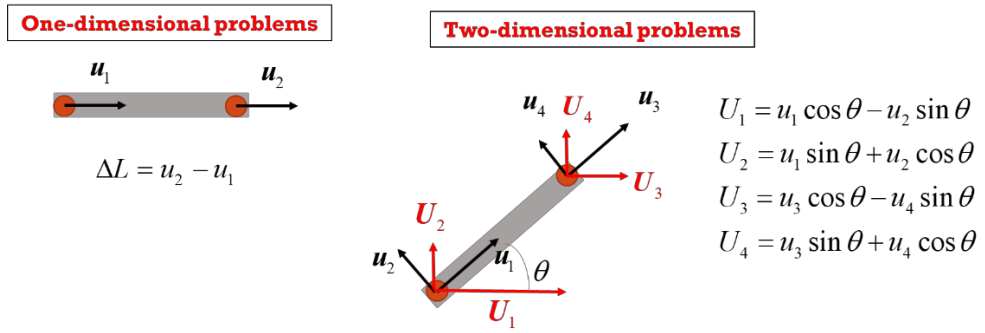


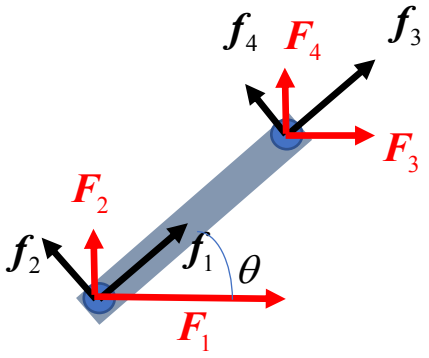
圖 2　一維和二維狀況下衍架的節點位移量

透過 Truss 有四個自由度的性質代表可以得到圖 2 中四條位移與外力的方程式，圖 2 的 u 是以 Truss 軸向為座標系(局部座標系)的位移量，U 是使用卡氏座標系(全域座標系)的位移量，U_1 為起點的 X 方向位移量、U_2 為起點的 Y 方向位移量、U_3 為終點的 X 方向位移量、U_4 為終點的 Y 方向位移量。可以發現 u 和 U 之間呈線性關係，因此我們可以將這四條方程式以矩陣排列，如圖 3：

$$\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} \Rightarrow \quad \boldsymbol{U = Tu}$$

圖 3 局部向量矩陣 $\boldsymbol{u}$、全域向量矩陣 $\boldsymbol{U}$ 與轉換矩陣的關係

圖 4 中的 f 是使用軸向座標表示的局部力，代表桿件內力，F 是使用卡氏座標表示的全域力，代表節點所受外力。從圖 4 和圖 5 中可以發現，我們可以獲得和 Truss 自由度相同數目之 f 和 F 的線性方程式，將方程式寫成矩陣表示後，兩者之間也是透過旋轉矩陣進行座標轉換。

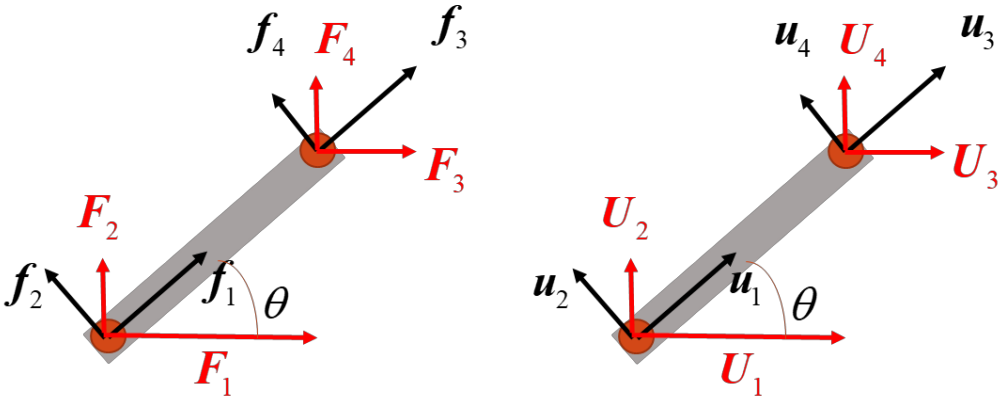


$$F_1 = f_1\cos\theta - f_2\sin\theta$$
$$F_2 = f_1\sin\theta + f_2\cos\theta$$
$$F_3 = f_3\cos\theta - f_4\sin\theta$$
$$F_4 = f_3\sin\theta + f_4\cos\theta$$

圖 4 局部力和全域力關係圖與關係式

$$\begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & \cos\theta & -\sin\theta \\ 0 & 0 & \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} \Rightarrow \boldsymbol{F = Tf}$$

圖 5 局部向量矩陣 f、全域向量矩陣 F 與轉換矩陣之間的關係



$$f_1 = k\Delta L = -k\left(u_2 - u_1\right)$$
$$f_3 = k\Delta L = k\left(u_2 - u_1\right)$$

$$\begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix} = \begin{bmatrix} k & 0 & -k & 0 \\ 0 & 0 & 0 & 0 \\ -k & 0 & k & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} \qquad \boldsymbol{f} = \boldsymbol{k}^e \boldsymbol{u}$$

圖 6　局部向量 $\boldsymbol{f}$ 和局部向量 $\boldsymbol{u}$ 的關係圖與關係式

透過公式(2)、圖 2 和圖 6，我們將使用局部座標的內力和位移的關係方程式轉換成矩陣的表示形式，即可得知了 f 和 u 兩者之間的關聯性，再加上圖 5 中不同座標系的 f 和 F 之間的關聯以及旋轉矩陣的特殊性質，最後統合出從局部座標系轉換成全域座標系的兩條關係式，$\boldsymbol{F} = \boldsymbol{KU}$ 求解即可得到正確答案：

$$\boldsymbol{f} = \boldsymbol{T}^{-1}\boldsymbol{F} = \boldsymbol{T}^{T}\boldsymbol{F} = \boldsymbol{k}^{e}\boldsymbol{u} = \boldsymbol{k}^{e}\boldsymbol{T}^{T}\boldsymbol{U} \tag{5}$$

$$\boldsymbol{F} = \boldsymbol{T}\boldsymbol{k}^{e}\boldsymbol{T}^{T}\boldsymbol{U} \Rightarrow \boldsymbol{F} = \boldsymbol{K}\boldsymbol{U} \tag{6}$$

$\boldsymbol{f}, \boldsymbol{u}, \boldsymbol{k}^e$ 是使用局部座標系的力、位移向量、剛度矩陣，$\boldsymbol{F}, \boldsymbol{U}, \boldsymbol{K}$ 是使用全域座標系的力、位移向量、剛度矩陣。由於我們是以單一 Truss 作為範例，因此公式(6)裡的 $\boldsymbol{K}$ 矩陣是範例的全域剛度矩陣，如果該結構是由多根 Truss 所構成，公式(6)的 $\boldsymbol{K}$ 矩陣就成為該結構的單位剛度矩陣。從 $\boldsymbol{K} = \boldsymbol{T}\boldsymbol{k}^{e}\boldsymbol{T}^{T}$ 這條關係式的 $\boldsymbol{k}^e$ 取出代表 $\frac{AE}{L}$ 的常數剛度 k 後，最後推導出公式(7)的結果：

$$\boldsymbol{K} = k\begin{bmatrix} \cos^2\theta & \sin\theta\cos\theta & -\cos^2\theta & -\sin\theta\cos\theta \\ \sin\theta\cos\theta & \sin^2\theta & -\sin\theta\cos\theta & -\sin^2\theta \\ -\cos^2\theta & -\sin\theta\cos\theta & \cos^2\theta & \sin\theta\cos\theta \\ -\sin\theta\cos\theta & -\sin^2\theta & \sin\theta\cos\theta & \sin^2\theta \end{bmatrix} \tag{7}$$

每個桿件都有相對應的 $\boldsymbol{K}$ 矩陣，但是由所得到的 $\boldsymbol{K}$ 矩陣，其行列式為零，主要代表剛體位移或旋轉時無窮多解的狀態，所以無法求得其反矩陣。故須根據給予的邊界條件，對 $\boldsymbol{K}$ 矩陣進行刪減，使其行列式不為零。接下來以範例解釋計算方式：

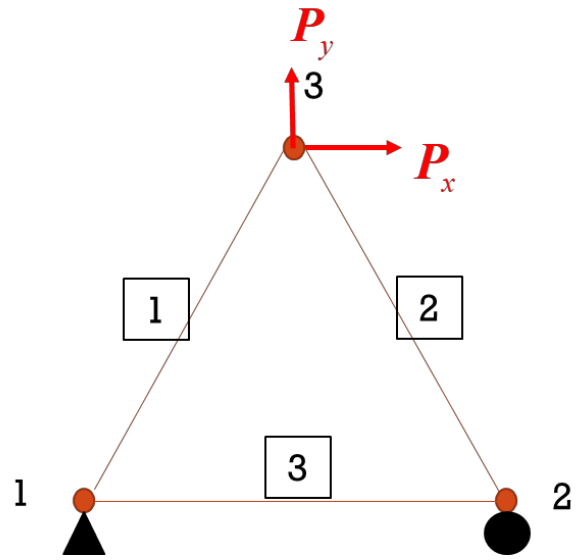


圖 7　Truss 計算範例

$$\begin{bmatrix} R_{1x} \\ R_{1y} \\ 0 \\ R_{2y} \\ P_x \\ P_y \end{bmatrix} = \begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \end{bmatrix} + \begin{bmatrix} F_{1x}^3 \\ F_{1y}^3 \end{bmatrix} + \begin{bmatrix} F_{2x}^2 \\ F_{2y}^2 \end{bmatrix} + \begin{bmatrix} F_{2x}^3 \\ F_{2y}^3 \end{bmatrix} + \begin{bmatrix} F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} + \begin{bmatrix} F_{3x}^2 \\ F_{3y}^2 \end{bmatrix}$$

圖 8　矩陣表示各自由度所受之力

Element one

$$\begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \\ F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} = K^1 \begin{bmatrix} U_{1x}^1 \\ U_{1y}^1 \\ U_{3x}^1 \\ U_{3y}^1 \end{bmatrix}$$

Element two

$$\begin{bmatrix} F_{2x}^2 \\ F_{2y}^2 \\ F_{3x}^2 \\ F_{3y}^2 \end{bmatrix} = K^2 \begin{bmatrix} U_{2x}^2 \\ U_{2y}^2 \\ U_{3x}^2 \\ U_{3y}^2 \end{bmatrix}$$

Element three

$$\begin{bmatrix} F_{1x}^3 \\ F_{1y}^3 \\ F_{2x}^3 \\ F_{2y}^3 \end{bmatrix} = K^3 \begin{bmatrix} U_{1x}^3 \\ U_{1y}^3 \\ U_{2x}^3 \\ U_{2y}^3 \end{bmatrix}$$

圖 9 各桿件之直接勁度法

圖 7 中的是由三個節點，三根 Truss 組合而成的結構體，此結構在節點 3 的位置承受了 X 方向的 P_x 和 Y 方向的 P_y 外力。由此結構的構成元素可以得知該結構體有 6 個自由度，每個自由度受到的外力和支承反力以圖 7　Truss 計算範例　　圖 8　矩陣表示各自由度所受之力等號左邊的矩陣排列，我們可將每個節點自由度之受力再分解成圖 7　Truss 計算範例　　圖 8　矩陣表示各自由度所受之力等號右邊的形式，方便觀察出計算的規律。從圖 7　Truss 計算範例　　圖 8　矩陣表示各自由度所受之力右邊的矩陣依據 F 符號右上角的桿件編號進行分類，分類完成後參考 $\boldsymbol{F} = \boldsymbol{KU}$ 分別列出圖 9 中每根 Truss 的直接勁度法。每根桿件的直接勁度法之中的 $\boldsymbol{K}$ 矩陣只是縮寫，它的展開情形為上面提到的公式(7)再將矩陣外的剛度 k 乘入矩陣中的每個元素之中，以圖 9 的 element one 為例，將它的 K^1 進行展開再和公式(7)進行對照，即可得知，$K_{12}^1 = k\sin\theta\cos\theta$，$K_{13}^1 = -k\cos^2\theta$，$K_{14}^1 = -k\sin\theta\cos\theta$ …逐一對應至矩陣中的位置即可得知 K^1 矩陣之中的所有元素。

$$\begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \\ F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} = K^1 \begin{bmatrix} U_{1x}^1 \\ U_{1y}^1 \\ U_{3x}^1 \\ U_{3y}^1 \end{bmatrix} \Rightarrow \begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \\ F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} = \begin{bmatrix} K_{11}^1 & K_{12}^1 & K_{13}^1 & K_{14}^1 \\ K_{21}^1 & K_{22}^1 & K_{23}^1 & K_{24}^1 \\ K_{31}^1 & K_{32}^1 & K_{33}^1 & K_{34}^1 \\ K_{41}^1 & K_{42}^1 & K_{43}^1 & K_{44}^1 \end{bmatrix} \begin{bmatrix} U_{1x}^1 \\ U_{1y}^1 \\ U_{3x}^1 \\ U_{3y}^1 \end{bmatrix}$$

$$\boldsymbol{K} = k \begin{bmatrix} \cos^2\theta & \sin\theta\cos\theta & -\cos^2\theta & -\sin\theta\cos\theta \\ \sin\theta\cos\theta & \sin^2\theta & -\sin\theta\cos\theta & -\sin^2\theta \\ -\cos^2\theta & -\sin\theta\cos\theta & \cos^2\theta & \sin\theta\cos\theta \\ -\sin\theta\cos\theta & -\sin^2\theta & \sin\theta\cos\theta & \sin^2\theta \end{bmatrix}$$

圖 10　K^1 矩陣之展開和矩陣間元素對照

$$\begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \\ F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} = K^1 \begin{bmatrix} U_{1x}^1 \\ U_{1y}^1 \\ U_{3x}^1 \\ U_{3y}^1 \end{bmatrix} \Rightarrow \begin{bmatrix} F_{1x}^1 \\ F_{1y}^1 \\ F_{3x}^1 \\ F_{3y}^1 \end{bmatrix} = \begin{bmatrix} K_{11}^1 & K_{12}^1 & K_{13}^1 & K_{14}^1 \\ K_{21}^1 & K_{22}^1 & K_{23}^1 & K_{24}^1 \\ K_{31}^1 & K_{32}^1 & K_{33}^1 & K_{34}^1 \\ K_{41}^1 & K_{42}^1 & K_{43}^1 & K_{44}^1 \end{bmatrix} \begin{bmatrix} U_{1x}^1 \\ U_{1y}^1 \\ U_{3x}^1 \\ U_{3y}^1 \end{bmatrix}$$

$$\begin{bmatrix} R_{1x} \\ R_{1y} \\ 0 \\ R_{2y} \\ P_x \\ P_y \end{bmatrix} = \begin{bmatrix} F_{1x}^1 + F_{1x}^3 \\ F_{1y}^1 + F_{1y}^3 \\ F_{2x}^2 + F_{2x}^3 \\ F_{2y}^2 + F_{2y}^3 \\ F_{3x}^1 + F_{3x}^2 \\ F_{3y}^1 + F_{3y}^2 \end{bmatrix} = \begin{bmatrix} K_{11}^1 & K_{12}^1 & 0 & 0 & K_{13}^1 & K_{14}^1 \\ K_{21}^1 & K_{22}^1 & 0 & 0 & K_{23}^1 & K_{24}^1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ K_{31}^1 & K_{32}^1 & 0 & 0 & K_{33}^1 & K_{34}^1 \\ K_{41}^1 & K_{42}^1 & 0 & 0 & K_{43}^1 & K_{44}^1 \end{bmatrix} \begin{bmatrix} U_{1x} \\ U_{1y} \\ U_{2x} \\ U_{2y} \\ U_{3x} \\ U_{3y} \end{bmatrix}$$

圖 11　K 矩陣之填入方式

將各桿件的直接勁度法列出後，接下來要代入整個結構體進行計算，結構體的 K 矩陣為(計算構件之自由度數量*計算構件之自由度數量)大小的矩陣，此題的結構物總共有 6 個自由度，因此這個系統的 K 矩陣可擴充為 6*6 的大小。將每根 Truss 的 K 矩陣元素對應到個別的 U 矩陣裡各元素的右下角之編號，即可知道要將自身的 K 矩陣元素填寫到結構體大 K 矩陣之中的哪個位置，如圖 11。依序將每根 Truss 的 K 矩陣填寫進去大 K 矩陣，再將位置重疊的元素進行線性疊加，就會呈現出圖 12 的結果。接著再進行矩陣運算即可從計算出的 U 矩陣得知各節點的 X,Y 方向的位移量，最後把 U 矩陣代回 $\boldsymbol{f}=\boldsymbol{k}^e\boldsymbol{T}^T\boldsymbol{U}$，即可獲得每一根 Truss 所承受之內力。

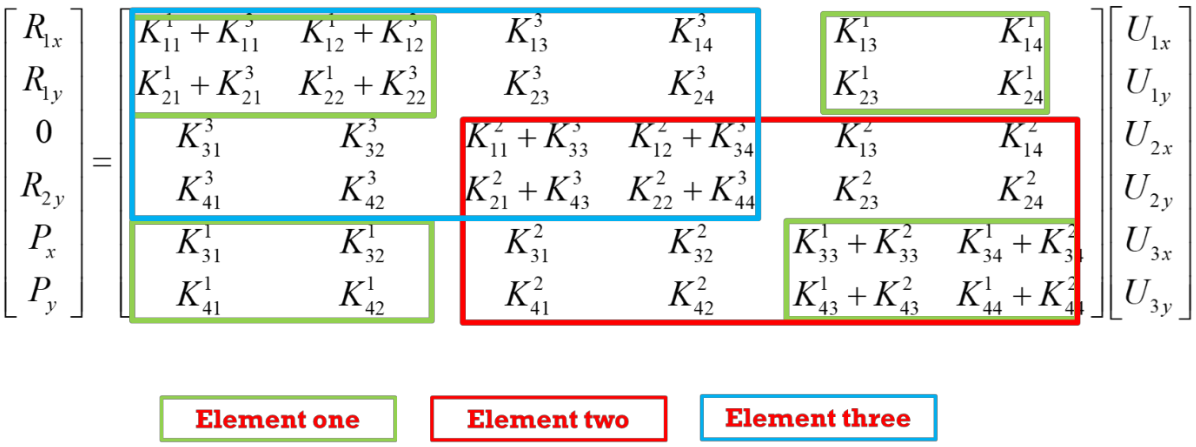


圖 12 *K* 矩陣之線性疊加

Beam(樑)在結構學的定義是設計用來傳遞軸向力、剪切力和彎矩的結構構件。Beam 可以承受垂直軸向的集中力(作用於一點的力)、分佈力(作用於較大面積的力)以及彎矩(垂直於橫截面之內力系的合力偶矩)三種施加負荷，它可以表現出軸向的形變、偏轉和旋轉現象，在一維的情況我們通常考慮外力與力矩施加在 Beam 上之節點所造成之剪切力與彎矩。Beam 在現實中的應用是承載垂直重力作用，但也可以承載水平方向作用力。它可將承受的荷載轉移至柱、牆或其他 Beam，後者再將荷載轉移至受壓構件將荷載卸除。由於樑的設計目的是為了可以承受垂直其軸向的作用力，所以在分析樑構件時主要是分析它的彎曲剛性(Bending rigidity)，彎曲剛性是用來描述一長條形物體，其彎曲力矩和樑形曲率的比例。彎曲剛度等於彈性模量 E 和梁截面關於特定軸的慣性矩 I 的乘積。換而言之，彎曲剛度就是 *EI*。

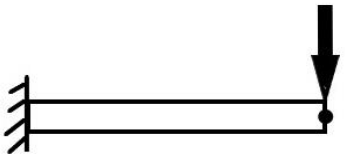


圖 13 集中力

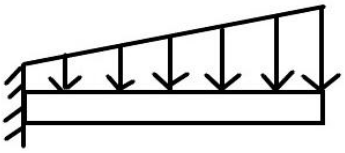


圖 14 分佈力



圖 15 彎矩

一維 Beam 和 Truss 的自由度差別在於，Truss 節點的自由度是 X 和 Y 方向可進行位移，而一維 Beam 之節點的自由度則是 Y 方向可進行位移與彎矩所造成的旋轉。在計算外力矩陣時，原本之 Truss(衍架)會帶入各點的外力，而 Beam 是帶入 Y 方向的外力與彎矩。Beam 和 Truss 一樣可適用直接勁度法，Beam 的單位剛度矩陣 ***K*** 之中的元素是由材料力學四階法所推得。

在分析樑時，我們的 APP 使用的是尤拉樑(Euler Beam)理論。尤拉樑係一種線彈性理論(linear elastic)的簡化，並用於計算樑之受力與變形的方法。線彈性理論為一種數學模型，簡單而言，我們說一個材料它的應力與應變是成比例的，那麼此材料稱為線彈性。四階法是尤拉樑假設下所呈現的特性，我們從樑的變形(位移)方程式，一步步地推導出角度、剪力與彎矩的方程式。尤拉理論告訴我們一根樑之任意位置的變形(位移)，與剪力和彎矩有積分與微分之關係。

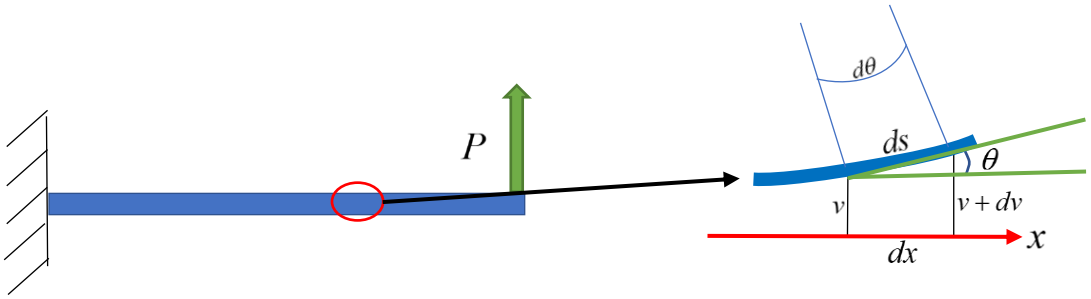


圖 16 樑之微小變形放大圖

欲了解位移變形量與彎矩之關係，我們先考慮如上圖。由圖 1 可看到，一根簡單的樑受到一外力 P 作用下，設其微小弧形變形 ds，微小垂直位移 dv，微小角度變化 $d\theta$，曲率半徑 ρ。由弧長公式我們知道

$$ds = \rho d\theta \tag{8}$$

曲率 κ 為曲率半徑 ρ 的倒數因此我們可以將曲率寫成

$$\kappa = \frac{1}{\rho} = \frac{d\theta}{ds} \tag{9}$$

變形曲線之斜率為 $\frac{dv}{dx}$，然而因為角度變化甚小，因此 $ds \approx dx$。我們可以將斜率寫成

$$\tan\theta = \frac{dv}{dx} \approx \frac{dv}{ds} = \sin\theta \approx \theta \tag{10}$$

而曲率 κ 可以改寫成

$$\kappa = \frac{d\theta}{ds} \approx \frac{d(\tan\theta)}{dx} = \frac{d}{dx}\left(\frac{dv}{dx}\right) = \frac{d^2v}{dx^2} \tag{11}$$

我們由材料力學知道，若一根樑滿足線彈性(linear elastic)和虎克定律(Hooke's law)，那麼其曲率(κ)為

$$\kappa = \frac{M}{EI} \tag{12}$$

M ：彎矩 E：楊氏模量 I：面積二次矩

使上面(11)與(12)式相等即得到彎矩和位移變形量之關係

$$\kappa = \frac{d^2v}{dx^2} = \frac{M}{EI} \tag{13}$$

$$M = EI\frac{d^2v}{dx^2} \tag{14}$$

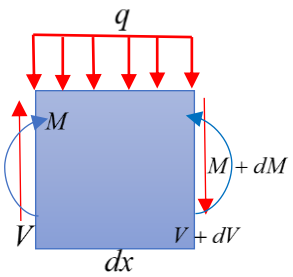


圖 17 解釋剪力微分後為什麼等於分布力

在上面我們推導如何從位移變形量得到彎矩。接下來推導彎矩和剪力還有剪力與分布力之關係。考慮一塊微小材料如圖 17。設微小長度dx，剪力為V彎矩為M，分布力則為q。此處定剪力向上為正，由力平衡我們得以下之平衡方程式

$$\sum F_{vert} = 0 \qquad V - qdx - (V + dV) = 0 \tag{15}$$

移項後，我們即可知道剪力與分布力之關係(正負號係因原先假設正負號方向所致)

$$\frac{dV}{dx} = -q \tag{16}$$

再來我們定彎矩在逆時針時為正。由彎矩平衡，我們可以得到下列式子

$$\sum M = 0 \quad -M - qdx\left(\frac{dx}{2}\right) - (V + dV)dx + M + dM = 0 \tag{17}$$

同樣在移項之後，我們就可以看出彎矩與剪力的關係

$$\frac{dM}{dx} = V \tag{18}$$

現在我們可以整理出整個尤拉樑所述之結論，尤拉樑之位移變形量與剪力和彎矩有下列關係

$$\begin{aligned}
\frac{dv}{dx} &= \theta(x) \\
EI\frac{d\theta}{dx} &= M(x) \\
\frac{dM}{dx} &= V(x) \\
\frac{dV}{dx} &= q(x)
\end{aligned} \tag{19}$$

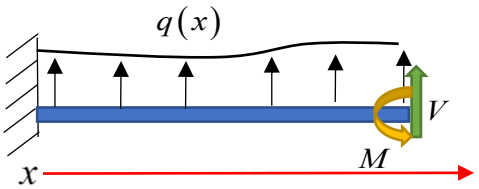


圖 18 樑受力模型

圖 18 是一根樑的模型，現在我們假設其位移變形量v，長度為L，角度變形量θ，剪力V與彎矩M，還有分布負載q。由尤拉樑我們可得知，位移變形量的一次微分是角度變形量，二次微分是彎矩，三次微分是剪力，四次微分則為分布負載。

$$\begin{aligned}
\frac{dv}{dx} &= \theta(\mathrm{x}) \\
EI\left.\frac{d^2v}{dx^2}\right|_{x=L} &= M \\
\left.\frac{d}{dx}\left(EI\frac{d^2v}{dx^2}\right)\right|_{x=L} &= -V \\
\frac{d^2}{dx^2}\left(EI\frac{d^2v}{dx^2}\right) &= q(x)
\end{aligned} \tag{20}$$

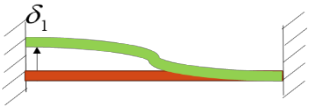
而要描述整根樑，我們還需要加上已知的邊界條件，分別為已知位移變形v_0、已知角度變形θ_0、外加剪力V_0和外加力矩M_0，v_0和θ_0為主要邊界條件(essential boundary condition)，V_0與M_0是自然邊界條件(natural boundary conditions)。主要邊界條件係解一個微分方程式所必需的條件，自然邊界條件則係在解完微分方程式後，自動滿足方程式的條件。現在假設此樑的方程式

$$
\begin{aligned}
q(\mathrm{x}) &= 0 \\
V(\mathrm{x}) &= c_1 \\
M(\mathrm{x}) &= c_1 x + c_2 \\
EI\theta(\mathrm{x}) &= \frac{c_1}{2}x^2 + c_2 x + c_3 \\
EIv(\mathrm{x}) &= \frac{c_1}{6}x^3 + \frac{c_2}{2}x^2 + c_3 x + c_4
\end{aligned}
\tag{21}
$$

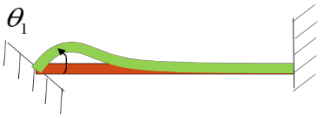


圖 19 樑端點之位移變形與角度變形

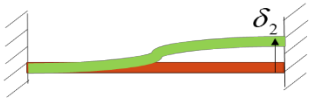
我們所需的主要邊界條件一共有四個，分別為前端之位移變形量v_I與角度變形量θ_I，與後端位移變形量v_J與角度變形量θ_J。假設一根樑上的每一個位置x之位移變形量係由這四個邊界條件所影響。經過整理，可寫成四條位移變形量方程式。第一條方程式我們假設v_I為 1，其餘邊界條件為 0。第二條則設θ_I為 1，其餘為 0。第三條設v_J為 1 其餘為 0，第四條設θ_J為 1 其餘為 0，$v_{p1}(x)$、$v_{p2}(x)$為分佈力所造成之位移變形，由於本程式尚未開發至施加分布力，故暫不解說。前四條推導後可得式(22)



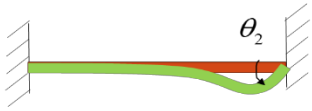
$$v_1(x) = v_I\left[1 - 3\left(\frac{x}{L}\right)^2 + 2\left(\frac{x}{L}\right)^3\right]$$



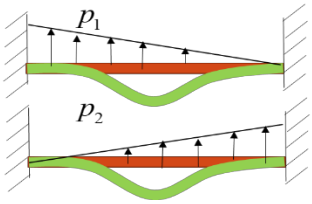
$$v_2(x) = L\theta_I\left[\left(\frac{x}{L}\right) - 2\left(\frac{x}{L}\right)^2 + \left(\frac{x}{L}\right)^3\right]$$



$$v_3(x) = v_J\left[3\left(\frac{x}{L}\right)^2 - 2\left(\frac{x}{L}\right)^3\right]$$



$$v_4(x) = L\theta_J\left[-\left(\frac{x}{L}\right)^2 + \left(\frac{x}{L}\right)^3\right] \tag{22}$$



$$v_{p1}(x) = p_1\frac{L^4}{120EI}\left[2\left(1-\frac{x}{L}\right)^2 - 3\left(1-\frac{x}{L}\right)^3 + \left(1-\frac{x}{L}\right)^5\right]$$

$$v_{p2}(x) = p_1\frac{L^4}{120EI}\left[2\left(\frac{x}{L}\right)^2 - 3\left(\frac{x}{L}\right)^3 + \left(\frac{x}{L}\right)^5\right]$$

我們將這四條方程式加總即為整根樑的位移變形方程式$v_T(x)$

$$v_T = v_1 + v_2 + v_3 + v_4 \tag{23}$$

由(16)式我們知道位移變形量與角度變形量之微分關係，因此我們將四條位移方程式作一次微分，即得到角度的變形方程式

$$\begin{aligned}
\theta_1(x) &= v_I\left[-6\left(\frac{x}{L^2}\right)+6\left(\frac{x^2}{L^3}\right)\right] \\
\theta_2(x) &= \theta_I\left[1-4\left(\frac{x}{L}\right)+3\left(\frac{x}{L}\right)^2\right] \\
\theta_3(x) &= v_J\left[6\left(\frac{x}{L^2}\right)-6\left(\frac{x^2}{L^3}\right)\right] \\
\theta_4(x) &= \theta_J\left[-2\left(\frac{x}{L}\right)+3\left(\frac{x}{L}\right)^2\right]
\end{aligned} \tag{24}$$

同樣地，我們把這四條方程式加總即得整根樑之角度變形方程式

$$\theta_T = \theta_1 + \theta_2 + \theta_3 + \theta_4 \tag{25}$$

將位移變形方程式作兩次微分可以得到力矩方程式，不過我們只要將上面四條角度變形方程式作一次微分即可求得力矩

$$\begin{aligned}
M_1(x) &= EIv_I\left[-6\left(\frac{1}{L^2}\right)+12\left(\frac{x}{L^3}\right)\right] \\
M_2(x) &= EI\theta_I\left[-4\left(\frac{1}{L}\right)+6\left(\frac{x}{L^2}\right)\right] \\
M_3(x) &= EIv_J\left[6\left(\frac{1}{L^2}\right)-12\left(\frac{x}{L^3}\right)\right] \\
M_4(x) &= EI\theta_J\left[-2\left(\frac{1}{L}\right)+6\left(\frac{x}{L^2}\right)\right]
\end{aligned} \tag{26}$$

同理，這四條方程式加總即為整根樑所表示的力矩方程式

$$M_T = M_1 + M_2 + M_3 + M_4 \tag{27}$$

最後，剪力方程式由位移變形方程式作三次微分求得，只要將力矩方程式作一次微分即可

$$\begin{aligned} V_1(x) &= EIv_I\left(\frac{12}{L^3}\right) \\ V_2(x) &= EI\theta_I\left(\frac{6}{L^2}\right) \\ V_3(x) &= EIv_J\left(\frac{-12}{L^3}\right) \\ V_4(x) &= EI\theta_J\left(\frac{6}{L^2}\right) \end{aligned} \tag{28}$$

這四條方程式加總即為整根樑上所表示的剪力方程式

$$V_T = V_1 + V_2 + V_3 + V_4 \tag{29}$$

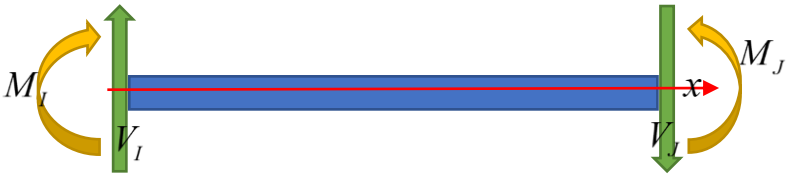


圖 20 樑剪力與彎矩之預設方向

現在我們可以解出 Beam(樑)之剛度矩陣中的元素。如圖 20 為一根樑，其前端點之剪力與彎矩為V_I、M_I，後端點之剪力與彎矩則為V_J、M_J。樑之前端與後端所受的彎矩與剪力可由(26)和(28)式求出。在前端點我們令x為零代入方程式，求得前端點之剪力為

$$\begin{aligned} &V_1(0) = EIv_I\left(\frac{12}{L^3}\right), V_2(0) = EI\theta_I\left(\frac{6}{L^2}\right) \\ &V_3(0) = EIv_J\left(\frac{-12}{L^3}\right), V_4(0) = EI\theta_J\left(\frac{6}{L^2}\right) \\ &\Rightarrow V_I = EI\left[v_I\left(\frac{12}{L^3}\right) + \theta_I\left(\frac{6}{L^2}\right) + v_J\left(\frac{-12}{L^3}\right) + \theta_J\left(\frac{6}{L^2}\right)\right] \end{aligned} \tag{30}$$

由於原本的彎矩公式在定義彎矩方向時係由內力觀念定出的，而此處彎矩所定義的方向係根據剪力與彎矩之作用位置而定義。兩者彎矩方向在前端之定義是相反的，因此要在方程式裡加上一負號

$$M_1(0)=EIv_I\left(\frac{6}{L^2}\right),M_2(0)=EI\theta_I\left(\frac{4}{L}\right)$$
$$M_3(0)=EIv_J\left(\frac{-6}{L^2}\right),M_4(0)=EI\theta_J\left(\frac{2}{L}\right) \tag{31}$$
$$\Rightarrow M_I=EI\left[v_I\left(\frac{6}{L^2}\right)+\theta_I\left(\frac{4}{L}\right)+v_J\left(\frac{-6}{L^2}\right)+\theta_J\left(\frac{2}{L}\right)\right]$$

而要求樑的後端點所受之剪力與彎矩，我們令x為L代入剪力與彎矩之方程式。由剪力方程式我們知道後端點的剪力為(此處之正負號同樣由於定義時方向是相反的，因此要在原本的方程式中加上一負號)

$$V_1(L)=EIv_I\left(\frac{-12}{L^3}\right),V_2(L)=EI\theta_I\left(\frac{-6}{L^2}\right)$$
$$V_3(L)=EIv_J\left(\frac{12}{L^3}\right),V_4(L)=EI\theta_J\left(\frac{-6}{L^2}\right) \tag{32}$$
$$\Rightarrow V_J=EI\left[v_I\left(\frac{-12}{L^3}\right)+\theta_I\left(\frac{-6}{L^2}\right)+v_J\left(\frac{12}{L^3}\right)+\theta_J\left(\frac{-6}{L^2}\right)\right]$$

我們一樣由彎矩方程式得到後端點之彎矩

$$M_1(L)=EIv_I\left[-6\left(\frac{1}{L^2}\right)+12\left(\frac{L}{L^3}\right)\right]=EIv_I\left(\frac{6}{L^2}\right)$$
$$M_2(L)=EI\theta_I\left[-4\left(\frac{1}{L}\right)+6\left(\frac{L}{L^2}\right)\right]=EI\theta_I\left(\frac{2}{L}\right)$$
$$M_3(L)=EIv_J\left[6\left(\frac{1}{L^2}\right)-12\left(\frac{L}{L^3}\right)\right]=EIv_J\left(-\frac{6}{L^2}\right) \tag{33}$$
$$M_4(L)=EI\theta_J\left[-2\left(\frac{1}{L}\right)+6\left(\frac{L}{L^2}\right)\right]=EI\theta_J\left(\frac{4}{L}\right)$$
$$\Rightarrow M_J=EI\left[v_I\left(\frac{6}{L^2}\right)+\theta_I\left(\frac{2}{L}\right)+v_J\left(-\frac{6}{L^2}\right)+\theta_J\left(\frac{4}{L}\right)\right]$$

我們將前後節點之剪力與彎矩整理出來

$$
\begin{aligned}
V_I &= EI\left[v_I\left(\frac{12}{L^3}\right)+\theta_I\left(\frac{6}{L^2}\right)+v_J\left(\frac{-12}{L^3}\right)+\theta_J\left(\frac{6}{L^2}\right)\right]\\
M_I &= EI\left[v_I\left(\frac{6}{L^2}\right)+\theta_I\left(\frac{4}{L}\right)+v_J\left(\frac{-6}{L^2}\right)+\theta_J\left(\frac{2}{L}\right)\right]\\
V_J &= EI\left[v_I\left(\frac{-12}{L^3}\right)+\theta_I\left(\frac{-6}{L^2}\right)+v_J\left(\frac{12}{L^3}\right)+\theta_J\left(\frac{-6}{L^2}\right)\right]\\
M_J &= EI\left[v_I\left(\frac{6}{L^2}\right)+\theta_I\left(\frac{2}{L}\right)+v_J\left(\frac{-6}{L^2}\right)+\theta_J\left(\frac{4}{L}\right)\right]
\end{aligned}
\tag{34}
$$

根據(34)這四條方程式我們可以將方程式寫成 Beam 的單位剛度矩陣(式 35)。我們比較這四條方程式與式(35)的矩陣，矩陣中的元素即是每條方程式的係數

$$
\begin{bmatrix} V_I \\ M_I \\ V_J \\ M_J \end{bmatrix} = EI \begin{bmatrix} \frac{12}{L^3} & \frac{6}{L^2} & \frac{-12}{L^3} & \frac{6}{L^2} \\ \frac{6}{L^2} & \frac{4}{L} & \frac{-6}{L^2} & \frac{2}{L} \\ \frac{-12}{L^3} & \frac{-6}{L^2} & \frac{12}{L^3} & \frac{-6}{L^2} \\ \frac{6}{L^2} & \frac{2}{L} & \frac{-6}{L^2} & \frac{4}{L} \end{bmatrix} \begin{bmatrix} v_I \\ \theta_I \\ v_J \\ \theta_J \end{bmatrix}
\tag{35}
$$

E：楊氏係數(Young's Modulus)，I：桿件截面面積二次矩，L：桿件長度

在寫出 Beam 之剛度矩陣後，我們同樣依照 Truss 的計算流程即可解出 Beam 起點之剪力 V_I、彎矩 M_I 和 Beam 終點之剪力 V_J、彎矩 M_J，即可繪製出剪力彎矩圖。剪力彎矩圖是對分析一個結構非常重要的一個項目。由這兩個圖，工程師可依圖分析其彎矩與剪力在某個位置的受力情況，那個位置是否合理或是有沒有安全疑慮諸如此類。因此在我們的 APP 中，我們依照求出來的方程式，建立剪力彎矩圖型還有順便將垂直位移與角度也放上。一眼即可看出其最大與最小之位置(未來會直接標出確切之距離)，方便使用者確認結果。

2.3 LU 矩陣分解

(1) LU 分解

LU 分解是將一個矩陣(A)拆解成上三角矩陣(U)與下三角矩陣(L)的乘積，可以被視為是高斯消去法的矩陣形式。

$$
\boldsymbol{A} = \boldsymbol{L}\cdot\boldsymbol{U} = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \cdot \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ u_{11}l_{21} & u_{12}l_{21}+u_{22} & u_{13}l_{21}+u_{23} \\ u_{11}l_{31} & u_{12}l_{31}+u_{22}l_{32} & u_{13}l_{31}+u_{23}l_{32}+u_{33} \end{bmatrix}
$$

$$\boldsymbol{A}=\begin{bmatrix} u_{11} & u_{12} & u_{13} \\ u_{11}l_{21} & u_{12}l_{21}+u_{22} & u_{13}l_{21}+u_{23} \\ u_{11}l_{31} & u_{12}l_{31}+u_{22}l_{32} & u_{13}l_{31}+u_{23}l_{32}+u_{33} \end{bmatrix}$$

觀察 L 矩陣和 U 矩陣中元素的規律推算出公式(36)：

$$\begin{aligned} & u_{1,j}=a_{1,j} \qquad j=1,2,...,n \\ & l_{i1}=\frac{a_{i1}}{u_{11}} \qquad i=2,...,n \\ & u_{kj}=a_{kj}-\sum_{p=1}^{k-1}l_{kp}u_{pj} \qquad j=k,...,n \\ & l_{ik}=\frac{a_{ik}-\sum_{p=1}^{k-1}l_{ip}u_{pk}}{u_{kk}} \qquad i=k+1,...,n \end{aligned} \tag{36}$$

透過通式(36)，我們就可以求出 L 矩陣 U 矩陣中每個元素的值。

$$\boldsymbol{U}=\begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} & u_{15} & u_{16} & u_{17} & u_{18} \\ & u_{22} & u_{23} & u_{24} & u_{25} & u_{26} & u_{27} & u_{28} \\ & & u_{33} & u_{34} & u_{35} & u_{36} & u_{37} & u_{38} \\ & & & u_{44} & u_{45} & u_{46} & u_{47} & u_{48} \\ & & & & u_{55} & u_{56} & u_{57} & u_{58} \\ & & 0 & & & u_{66} & u_{67} & u_{68} \\ & & & & & & u_{77} & u_{78} \\ & & & & & & & u_{88} \end{bmatrix} \qquad u_{kj}=a_{kj}-\sum_{p=1}^{k-1}l_{kp}u_{pj} \qquad j=k,...,n$$

$$\boldsymbol{L}=\begin{bmatrix} 1 & & & & & & & \\ l_{21} & 1 & & & & & & \\ l_{31} & l_{32} & 1 & & & 0 & & \\ l_{41} & l_{42} & l_{43} & 1 & & & & \\ l_{51} & l_{52} & l_{53} & l_{54} & 1 & & & \\ l_{61} & l_{62} & l_{63} & l_{64} & l_{65} & 1 & & \\ l_{71} & l_{72} & l_{73} & l_{74} & l_{75} & l_{76} & 1 & \\ l_{81} & l_{82} & l_{83} & l_{84} & l_{85} & l_{86} & l_{87} & 1 \end{bmatrix} \qquad l_{ik}=\frac{a_{ik}-\sum_{p=1}^{k-1}l_{ip}u_{pk}}{u_{kk}} \qquad i=k+1,...,n$$

接著進行實例演練，假設今天有一個矩陣 $A=\begin{bmatrix} 2 & 4 & 2 \\ 1 & 5 & 2 \\ 4 & -1 & 9 \end{bmatrix}$ 拆成 L 矩陣與 U 矩陣，以下

為拆解步驟：

$$A=\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}=\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}$$ (Step.1)

欲使方框內的第二與第三列為 0，則須將第一列乘以$(-\frac{1}{2})$加入第二列將第一列乘以 (–2) 後，加入第三列。同時，等式左端乘上相對應的矩陣。

$$\begin{bmatrix}1&0&0\\-\frac{1}{2}&1&0\\-2&0&1\end{bmatrix}\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}=\begin{bmatrix}2&4&2\\0&3&1\\0&-9&5\end{bmatrix}$$ (Step.2)

同上，欲使方框內的第 3 列為 0，則須將第二列乘以 3 後，加入第三列。

$$\begin{bmatrix}1&0&0\\0&1&0\\0&3&1\end{bmatrix}\begin{bmatrix}1&0&0\\-\frac{1}{2}&1&0\\-2&0&1\end{bmatrix}\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}=\begin{bmatrix}2&4&2\\0&3&1\\0&0&8\end{bmatrix}$$ (Step.3)

接著，依據 $AA^{-1}=I$ 的矩陣特性，將左方兩矩陣的反矩陣依序乘到右方的矩陣前(由於由於矩陣 ABC≠ACB 的特性，所以乘入的順序不能改變，如 Step.4 所示)

$$\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}=\begin{bmatrix}1&0&0\\-\frac{1}{2}&1&0\\-2&0&1\end{bmatrix}^{-1}\begin{bmatrix}1&0&0\\0&1&0\\0&3&1\end{bmatrix}^{-1}\begin{bmatrix}2&4&2\\0&3&1\\0&0&8\end{bmatrix}$$ (Step.4)

將(Step.4)中的矩陣整理後，可將原矩陣分寫成 L 和 U 矩陣，如**錯誤! 找不到參照來源。**：

$$\begin{bmatrix}2&4&2\\1&5&2\\4&-1&9\end{bmatrix}=\begin{bmatrix}1&0&0\\-\frac{1}{2}&1&0\\-2&-3&1\end{bmatrix}\begin{bmatrix}2&4&2\\0&3&1\\0&0&8\end{bmatrix}$$

□為 L 矩陣，□為 U 矩陣。

需要注意的事，欲拆解一矩陣 A 為 L 與 U 矩陣前，需先確認其行列式是否為零。

(2) 應用 1：求解 $A\bullet x=b$

適用於求解多個不同的 b 時。

$\begin{cases}A\bullet x=b\\A=L\bullet U\end{cases}\Rightarrow A\bullet x=L\bullet U\bullet x=b$，假設 z=U·x，即 $L\bullet z=b$

$$L\bullet z=b \Rightarrow \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \cdot \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \Rightarrow \begin{cases} z_1 = b_1 \\ z_i = b_i - \sum_{j=1}^{i-1} l_{ij} z_j \quad , \quad i = 1,2,3,...,n \end{cases} \tag{37}$$

上式 L·z=b 的矩陣關係式，由於 L 是一個下三角矩陣，可透由高斯法求得正確的 z 值，如上式所示。又 U·x=z，表示了 z 值與 x 值的關係，可藉由下面的通式將 x 值一步一步的求解出來

$$U\bullet x=z \Rightarrow \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \Rightarrow \begin{cases} x_i = \dfrac{z_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}} \quad , \quad i = n-1,...,3,2,1 \\ x_n = \dfrac{z_n}{u_{nn}} \end{cases} \tag{38}$$

(3) 應用 2：求反矩陣(Inverse)

假設 A 為一個矩陣，A^{-1} 為其反矩陣，I 則是單位矩陣。$A\bullet A^{-1} = A^{-1}\bullet A = L\bullet U\bullet A^{-1} = I$ 接者透過 L•Y = I 的性質反推出 $Y = L^{-1}$，再使用式(39)計算出 Y 矩陣中每個元素的值。最後使用式(40) $U\bullet A^{-1} = L^{-1} = Y$ 即可計算出 A 矩陣的反矩陣 A^{-1}。

$$\begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ y_{31} & y_{32} & y_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{cases} y_{11} = 1 \\ l_{21} y_{11} + y_{21} = 0 \Rightarrow y_{21} = -l_{21} \\ l_{31} y_{11} + l_{32} y_{21} + y_{31} = 0 \Rightarrow y_{31} = -l_{31} + l_{32} l_{21} \\ \vdots \\ \vdots \end{cases} \tag{39}$$

$$\begin{cases} L\bullet U\bullet A^{-1} = I \\ Y = L^{-1} \end{cases} \Rightarrow U\bullet A^{-1} = L^{-1} = I \Rightarrow \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix} \cdot \begin{bmatrix} \alpha_{11} & \alpha_{12} & \alpha_{13} \\ \alpha_{21} & \alpha_{22} & \alpha_{23} \\ \alpha_{31} & \alpha_{32} & \alpha_{33} \end{bmatrix} = \begin{bmatrix} y_{11} & y_{12} & y_{13} \\ y_{21} & y_{22} & y_{23} \\ y_{31} & y_{32} & y_{33} \end{bmatrix} \tag{40}$$

$$\Rightarrow u_{nn}\alpha_{ni} = y_{ni} \quad , \quad i = 1,2,...,n$$

第三章 JAVA 介面開發

3.1 自定義 View

Android Studio 裡有許多好用的預設物件可以使用，但是絕對不可能涵蓋到所有人的需求，我們想要製作的使用者繪圖功能就是例外。這時候的解決方法唯有建立自定義 View(CustomView，自定義視窗)這個選項。自定義 View 雖然製作起來較為麻煩，但是可開發空間十分龐大，基本上可以滿足每位開發者想要製作的效果。建立自定義 View 時通常會繼承 View(視窗)、ScrollView(滾動視窗)或其他的 View 物件。圖 21 為一個自定義 View 從建立到活動結束的生命週期。建立自定義 View 時要參考圖 21 的程式調用鍊，決定其他套件的建立時間點。

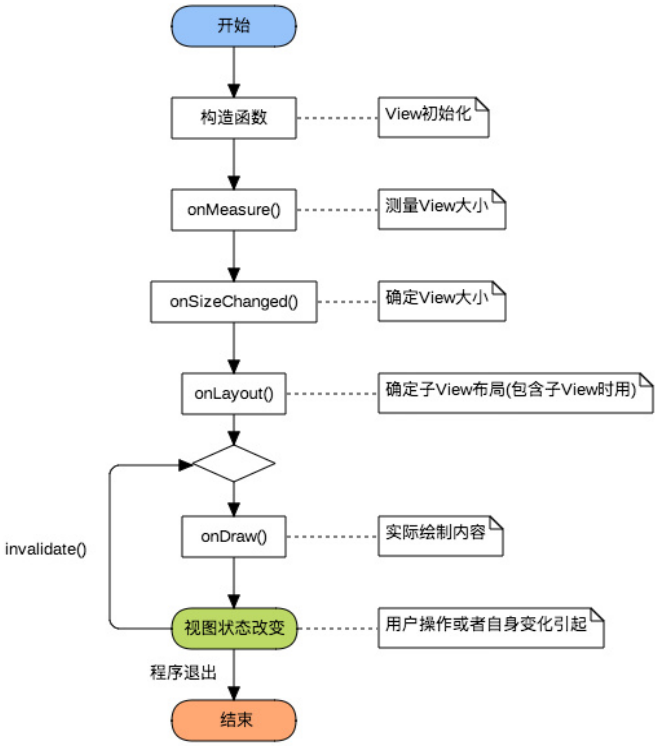


圖 21 自定義 View 的程式調用鍊

首先是構造函數，構造函數在 Java 中的作用是將其他地區的參數傳入欲建立的 class(定義類別)中，也就是 class 的入口，傳遞參數就好比客人帶著行李進入家門一樣，它可用於初始化部分內容，以及獲取自定義屬性。要使用構造函數時需要注意一點，方法必須和使用的 class 同名，否則無法使用。圖 22 即為設置構造函數的一種例子：

```
public class TrussPaintView{
    public TrussPaintView(Float x,Float y) {
    }
}
```

圖 22　構造函數範例

接著在別的 class 裡應用這個構造函數，並將要傳遞的參數放置在括號中，即可完成參數的傳遞。

由於自定義 View 繼承了 View，所以必須使用 View 內建的構造函數去構築自定義 View，View 的構造函數目前有 4 種，不過我們的程式只用了圖 24 中較常使用的這兩種。只有一個參數的構造函數一般在直接 New 一個 View 的時候會套用，有兩個參數的構造函數的調用機制是當開發者有在 layout 的 xml 中設定該 View 時調用，這個構造函數會讀取到關於他的所有屬性(包含自定義屬性)並將其包含在 attrs 中傳遞進來。

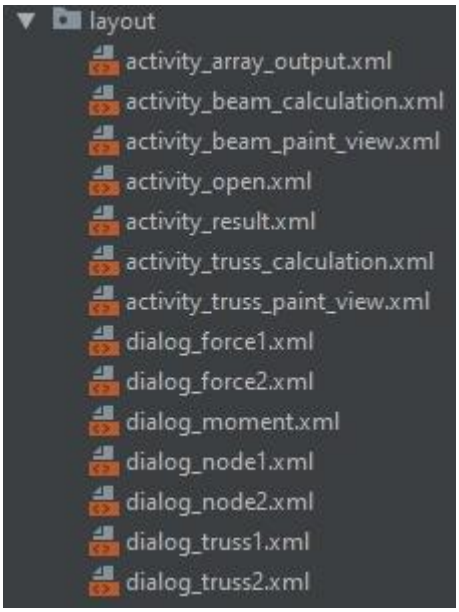


圖 23　layout 資料夾中放置的 xml 格局檔

```
//一個參數的構造函數
public TrussPaintView(Context context) {
    this(context, null);
}
//兩個參數的構造函數
public TrussPaintView(Context context, AttributeSet attrs) {
    super(context, attrs);
}
```

圖 24　常用的自定義 View 構造函數

圖 25 是調用一個參數的構造函數：

```
TrussPaintView trussPaintView = new TrussPaintView(this); //在 Activity 中
```

圖 25 調用設有單一參數構造函數的自定義 View

圖 26 是調用兩個參數的構造函數：

```
//在 layout 文件中，格式為：package 名稱.View 的名稱
<com.example.Eric_app.TrussPaintView
      android:id="@+id/TrussPaintView"
      android:layout_width="match_parent"
      android:layout_height="match_parent"/>
```

圖 26　調用設有兩個參數構造函數的自定義 View

自定義 View 構造函數的使用方式大概就是如此。另外，構造函數是 View 在建立的同時就存在了，也就是說，構造函數是自定義 View 中生命週期最長的部分。我們可以利用這一點將整個自定義 View 裡常使用到的物件在構造函數中就建立、設定屬性，這麼做會讓開發者在這個 class 中調用這些常用物件的時候相當方便。我們的做法是在構造函數中就建立畫筆，並設定好每一支畫筆的屬性，例如：筆寬、顏色、塗色樣式、抗鋸齒…etc.，其他在這時期建立的物件還有：Canvas(畫布)、Bitmap(位圖)、Canvas 的 Matrix，會在後面內容詳細解說。

在構造函數後的第二步驟是測量並確定 View 的大小，View 本身的大小不僅僅是由自身決定，也受到父控件的影響，為了適應各種狀況，例如：不同尺寸大小的手機螢幕，我們需要測量 View 的大小，也就是獲得長寬等等數據。onMeasure 是測量 View 的初始大小，onSizeChange 是當 View 的大小受到變更的時候，例如：旋轉螢幕、縮放平移時，確定新的 View 的大小，日後欲擴充的縮放功能就會使用到 onSizeChange 中的動態數據。在這一步驟中我們使用生命周期較長的變數去儲存 View 的長寬，後面會常常使用這兩個數據進行程式的設計。onLayout 方法是在建立 ViewGroup 布局的時候才會使用到，用來確定 ViewGrup 中子 View 的布局，本程式沒有使用到此功能故略過解說。最後就是 onDraw 方法，是自定義 View 裡的實際繪製內容，也就是我們實際關心的內容，我們使用的是 Canvas 實現讓使用者繪圖的功能，會在 3.3 繪圖方法中詳細說明運作機制。

3.2 OnTouchListener 監聽介面

當使用者碰觸畫面時，程式要如何偵測到這個碰觸事件呢？這時候就要將想處理碰觸事件的物件應用 onTouchListenr 監聽介面，只要是設定過 onTouchListener 的物件，都可以處理觸碰事件，**錯誤! 找不到參照來源。** 為 onTouchListener 可處理的單點觸碰事件，表格 2 為碰觸事件中可取得參數的方法。

表格 1 可處理之單點碰觸事件

事件	簡介
ACTION_DOWN	手指初次接觸到屏幕時觸發。
ACTION_MOVE	手指在屏幕上滑動時觸發，會多次觸發。
ACTION_UP	手指離開屏幕時觸發。
ACTION_CANCEL	事件被上層攔截時觸發。
ACTION_OUTSIDE	手指不在控件區域時觸發。

表格 2 在觸摸事件中可使用之方法

方法	簡介
getAction()	獲取事件類型。
getX()	獲得觸摸點在當前 View 的 X 軸座標。
getY()	獲得觸摸點在當前 View 的 Y 軸座標。
getRawX()	獲得觸摸點在整個屏幕的 X 軸座標。
getRawY()	獲得觸摸點在整個屏幕的 Y 軸座標。

在本程式中我們將繪圖的 View 設定為 onTouchListener 監聽器，偵測的碰觸事件只有設定 ACTION_DOWN。每當使用者點擊一次螢幕，就會執行一次設定的繪圖或選取事件，我們再利用這個運作頻率去設計我們想要執行的動作。假設現在 Actitivy 中有數個 View 並且部分重疊在一起，在這種情形下碰觸事件會如何處理？



圖 27 簡化後的物件責任鍊

圖 27 為簡化後的物件責任鍊，處理碰觸事件的時候，這個事件會依照圖 27 的順序進行分發，直到有單位將事件消費掉。如果任何 View 都沒消費掉事件，這個事件會從 View 開始反方向回傳，最終傳回給 Actitivy，如果最後 Activity 也沒有處理，本次事件才會被拋棄。我們在程式中使用 onTouchEvent 的相關方法 onTouch，這個方法會回傳一個 boolean 類型的返回值，透過返回 true 和 false 來控制事件的流程。判斷碰觸事件是否被消費是由返回的 boolean 值決定，如果 onTouch 方法的回傳值是 true，事件就會被消費並停止傳遞。現在回歸一開始的命題，當數個已設定可處理觸碰事件的 View 有重疊部分，使用者點擊重疊部分時，這些相同層級的 View 裡會由最後建立的，也就是最外層的 View 消費掉碰觸事件。這個機制可以用來處理本 APP 程式中答案數據視窗和繪圖介面同時存在產生的問題。

3.3 繪圖方法&建立數據庫

這個 APP 要繪製的物件有節點、桿件、外力、支承，還有樑的剪力彎矩圖，Android Studio 中能實現繪圖功能的工具套件有：Canvas、Bitmap、Paint、Path。為了使用上的方便，在整個繪圖 class 中常用的套件推薦在圖 21 中的構造函數階段就將其設定、建立完成，ex:Paint、Canvas、Bitmap。Canvas 和 Bitmap 是繪圖介面中的背景，這兩個套件的屬性設定時間點是在 onSizeChange 方法中完成，Canvas 的作用就是設定一塊畫布，它會儲存所有的繪圖結果，Bitmap 的功能是決定要顯示 Canvas 中的哪一部分和 Canvas 放置的位置，設定屬性的程式碼如下：

```
bitmap = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888);
canvas = new Canvas(bitmap);
```

圖 28　Canvas(畫布)和 Bitmap(位圖)的建立

Paint 的作用是設定畫筆，我們參考 GcsSloop 畫筆文章[8]裡面的介紹的方法設定畫筆的顏色、粗細、筆頭樣式...等諸多屬性，我們為每種物件和要在繪圖視窗輸出的文字都設定一個畫筆以達到差異化的效果，讓使用者能第一眼就認出哪個圖形代表哪種資料，以下為建構一支畫筆的程式碼：

```
//設定畫點的筆
nodepen = new Paint();
nodepen.setStyle(Paint.Style.FILL);
nodepen.setStrokeWidth(2);
nodepen.setColor(DOT_COLOR);
nodepen.setStrokeJoin(Paint.Join.ROUND);
nodepen.setStrokeCap(Paint.Cap.ROUND);
nodepen.setAntiAlias(true);
```

圖 29 設置畫筆之例子

在圖 21 裡的 ondraw()方法中繪製的圖形，會被設置為 Canvas 的預設背景，就算清除整個 Canvas，在 ondraw()方法中繪製的圖形仍然不會消失。如果想要讓繪圖介面一開始就顯示想要的圖形，就要將欲繪製之圖形的程式碼編寫在其中。在這個 APP 裡，我們想要在繪圖介面上顯示提供給使用者參考用的網格以及座標軸，因此必須將繪製參考網格線和座標軸的程式碼編寫在 ondraw()方法之中。

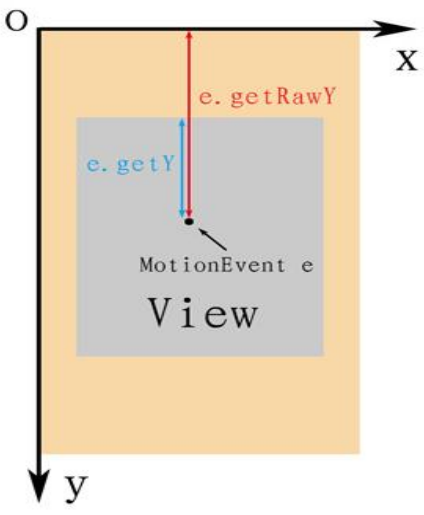


圖 30 手機螢幕預設座標系

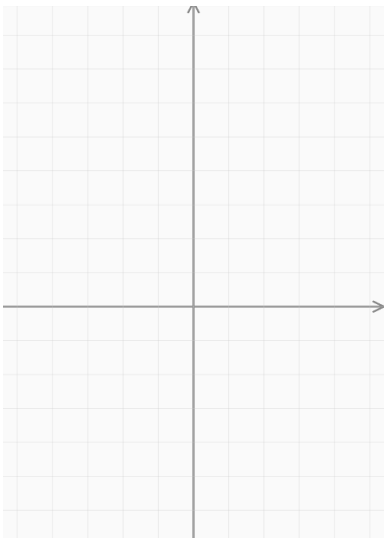


圖 31 繪圖介面座標系

第一步是設定 Canvas 的背景顏色，這部分是依照開發者的需求去設定就好。接下來，從圖 30 中我們知道了手機螢幕的預設座標系，我們希望繪圖介面預設的原點是在繪圖介面的中間，而且正向 X 方向為向右，正向 Y 方向為向上，圖 30 是設計完成的預設繪圖介面，那麼要如何將圖 300 的預設座標系轉換成圖 30 手機螢幕預設座標系圖 31 繪圖介面座標系 1 的座標系？在 Canvas 套件中有封裝對建立過的 canvas 進行座標轉換的方法，以下即為將設立過的 canvas 進行座標轉換的程式碼：

```
canvas.translate(fwidth / 2, fheight / 2);//將座標原點移動至 View 中央
canvas.scale(1, -1);//將座標上下反轉
```

圖 32 Canvas 座標移動

第一行是將先前獲得的長寬數據各除以 2 以後拿來做為座標原點的平移量，第二行是將預設座標系的 y 正值的方向反轉至向上。在這裡有一點需要特別注意，座標轉換的原理是矩陣乘法，故轉換的先後順序非常重要，若忽略這點會讓 Canvas 無法做到預期中的轉換座標效果。

座標轉換完成後，再寫入以新的座標系作為基準的繪製圖形程式碼。以下為繪製 xy 軸的方法，CanvasAidUtil 是使用 GcsSloop 編寫的輔助工具包，用來繪製介面的 x 軸和 y 軸，前兩個參數是從 x 座標中負值多少繪製直線到正值多少，後兩個參數是從 y 座標中負值多少繪製到正值多少。

```
//繪製 x,y 軸
private void drawTranslateCoordinateSpace(Canvas canvas) {
    canvas.save();
    CanvasAidUtils.set2DAxisLength(fwidth / 2, fwidth / 2,
                                   fheight / 2, fheight / 2);
    CanvasAidUtils.setLineColor(Color.GRAY);
    CanvasAidUtils.draw2DCoordinateSpace(canvas);
    CanvasAidUtils.draw2DCoordinateSpace(canvas);
    canvas.restore();
}
```

圖 33 繪製 xy 軸之副程式

方法中的 canvas.save()是將 canvas 先前的狀態儲存起來，canvas.restore()是將 canvas 經操作後的結果提取出來，在這兩行中間是畫布操作和圖形繪製，是 ondraw()方法裡經常使用的格式。網格線的繪製是使用 for 迴圈繪製而成，我們將 100 pixel 作為每一格之間的間隔，再用先前取得的長寬分別除以 100 得到 x 方向和 y 方向分別要繪製幾條直線，將其作為 for 迴圈的次數上限，接著輸入在 ondraw()方法中即可完成。

設定好畫布和筆之後，就可以開始繪製物件了。每種物件的繪圖方法的流程大致上都相同，步驟如下：(1)建立一個可讓 Activity 調用的 public 公共方法，在方法中將被當作繪圖介面的 View 設定 onTouchListener 監聽介面，setOnTouchListenr 的構造函數是直接 new 一個 OnTouchListener，Android Studio 會自動調用介面方法。MotionEvent 是偵測使用者的觸碰行為的類別，motionEvent 才是 onTouch 方法中會調用的物件。用 switch 邏輯判斷去對應 motionEvent 的變化，讓程式去執行相對應的動作，最後 return true 完成碰觸事件的消費。本程式只有使用到事件只有表格 1 的 ACTION_DOWN，碰觸螢幕後，使用表格 2 在觸摸事件中可使用之方法的 getX()和 getY()取得碰觸的 X,Y 座標，要注意此處獲得的座標是圖 30 手機螢幕預設座標系　　圖 31 繪圖介面座標系的預設座標系，作為範例的程式碼如下：

```
public void setDot() {
    this.setOnTouchListener(new OnTouchListener() {
        @Override
        public boolean onTouch(View view, MotionEvent motionEvent) {
            switch (motionEvent.getActionMasked()) {
                case MotionEvent.ACTION_DOWN:
                    downX = motionEvent.getX();
                    downY = motionEvent.getY();
                }
            return true;
        }
    });
}
```

圖 34　將 View 設定為 onTouchListener 監聽器之方法

(2)建立須獲得 X 座標和 Y 座標參數的副程式，副程式裡包裝的是繪圖方法和數據產生，在這裡編寫完成後，直接在 onTouchListener 裡面的 onTouch 方法中調用即可。這個做法的好處有可以先在第一步驟裡將預設座標轉換為畫布座標後再丟入副程式進行繪畫，減少出現問題的機會。範例如下：

```
public void DrawNode(float downX, float downY)
```

圖 35　繪製節點副程式

(3)在第二步驟的副程式中用程式進行繪圖和建立物件代表的數據，這些數據會在物件在畫面上創立的同時生成，加入數據庫提供給後台的計算程式進行結構分析。

(4)在第一步驟的 public 方法加入能實現特殊效果的邏輯判斷以及建構完成的繪圖副程式，最後將完成的 public 方法實作在 Activity 中 ActionBar 選單的相對應選單按鈕之中。

在繪圖副程式使用到的工具套件有：Path、Region、Paint、Collection、Matrix，接下來逐個介紹各個套件的功能。要繪製圖形的時候，除了 canvas 本身封裝的基本圖形和繪製文字以外，大多數是使用 Path(路徑)套件。Path 的使用概念是設定一個路徑、輪廓，套件中封裝的方法讓開發者可以進行畫線、畫圓、繪製貝賽塔曲線、縮放旋轉、複合數個 Path 路徑…等等諸多功能，得以進行更彈性化的繪製。我們參考 GcsSloop 編寫的 Path 操作文章[9]中介紹的方法來構思欲繪製的圖案。Region 是偵測碰觸行為是否在一個區域裡的受體，設定的方式為將已建立過的 Path 和整個 View 做交集而建立完成。我們透過 taurus770423 於 2015 年寫的文章[17]知道了 Collection(集合)套件的存在。並詳讀 Tsai,Wen Lung.的書[16]更深入的了解 Collection 各個套件的功能和使用方式。Collection 套件在本程式中是擔當數據庫的角色，我們使用其中的 Arraylist 和 Map 套件。Arraylist 和 map 的共同點是都可以動態增減，不需要去擔心是否達到上限，Arraylist 的排序依據的是物件加入的時間點，使用從 0 開始的 index 作排列，而 map 是對為加入其中的物件宣告的 key 值作為排序依據。以 node 為例，建立節點後將轉換後的座標存入 nodearray1，手機預設座標系的座標存入 oriNodearray，node 的圓形 path 存入 nodepatharray，node 的點擊區域 region 存入 noderegionarray。<>中的字元是指定要在 Arraylist 或 map 中放入的類型，如果放入的物件不符合該類別，除非進行強轉否則無法放入，

```
public static ArrayList<ArrayList> nodearray1 = new ArrayList<>();
public static ArrayList<ArrayList> oriNodearray = new ArrayList<>();
public static ArrayList<Path> nodepatharray = new ArrayList<>();
public static ArrayList<Region> noderegionarray = new ArrayList<>();
```

圖 36　建立 Arraylist 之範例

最後，Arraylist 和 map 都可以包裝成許多層，並且只要知道想要取得的物件是在哪個位置(index)或 key，也可以進行多重引用，如下：

```
Coordinate[i][0] = (float)TrussPaintView.nodearray1.get(checkn.get(i)).get(1);
```

圖 37　Arraylist 多重讀取參數

Matrix(矩陣)：Matrix 是一個矩陣，是一個一直在後台默默工作的勞動模範，我們所有看到的 View 背後都有 Matrix 的功勞。Matrix 的主要功能是座標映射和數值轉換，我們從 GcsSloop 的 Matrix 介紹文章[11] GcsSloop，Matrix 原理 和維基百科的矩陣乘法條目[18]維基百科，矩陣乘法了解 Matrix 的運作原理和。它的特色有：廣泛的作用範圍，在可以 View、圖片、動畫中都可以使用使用。由於 Matrix 是比 Canvas 操作更為底層的套件，因此它可以進行更靈活的運用。本身完整的封裝讓開發者能方便的使用這個套件。Matrix 的其餘可執行的動作可參考 GcsSloop 編寫的 Matrix 詳解文章[12]。在此 APP 裡，繪製外力和繪製 support 就使用了 Matrix 中的旋轉功能，不需要重新設計新的圖形，只要修改些許參數就能達到我們想要的結果，節約了大量的開發時間。

接下來一一介紹每個繪圖副程式的設計思路，由於繪製思路重複的概念很多，因此一個概念解說過一次後，後續會簡略帶過，只會介紹稍微不同或是需要注意的部分。

1. 節點(Node)：

在副程式中先建立要繪製節點的 Path，以及一個不塗色的透明 Path，這兩個物件的生命週期只有在副程式中，副程式結束時，這兩個 Path 也會結束。節點的繪製方式是先使用 nodepath.addCircle(圓心 x 座標,圓心 y 座標,半徑,Path.Direction.CCW)，再使用 canvas.drawPath(nodepath,nodepen)方法繪製到 canvas 上。另一個透明的 Path 是用來增加節點的可觸碰範圍，這個透明的 Path 同樣使用 addCircle 畫圓，和繪製節點的 Path 的差異是它的半徑是前者的兩倍，接著用 Region.setPath 將這個透明的 Path 和整個繪圖介面取交集並轉換成 Region 類別的物件，即為一個節點的可碰觸區域。繪製完成的有色 Path 會放入已建立的 nodepatharray 這個 Arraylist，之後選取物件的變色功能會讀取該 Arraylist 裡的物件進行變色的動作，而透明的觸碰 Region 會放入已建立的 noderegionarray 這個 Arraylist，是後續偵測使用者點擊哪個節點的數據來源。

我們這時候獲得的座標值是手機預設的座標值，但是繪製圖形的時候是以 canvas 經過座標轉換後的座標進行繪圖。這樣會產生觸碰座標和繪製圖形對不上的問題，實際發生的情形就像是，點擊手機預設座標的[0,0]，繪製圖形方法會會在 Canvas 座標中的[0,0]，也就是在畫面的正中心。我們的解決方案是使用 Matrix 進行即時座標轉換。利用 Matrix 解決觸碰區域和繪製圖形不吻合的方法為，取得 Canvas 經平移縮放後的 Matrix，再使用 inverse 方法取得 Canvas 的反矩陣，這樣操作是將全局座標系使用當前 Canvas 的反矩陣轉化為畫布座標系，再使用 Matrix 封裝的 mapPoint 方法將觸摸的位置轉換成畫布座標系即可。除了繪製節點使用 Matrix 做碰觸位置修正外，此 APP 裡繪製外力和支承時就使用了 Matrix 中的旋轉功能，讓圖形只要改變些許參數就能達到我們想要的結果，節約了大量的開發時間。

由於節點繪製方法是每當畫面被點擊一次就執行一次，我們利用這個週期去儲存數據。這是關於這個 APP 建立數據庫方面的核心思路，所有的繪製副程式都會使用到。當使用者點擊繪圖介面時，onTouchListener 會傳出一組使用手機預設座標系的碰觸點 xy 座標，接著利用在 onSizeChange 方法中獲得的 View 長寬數據進行座標轉換，將碰觸點 x 座標加上 Canvas 的 x 方向平移量(fwidth/2)，以及將碰觸點 y 座標加上 Canvas 的 y 方向平移量(fheight/2)再乘上-1(因為上下反轉)，這樣即可將獲得的碰觸點 xy 數據轉換成使用 Canvas 座標的 x 座標和 y 座標。

```
//座標轉換後除以 100 將單位轉換成公尺
DotX = (downX - fwidth / 2)/100;
DotY = (-downY + fheight / 2)/100;
```

圖 38　碰觸座標之座標轉換

接下來在副程式中建立兩個儲存類別為 float 的 Arraylist，將轉換前的 xy 座標和轉換後的 xy 座標分別存進這兩個不同的 Arraylist 裡，最後將儲存轉換前座標的 Arraylist 放入先前建立的名為 orinodearray 這個 Arraylist 之中，後續其他物件的繪圖方法會從此

Arraylist 中取出繪圖用的座標。儲存轉換後座標的 Arraylist 會放入已建立過的名為 nodearray1 這個 Arraylist 裡，這個 Arraylist 是結構分析計算程式的數據來源。除此之外，我們想要將節點賦予額外兩種屬性，分別是自由度和外力，未設置限定條件的節點預設的自由度為[1,1,1]，矩陣中這三個數值代表：[x 方向是否可移動，y 方向是否可移動，是否可施加力矩]，將這個矩陣宣告和節點編號同名的 key 以後放入儲存限定條件的名位 supportmap 的 map 裡面。未設置外力和力矩的外力屬性為[0,0,0]，這三個數值分別代表：[受多少 x 方向的外力，受多少 y 方向的外力，和 x 軸的夾角]，將這個矩陣宣告和節點編號同名的 key 以後放入儲存外力資料的名為 forcemap 的 map 之中。

此外，我們還想將節點的編號繪製在 canvas 上面，繪製方式是先在構造函數中建立用來繪製文字的 Paint 畫筆物件，設定好要繪製的文字大小和顏色以及其他屬性。接著使用 canvas.drawtext(要輸出的字串,x 座標,y 座標,繪製文字的 Paint)，上面的方法是在畫布上建立一個文字方塊，在文字方塊中繪製要輸出的文字。x 座標可以是文字方塊的最左邊、中點、最右邊，端看繪製畫筆設定的屬性。y 座標是文字方塊的底線，知道方法中 xy 座標參數代表的意義後就可以依照開發者的需求決定文字要放置的位置。在這裡的 drawtext 只會顯示數字，但是隨著建立的節點數目增加，位數的增大會使顯示的數字和物件之間的距離被拉開，故增加使用 if…else 邏輯判斷讓每當節點編號的位數加一位，就把文字方塊的 x 座標向左移動一點的修正方法，讓數字和物件的距離不會太大而讓使用者混淆。

2. 桿件(Truss & Beam)：

由於桿件這種物件是有起點和終點的，因此建立繪製 Truss 或 Beam 的副程式需要獲得起點的 xy 座標和終點的 xy 座標

```
public void DrawTruss(float startX,float startY,float endX,float endY)
```

圖 39 繪製桿件之副程式

和節點相同，建立一個顯示用的 Path 和另一個用來建立可偵測 Region 的 Path，繪製顯示用 Path 的方法是先使用 Path.moveTo(x,y)移動到桿件的起點，再使用 Path.lineTo(x,y)連結到終點，最後將此直線的 Path 使用 canvas.drawPath 方法繪製到畫布上即可完成。建構透明 Path 的方法是以繪製的直線為中心軸，將直線分割成 2000 個正方形，利用 for 迴圈建立許許多多的正方形 Path，最後將其統合成一個 Path，即可用來創造經擴張的偵測觸碰 Region。最後再將有色的 Path 放進 trusspatharray，而無數正方形組成的統合 Region 則會放入 trussregionarray 之中。桿件編號使用的繪製方法也和節點相同，文字方塊位置是在桿件直線的中點上，再依照桿件的斜率進行適當的偏移，盡量避免了編號數字和桿件重疊在一起而造成混淆的狀況。

3. 外力(Force)：

在這個 APP 中，為自由體圖施加外力的方式是點擊欲施加外力的節點，因此外力繪製副程式是以使用者點擊的節點座標為原點進行繪圖。我們在這個副程式裡新增了 3 個

Path，1 個 Region 和 1 個 Matrix 共 5 個物件，這三個 Path 之中，一個用來繪製箭頭的三角形部分，一個用來繪製箭頭的長方形部分，最後一個 Path 是將這兩個 Path 統合起來的容器，將前兩個 Path 加入其中，結合起來的圖形為箭頭的三角形尖端黏著節點，箭頭指向 x 軸正方向的圖形，最終圖形會放入儲存外力圖形的 forcepatharray。而 Region 的建立是將箭頭 Path 和整個 View 取交集而設定完成，在將其放入 forceregionarray 之中。在經過和教授、學長的討論後，我們將箭頭的預設方向從向右更改成向下，但是我們不想再重新設計一個指向下方的新箭頭圖形，因此使用了 Matrix 套件封裝的旋轉方法旋轉原本設計的箭頭。這個做法不僅省下了重打程式碼的過程，而且可以隨著使用者更改外力屬性進行動態旋轉。使用方法是先建立一個新 Matrix 物件，先使用 preScale(1,-1,x,y)將 Matrix 的座標方向上下反轉成和 Canvas 相同，接著使用 preRotate(forceangle,x,y)讓 Matrix 的效果變成以節點的座標為中心旋轉 forceangle 角度(預設為-90 度)，最後，將箭頭圖形的 Path 使用 transform(Matrix)方法套用 Matrix 的轉換效果，即可完成圖形旋轉的工作。

設定外力的屬性的作法，也是在副程式中先建立生命週期短的 Arraylist，將想要設定的數值丟入其中後再放入儲存外力屬性的 forcemap 之中。該 Arraylist 的參數分別代表為{施加在 x 方向的力(fx,單位 kN)，施加在 y 方向的力(fy,單位 kN)，和 x 軸之間的夾角(forceangle,單位為度)}，fx 預設值為 0，fy 預設值為-10，forceangle 預設為-90 度。由於先前點擊節點的時候取得和節點編號相同的目錄值(index)，我們將此 index 宣告為 Arraylist 的 key 值，並使用 forcemap.put(String.valueOf(index),arraylist)的方法將原本和該 index 同名的 key 攜帶的 Arraylist{0,0,0}覆蓋掉，外力的屬性設定就大功告成了。

4. 力矩(Moment)：

繪製 Moment 的副程式基本上和 Force 大同小異，也是將力矩圖形分成兩個 Path 繪製，最後整合成一個 Path 並利用它建立可點擊之 Region，接著把預設屬性丟入 Arraylist，再使用 put 方法覆寫在 momentmap 之中。在這裡值得一提的是，我們想要把代表 moment 的圖形設計為以節點為圓心的弧形箭頭，那弧形是如何繪製的？Path 繪製弧形的原理是先建立一個矩形，以矩形中的內切圓作為繪製依據，再決定要繪製多少角度的弧形。

```
RectF rectF= new RectF(x-50,y-50,x+50,y+50);
momentpath2.arcTo(rectF,0,-90,false);
```

圖 40　繪製圓弧之方法

arcTo()方法裡各參數分別代表的意義為，(要參考的矩形，起始繪製角度，繪製結束角度，是否要將原畫筆的最後一點和圓弧起點連接)。將兩個 Path 設計完成後，我們改成先把這兩個 Path 繪製在 Canvas 上，繪製完再將這兩個 Path 合而為一，如果不這麼做，後續選取變色的時候會產生變色圖形無法完全覆蓋的問題。圖 41 為繪製力矩圖形的畫筆屬性。

```
//設定力矩畫筆
momentpen = new Paint();
momentpen.setStyle(Paint.Style.STROKE);
momentpen.setStrokeWidth(15);
momentpen.setColor(FORCE_COLOR);
momentpen.setTextSize(36);
momentpen.setStrokeJoin(Paint.Join.ROUND);
momentpen.setStrokeCap(Paint.Cap.BUTT);
momentpen.setAntiAlias(true);
```

圖 41　繪製力矩畫筆之屬性

5. 支承(Support)：

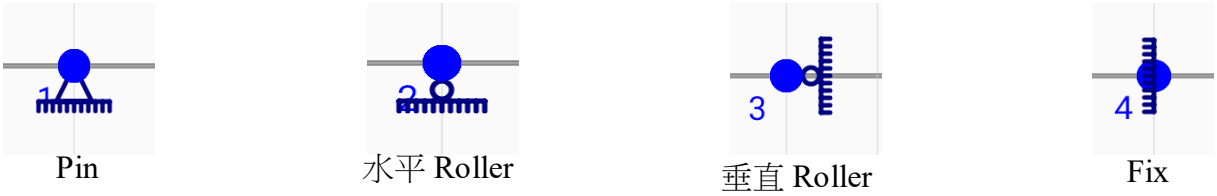


圖 42 支承示意圖

支承的作用是為節點設置限制條件，在這個 APP 中繪製的支承有四種，分別是 Pin(屬性{0,0,1}、和 x 軸平行的 Roller(屬性{1,0,1})、和 x 軸垂直的 Roller(屬性{0,1,1})，Fix 這(屬性{0,0,0})四種，如圖 42 支承示意圖。這四個物件分別代表著不同的限制條件，建立 Support 的手段也是點擊已繪製過的節點，將被點擊節點的座標作為繪製參數，繪製方法也是把圖形分成兩個 Path，分別是支承主體和支承底部黏接區域，再將兩者合而為一，而和 x 軸平行的 Roller 和與 x 軸垂直的 Roller 的差別是有無使用 Matrix 做旋轉，最後將各支承代表的屬性複寫到 supportmap 之中。

6. 剪力、彎矩、變形、扭轉角圖表：

我們希望使用者按下 Beam 的開始計算後，可以看到剪力、彎矩、扭轉角、變形隨著桿件長度變化的折線圖，但是原來的繪圖介面已經沒有空間了，所以新增一個視窗用來放入這四張折線圖，並且在原繪圖視窗中新增一按鈕連結到圖表視窗。每一個圖表都是一個自定義 View，繪圖方法都是寫在 ondraw()之中，讓使用者切換到此視窗後不必進行多餘操作就可以看到結果。

各個圖表的基本架構都是相同的，不同點只有用來繪製的函數不同而已。一樣參考圖 1 的程式調用鍊進行建構，先在構造函數建立用來繪製函數圖形的畫筆和繪製數字的畫筆，接著在 onSizeChange 中建立 Bitmap 和 Canvas，再取得該圖表自定義 View 的長與寬進行後續的 canvas 座標平移。在 ondraw 中設定 canvas 座標偏移量的程式碼如下：

```
canvas.translate(30,fheight/2);
canvas.scale(1,-1);
```

圖 43　繪製折線圖之 View 的座標偏移量

接著使用牛頓逼近法取得函數的極值，將所有極值的絕對值最大者設為圖表 y 座標的上下限，函數 f(x)值轉換成圖表 y 座標的公式為：(於目前 x 座標的 f(x)值/極值的絕對值)*(fwidth/2 - 50)。至於 x 座標則是將底線設置為(fwidth-100)，Beam 長度轉換成 x 座標的長度公式為：(目前 Beam 的長度/所有 Beam 的總長)*(fwidth-100)。這樣設計的話，函數折線圖就不會因數值超出 View 的長(fwidth)和寬(height)而顯示不完全。接著已繪製的 Beam 長度除以 2000，將其設定為函數的 dx 單位變化量，再使用 for 迴圈把 dx 代入函數後連結出函數圖形。繪製成品如下：

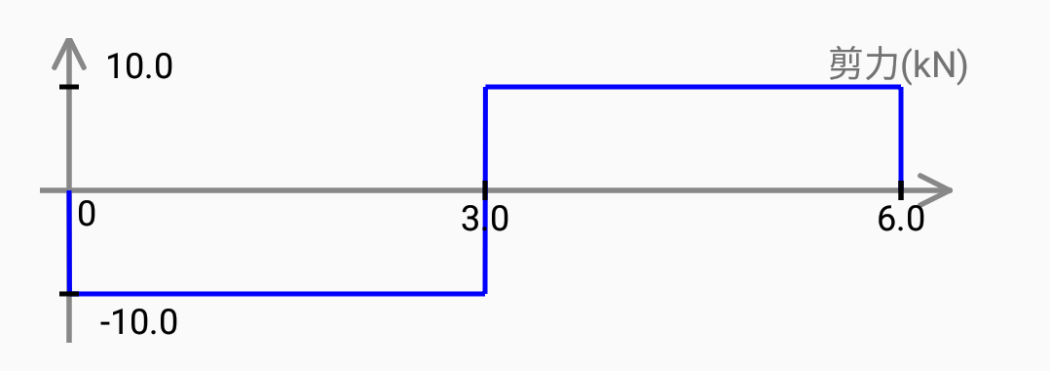


圖 44 剪力折線圖

3.4 鎖點

我們一開始設計出來的版本在建立節點的時候是沒有鎖定在參考網格線或座標軸上面的，節點的座標數值相當難看，使用者也不可能剛好點擊在整數上，蒐集這些使用者的使用心得後，我們決定加入鎖點功能。鎖定方式是在設置節點的副程式之中會得到使用者觸碰的 x,y 值，將取得的 x,y 座標值轉換成畫布座標後進行座標修正，再將修正後的座標丟入繪圖副程式之中就完成了。座標修正的思路以 x 座標為例子，圖 45 為 x 座標修正之程式碼：

```
if (downX - fwidth / 2 >= 0) {
    if (Math.abs(downX - fwidth / 2) % 100 <= 50) {
        downX = downX - ((downX - fwidth / 2) % 100);
    } else if (Math.abs(downX - fwidth / 2) % 100 > 50) {
        downX = downX + 100 - ((downX - fwidth / 2) % 100);
    }
} else if (downX - fwidth / 2 < 0) {
    if (Math.abs(downX-fwidth/2) % 100 > 50) {
        downX = downX - 100 + (Math.abs((downX - fwidth / 2) % 100));
    } else if (Math.abs(downX-fwidth/2) % 100 <= 50) {
```

```
            downX = downX +(Math.abs((downX - fwidth / 2) % 100));
        }
}
```

圖 45　x 方向之鎖點判斷

當轉換後的 x 座標大於 0，依照轉換後的 x 座標/100 後的餘數來判斷節點要鎖定在網格的左邊邊界還是右邊邊界，如果餘數<=50，修正方式為：觸碰的 x 座標值-(轉換後座標除以 100 的餘數)，這樣節點會鎖定在網格的左邊界上。如果餘數>50，修正方式為：觸碰的 x 座標值-(轉換後的座標除以 100 的餘數)+100，這樣節點就會鎖定在網格的右邊界上。Y 座標的修正邏輯也大同小異，圖 46 為 y 座標修正之程式碼：

```
if ((-downY + fheight / 2) >= 0) {
        if (Math.abs(downY - fheight / 2) % 100 <= 50) {
            downY = downY + (Math.abs(downY - fheight / 2) % 100);
        } else if (Math.abs(downY - fheight / 2) % 100 > 50) {
            downY = downY - (100-Math.abs(downY - fheight / 2) % 100);
        }
} else if ((-downY + fheight / 2) < 0) {
        if (Math.abs(downY - fheight / 2) % 100 > 50) {
            downY = downY + 100 - (Math.abs(downY - fheight / 2) % 100);
        } else if (Math.abs(downY - fheight / 2) % 100 <= 50) {
            downY = downY - (Math.abs(downY - fheight / 2) % 100);
        }
}
```

圖 46　y 方向之鎖點判斷

最後，為了避免使用者連續點擊造成節點重疊，我們設計了圖 47 中的防重複邏輯：

```
public boolean searchnode(float x,float y) {
        boolean s = true;
        for (int i = 0; i < oriNodearray.size(); i++) {
            float sx = (float) oriNodearray.get(i).get(0);
            float sy = (float) oriNodearray.get(i).get(1);
            if (x == sx && y == sy) {
                s = false;
                break;
            } else {
                s = true;
            }
        }
```

```
    return s;
}
```

圖 47 防重複建立節點之邏輯

這個副程式會搜尋已建立的節點座標數據庫中每一筆數據，如果修正後的座標在數據庫已經有相同的數據，這個副程式會回傳 false，節點就不會繪製，其他狀況會回傳 true，執行繪製節點副程式。

3.5 偵測 & 選擇物件之邏輯

除了節點以外，Truss&Beam、Force、Moment、Support 這幾種物件都是選取畫面上已繪製的節點為參數來建立的。那麼物件的選取邏輯是如何運作的？繪製節點後，我們手邊有儲存節點 path 的 Arraylist 和儲存節點 region 的 Arraylist 數據庫。在 3.3 繪圖功能開頭得知 Arraylist 會為裡面的物件給予 index 作為排序，所以建立選取邏輯的思路就是獲得選取物件的 index 為目的進行設計，圖 48 為取得節點目錄值的副程式碼：

```
//搜尋 DotRegion 目錄
boolean nodesearch = false;
int FindNodeRegionIndex(int inputX, int inputY) {
    int ans = 0;
    for (int i = 0; i < noderegionarray.size(); i++) {
        nodesearch = noderegionarray.get(i).contains(inputX, inputY);
        if (nodesearch == true) {
            ans = noderegionarray.indexOf(noderegionarray.get(i));
            break;
        }
    }
    nodesearch = false;
    return ans;
}
```

圖 48 搜尋物件目錄值的邏輯

該邏輯的設計思路為：先建立一個偵測用的 boolean 值，接著建立一個會回傳 int 整數的副程式 FindNodeRegionIndex，此副程式中取得的參數 inputX 和 inputY 會是使用者碰觸螢幕傳出的 x,y 座標，下一步使用 for 迴圈去搜尋 noderegionarray 中的每一個物件，Region 裡封裝的 contains 方法是判斷觸碰點座標是否在該物件的碰觸區域，它會回傳一個 boolean 值，而前面建立的 nodesearch 會接收它，接著使用 if 去判斷當 nodesearch 為 true 的時候，使用 Arraylist 封裝的 indexOf 去取得該物件所在的 index 值，將這個 index 值放置在 ans 變數之中將其回傳並跳出迴圈，再把 nodesearch 回歸為 false。這整個流程完成後就會讓使用者點擊物件後取得物件的目錄值，後續會經常使用這部分回傳的目錄值進行從數據庫中讀取物件的動作。

參考圖 49，當使用者選擇到該物件後，畫面需要同步作出更動給予使用者回饋，讓使用者知道他選取到物件，我們做出的效果是當使用者選取到物件，該物件變為亮綠色，如果重複點擊或點擊其他區域，回復原本的顏色。使用 FindNodeRegionIndex 取得目錄值後，用 nodeindex 這個變數將副程式回傳的整數儲存起來，圖 49 中 if 判斷式中的內容代表當碰觸點座標在該 index 的物件區域的時候，例如：碰觸點在第三個節點的區域。這時候將繪製節點的畫筆設置為綠色，設置一個 Path 代表該 index 的物件路徑，例如：第三個節點的圖形，再拿變成綠色的筆繪製 Node_p 路徑到 canvas 上，即可完成變色之動作。

```
nodeindex = FindNodeRegionIndex(x, y);
if (noderegionarray.get(nodeindex).contains(x, y)) {
        nodepen.setColor(SELECT_COLOR);
        Node_p = nodepatharray.get(nodeindex);
        canvas.drawPath(Node_p, nodepen);
}
```

圖 49　選取物件使其變色的方法

接著，以建立外力的方法做為另外一個例子。使用者碰觸節點後，取得 nodeindex，接著利用該 nodeindex 從儲存繪製用座標的 Arraylist – orinodearray 之中取得繪製用 x,y 座標值，要讀取這組 xy 座標需要用另一個容器去儲存，我們建立一個名為 oriForce 的 Arraylist 的容器用來儲存取得的繪製用 xy 座標，再將此容器中儲存的 xy 座標丟入繪製外力的副程式。在此時，給建立外力方法專用的計數器 forcecount 加一，我們在這裡設置一個邏輯判斷，當 forcecount 等於 1 或者使用者沒點到節點的時候，forcecount 歸零，清除 oriForce 容器。設立 forcecount 的主要目的是為了判斷清除容器的時機，防止垃圾數據過於龐大還有防止後台計算程式讀取到錯誤數據的問題。

最後要提到的是，關於桿件的建立機制，由於桿件需要使用者選擇兩個節點以後才可以建立，但是使用者可能會有重複建立桿件的情形發生。為了防止這種情況我們設置了一個防呆邏輯，當使用者點擊第一個節點，計數器 trusscount 加一，此時用一個變數 a 將點擊第一點的 index 儲存起來，接著點擊第二個節點的時候，trusscount 再加一，此時用另一個變數 b 將點擊第二點的 index 儲存起來，將 a 和 b 放入一個在點擊第二個節點的時候才建立的名為 truss 的 Arraylist 容器之中，再將 truss 放入最終儲存桿件數據的名為 element 的 Arraylist 裡作為後續數據計算的數據來源之一，最後將 trusscount 歸零。當使用者點擊畫面上節點以外的區域、點擊相同的點、重複選取相同的節點例如：建立過連接 1,2 點的桿件，再點擊第 2 個節點和第 1 個節點，發生上述三種狀況時，都會取消建立桿件。利用的原理是偵測 a 和 b 是否同時和過往數據相同而決定是否建立該桿件。

3.6 清除畫面&重繪

Canvas 在使用上有個要注意的地方，就是繪製在單一圖層上的圖形是不可逆的，簡單的說，繪製過的圖形會一直留在 Canvas 上面，要清除畫面的方法有兩個，建立透明

的圖形覆蓋在舊的圖形上面，或者是使用 Canvas 封裝好的方法清空整個 Canvas，前者過於瑣碎而且可能會有覆蓋不完全或是覆蓋太多的問題，所以我們選擇後者為清除畫面的方式。在使用者更改過數據後，要如何進行重新繪製的動作？我們的作法是將之前繪製過的物件的路徑和碰觸區域先儲存進 Arraylist 動態陣列中，接下來的步驟是先把畫布全部清空，將更改數據後繪製的圖形一一覆蓋掉舊的資料，最後再使用 for 迴圈讀取更新後的圖形資料 Arraylist 繪製到 Canvas 上面，這樣就可以讓使用者更改完數據後，讓畫面同步進行更新。

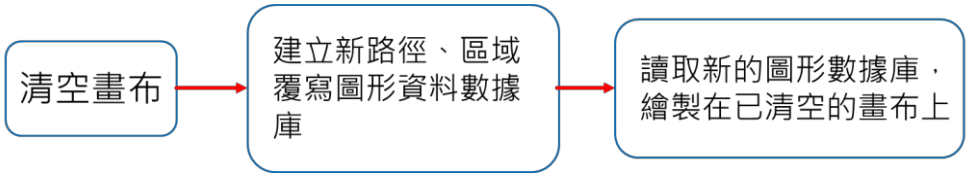


圖 50 重新繪製流程

3.7 其他介面物件

介面物件為使用者能在使用者介面上能看到的元素，在此 APP 中能看到的介面物件主要有：工具列(Action bar)、彈出視窗(Dialog)、滾動視窗(Scroll View)和繪圖視窗，此部分為解說除了繪圖視窗以外的介面物件。

(1) 工具列 (ActionBar)

在介面的上方的物件為工具列(ActionBar)，使用者需透過選擇工具列的彈出選單才能建立要進行結構分析的物件，每個按鈕皆代表一種功能，分別有節點 (Node)、桿件種類 (element)、外力(force)、顯示資料、清除資料，還有開始計算等等，需要點擊這些圖示才能夠執行此項功能。ex: 選擇建立點後，才能繪出節點。

設計思路：

Android 工具列(Action Bar)是透過在欲顯示之 Actitivy 中 onCreate 時期就讀取 menu 資料夾裡的 xml 格局檔來建立，之後建立 onOptionsItemsSelected 介面並應用 switch 邏輯判斷來進行點擊相對應按鈕完成相對應行動的動作。在設計初期時，雖然能執行已選取的行為，但在切換不同功能的時候，之前建立的結果會留存下來。解決此問題的方式為：switch 中的每個 case 裡除了欲執行的建立行動方法以外，還需要加入重製畫面或數據的程式碼。此外，觸發彈出視窗的機制也寫在這一部分，我們在繪圖介面中有寫偵測碰觸的方法，因此只要在偵測碰觸方法中回傳 boolean 值(true & false)，透過 boolean 值來判斷是否選取到物件而決定要不要跳出屬性選單。

(2) 彈出視窗 (Dialog)

彈出視窗的出現時機為使用者選取到繪圖視窗的物件後，再按一次工具列中的編輯按鈕，彈出視窗就會顯現，讓使用者去設定被選擇之物件的屬性(ex:座標、外力、材質…

等)，由於要使用的彈出視窗並非 Android Studio 的預設格局，故我們需要先為每個彈出視窗建立一個相對應的 xml 格局檔，再從為每一個彈出視窗建立有繼承 DialogFragment 的 class 檔中讀取，才能使彈出視窗出現並執行我們需要的動作。

設計思路：

設計彈出視窗最主要的功能是提供使用者一個可輸入數據覆寫到共用數據庫的介面，覆寫的思路是當使用者選取繪圖介面的物件後，會回傳一個目錄值，而每一種物件在建立時皆分別儲存進相對應物件的獨立數據庫，因此只要使用者輸入的數據至鍵入視窗中，覆蓋到相對應物件數據庫中的指定目錄，就可以完成更改數據。由於在數據庫建立的時候，已經將許多可能會共用的變數和資料庫設定為 static 類別，進行此宣告即可讓更改在不同 class 的變數的行為方便實行。讀取其他 static 變數和讀取原本 class 中變數的方法有所不同，必須先告訴程式是要讀取哪個 class 的 static 變數，例如：TrussPaintView(static 變數設置的 class).nodearray(static 物件名稱)，才可以進行後續的變數操作。使用 static 讀取變數雖然方便，但在後續維修的時候很容易造成混亂，使用 static 也會使程式的效率降的，因此 static 變數需要謹慎使用。

(3) 滾動視窗 (ScrollView)

滾動視窗在 APP 中的作用為輸出答案數據到畫面上，加入此物件的原因是當繪圖視窗中物件數量較多的話，數字和圖形會過多且互相覆蓋導致畫面過於擁擠、混亂，此外使用滾動視窗就不需擔心數據數目太多導致無法輸出的問題，因此我們選擇使用此工具用來顯示計算後的數據。

設計思路：

ScrollView 內預設使用的是線性布局，我們想要每列出一筆資料就換行的效果，可以實現的做法是利用程式的生命週期。先建立 for 迴圈，遞迴次數為你要輸出幾筆資料，在迴圈中每遞迴一次就新增一個新的 TextView，設定完文字屬性和欲輸出之文字後，將新增的 TextView 加入線性佈局中，到這裏就完成了一次遞迴。這樣就可以作出在 ScrollView 中換行列出數據的效果。剛設計完成的時候發現過一個問題，在進行編輯功能的時候會有畫面閃爍，甚至是畫面清空的情形出現，問題的主因似乎是 ScrollView 和繪圖視窗互相覆寫所導致。我們的解決方式為調整 ScrollView 的可視屬性。當使用者點擊計算按鈕的時候，將可視屬性設定為 visible，也就是當使用者點擊計算按鈕後才可以看見此視窗，切換選單的其他功能時就將可視屬性設定為 invisible 或 gone 使其隱藏。

3.8 切換 Actitivy

我們現在設計了 APP 首頁、Truss 繪圖介面、Beam 繪圖介面、折線圖表介面。那這四個介面之間是如何連接、切換的？Android Studio 裡封裝的意圖(Intent)套件可以實現這個功能。使用方法為，建立一個 Intent 物件，在他的建構方法中宣告要從哪個 Actitivy 對應的 class 切換到另一個 Actitivy 對應的 class，接著使用 startActivity 方法，

在括號中放建立好的 Intent 物件，即可完成切換介面的動作，如圖 51 之程式碼：

```
Intent intent = new Intent(BeamActivity.this, ResultActivity.class);
startActivity(intent);
```

圖 51　Intent(意圖)使用方法

Intent 可以做到切換視窗以外，還可以攜帶原 class 的數據到另一個 class(限制必須為不同的 Actitivy 才可實行)。Intent 就好像交通工具，我們可以在交通工具中塞入包裹讓其攜帶到不同地方，在 Android Studio 中，包裹就是 Bundle 物件。Bundle 可塞入任何東西，將要傳遞的物件放入 bundle 之後，使用 intent.putextra()在 Intent 物件中放入包裝好的 Bundle，再執行 startActitivy(Intent)切換 Actitivy，這樣即可將物件傳遞到不同的 Actitivy 之中。要注意的是，傳遞完成後，要在接收的 class 處建立類別相同的物件放置傳遞過來的 Bundle，使用 intent.getBundleExtra(bundle)將物件從 bundle 之中取出。

當開發者要對不同 class 的變數進行傳遞或使用，可以利用 Intent 套件或者將物件設定為 static 類別，兩者各有特色。Intent 除了能使不同 Actitivy 的交流切換以外還可以開啟其他應用程式，例如許多社群軟體的拍照功能就是使用 Intent 開啟手機的相機應用程式，但它不能處理相同 Actitivy 中不同 class 之間的更改參數動作，要實現不同 class 的更改參數，將物件宣告為 static 會方便許多。

第四章 JAVA 計算程式開發

第二章提到本 APP 所使用到的理論，接下來是說明如何將理論轉換成 Java 程式。在程式中，我們先建立數據所需的矩陣，包括點資料、桿件資訊、節點類型判斷、外力資料，這些是為了承接使用者輸入的資訊。接下來則是建立計算上的矩陣，分別為第二章闡述之外力矩陣 **F**、全域剛度矩陣 **K** 和位移矩陣 **U**。

4.1 直接徑度法

(1) Truss

Truss 的程式開發大致分為 **K** 矩陣的填入、待求數據的矩陣刪減、各點受力計算和桿件內力計算。而關於自由度的編號，我們依照使用者建點的順序建立，第 1 個點之自由度即為 1 和 2，第二個點的自由度則是 3 和 4，依此類推。計算程式的設計依步驟分為：**K** 矩陣填入、矩陣刪減、各點受力計算、桿件內力計算。

(1) **K** 矩陣填入：**K** 矩陣填入方式我們是先紀錄桿件起點與終點之點號，再依照點號拿到起點與終點座標，此時桿件長度即可透過畢氏定理求出。節點數據在前端介面，也就是繪圖介面的時候就已經建立好使用點號排序之節點所在座標的數據庫，並且已經把數據庫設定為 static 類別，也就是所有的 class 都可以取得，因此設計計算程式的 class 只要直接使用 for 迴圈將所有數據從節點座標數據庫取出即可，如圖 52。

```
for (int i = 0; i < n; i++) {
        Coordinate[i][0] = (float)TrussPaintView.nodearray1.get(checkn.get(i)).get(1);
        Coordinate[i][1] = (float)TrussPaintView.nodearray1.get(checkn.get(i)).get(2);
        Log.d("coordx",String.valueOf(Coordinate[i][0]));
        Log.d("coordy",String.valueOf(Coordinate[i][1]));
}
```

圖 52　取得節點數據

我們先分析整個結構有多少個自由度，先開好(自由度數量*自由度數量)大小的矩陣。由於自由度與點在設計時是有關聯的，因此在填入 **K** 矩陣時，我們只要知道它是哪個點就可以了解該點所帶有的自由度。如此一來，只要依照其自由度填入相應的的矩陣位置即可(參考圖 11)。

```
K[(2 * dot1)][(2 * dot1)] += EA * ((cos * cos) / L);
K[(2 * dot1)][(2 * dot1) + 1] += EA * ((cos * sin) / L);
K[(2 * dot1)][(2 * dot2)] += EA * ((-(cos * cos)) / L);
K[(2 * dot1)][(2 * dot2) + 1] += EA * ((-(cos * sin)) / L);
```

```
K[(2 * dot1) + 1][(2 * dot1)] += EA * ((cos * sin) / L);
K[(2 * dot1) + 1][(2 * dot1) + 1] += EA * ((sin * sin) / L);
K[(2 * dot1) + 1][(2 * dot2)] += EA * ((-(cos * sin)) / L);
K[(2 * dot1) + 1][(2 * dot2) + 1] += EA * ((-(sin * sin)) / L);

K[(2 * dot2)][(2 * dot1)] += EA * ((-(cos * cos)) / L);
K[(2 * dot2)][(2 * dot1) + 1] += EA * ((-(cos * sin)) / L);
K[(2 * dot2)][(2 * dot2)] += EA * ((cos * cos) / L);
K[(2 * dot2)][(2 * dot2) + 1] += EA * ((cos * sin) / L);

K[(2 * dot2) + 1][(2 * dot1)] += EA * ((-(cos * sin)) / L);
K[(2 * dot2) + 1][(2 * dot1) + 1] += EA * ((-(sin * sin)) / L);
K[(2 * dot2) + 1][(2 * dot2)] += EA * ((cos * sin) / L);
K[(2 * dot2) + 1][(2 * dot2) + 1] += EA * ((sin * sin) / L);
```

圖 53　依照自由度的編號填寫 K 矩陣

經過這個操作將建立的所有個別桿件的單位剛度矩陣填入後，就可以合併出使用者在畫面上繪製之結構的全域剛度矩陣 **K**。

(2) 矩陣刪減：因為我們想要求得桿件上未知的受力和變形，我們要先刪除未知的力和已知的位移，藉由留下來的已知力求出未知的位移，最後將求得的位移代回原本的 **F** = **KU** 矩陣即可求出未知的力，求出的力即為結構上節點的支承反力。此部分我們使用的方法為記錄可動自由度的點號。如此藉由點號紀錄把想留的部分填到新建的陣列上，刪除動作即完成，接下來代入 LU 逆運算即可依序求解。

(3) 各點受力計算：在 LU 逆運算之後，我們已經知道每個自由度的移動量(也就是全域位移向量矩陣 **U**)，將其回填至新建立之位移矩陣。想得到最後的各點受力(全域力向量矩陣 **F**)，把 **K** 與新建立的位移矩陣作直接勁度法的矩陣相乘(**F** = **KU**)即可。

(4) 桿件內力計算：將各桿件所得到的自由度移動量 **U**，分別依該桿件的點代回 $\boldsymbol{f}=\boldsymbol{k^eT^TU}$ 進行矩陣運算，即可求出每根桿件的單一受力，也就是桿件內力。

(2) Beam

Beam 的直接勁度法與 Truss 雷同，不同點只在於單位剛度矩陣之元素而已。而 Beam 程式大致分為 **K** 矩陣的填入，待求數據的矩陣刪減，各點受力計算和剪力彎矩圖的繪製。唯一與 Truss 不一樣的只有要依據求出之端點的位移與角度變形，代入由四階法所推導之彎矩(式 26)、剪力(式 28)方程式。首先，我們先將垂直位移、旋轉徑度、彎矩和剪力之方程式輸入至程式中，將他們寫成一個一個方法以供使用。此部分完成後，再提取彎

矩(圖 54)、剪力(圖 55)、旋轉徑度(
圖 56)和垂直位移(圖 57)的副程式代入新視窗中對應的的繪圖視窗，依序繪製剪力、彎矩、旋轉徑度和垂直位移之折線圖。下列的四張程式碼截圖中有數個共通變數，L 是使用者建立的個別桿件之長度，名為 BeamData 的 Arraylist 放置的是起點的位移變形 v_I、角度變形 θ_I 與終點的位移變形 v_J、角度變形 θ_J。名為 attritrussarray 的 Arraylist 放置的是楊氏模數 E 和桿件截面面積二次矩 I。

```
//扭矩
public static float Mt(float x, int i) {
    float Mt, M1 , M2 , M3 , M4 , L ;
    L = (float) LengthM.get(i).get(0);
    M1 = -(((float) BeamData.get(i).get(0)) * (float) ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2)
            * ((12 * x / pow(L, 3)) - ((6) / pow(L, 2)))));
    M2 = -(((float) BeamData.get(i).get(1)) * (float) ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2)
            * ((6 * x) / pow(L, 2) - ((4) / L))));
    M3 = -(((float) BeamData.get(i + 1).get(0)) * (float) ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2)
            * (((-12 * x) / pow(L, 3)) + (6 / pow(L, 2)))));
    M4 = -(((float) BeamData.get(i + 1).get(1)) * (float) ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2)
            * (((6 * x) / pow(L, 2)) - (2 / L))));
    Mt = (M1 + M2 + M3 + M4);
    Log.d( tag: "Mt",  msg: M1 + "," + M2 + "," + M3 + "," + M4);
    return Mt;
}
```

圖 54 彎矩函數之程式碼

```
//剪力
public static float Vt(int i) {
    float Vt,V1,V2,V3,V4,L;
    L = (float)LengthM.get(i).get(0);
    V1 = ((float) BeamData.get(i).get(0)) * ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2))
            * (float) (12 / pow(L, 3));
    V2 = ((float) BeamData.get(i).get(1)) * ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2))
            * (float) (6 / pow(L, 2));
    V3 = ((float) BeamData.get(i + 1).get(0)) * ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2))
            * (float) (-12 / pow(L, 3));
    V4 = ((float) BeamData.get(i + 1).get(1)) * ((float) BeamPaintView.attritrussarray.get(i).get(0)
            * (float) BeamPaintView.attritrussarray.get(i).get(2))
            * (float) (6 / pow(L, 2));
    Vt = (V1 + V2 + V3 + V4);
    return Vt;
}
```

圖 55　剪力函數之程式碼

```
//扭轉角
public static float qt(float x,int i){
    float qt,q1,q2,q3,q4,L;
    L = (float) LengthM.get(i).get(0);
    q1 = ((float) BeamData.get(i).get(0)) * (float) ((6 * pow(x, 2) / pow(L, 3)) - 6 * x / pow(L, 2));
    q2 = ((float) BeamData.get(i).get(0) * (float) (3 * pow(x, 2) / pow(L, 2) - (4 * x / L) + 1));
    q3 = ((float) BeamData.get(i + 1).get(0) * (float) (-6 * pow(x, 2) / pow(L, 3) + 6 * x / pow(L, 2)));
    q4 = ((float) BeamData.get(i + 1).get(0) * (float) (3 * pow(x, 2) / pow(L, 2) - 2 / L));
    qt = q1 + q2 + q3 + q4;
    return qt;
}
```

圖 56 旋轉徑度函數之程式碼

```
//變形量
public static float vt(float x, int i) {
    float vt, v1, v2, v3, v4, L;
    L = (float) LengthM.get(i).get(0);
    v1 = ((float) BeamData.get(i).get(0)) * ((float) (((2 * pow(x, 3)) / pow(L, 3)) - (float) ((3 * pow(x, 2)) / pow(L, 2)) + 1));
    v2 = ((float) BeamData.get(i).get(1)) * ((float) (((pow(x, 3)) / pow(L, 2)) - (float) ((2 * pow(x, 2)) / pow(L, 1)) + x));
    v3 = ((float) BeamData.get(i + 1).get(0)) * ((float) (((-2 * pow(x, 3)) / pow(L, 3)) + (float) ((3 * pow(x, 2)) / pow(L, 2))));
    v4 = ((float) BeamData.get(i + 1).get(1)) * ((float) (((pow(x, 3)) / pow(L, 2)) - (float) ((pow(x, 2)) / pow(L, 1))));
    vt = v1 + v2 + v3 + v4;
    Log.d( tag: "V",  msg: v1 + "," + v2 + "," + v3 + "," + v4);
    return vt;
}
```

圖 57 垂直位移之程式碼

4.2 LU 逆矩陣運算

LU 的程式其實非常容易，唯一的困難處只在於分解 L 與 U 時的邏輯判斷而已。而在寫 LU 運算之前，由於 JAVA 沒有內建陣列計算，因此不管是加減乘除，或是轉置矩陣，皆須先自行寫成方法以供使用。完成需要之陣列基本運算後(圖 58)，再編寫正向迭帶與反向迭帶的程式碼，並將最後取得的答案矩陣提取出來即可完成整個 LU 運算動作，因通式已經在基本理論中推導完畢，因此在設計這部分程式的時候，只需要將通式輸入迴圈計算即可。

```
public static float[][] MMultiply(float [][] A,float [][]B,int m,int n,int p){ //矩陣相乘
    C =new float[m][p];
    0;i<m;i++){
        for(int j=0;j<p;j++){
            float I=0;
            for(int k=0;k<n;k++) {
                I=I+A[i][k]*B[k][j];
            }
            C[i][j] = I;
        }
    }
    return C;
}
```

圖 58 矩陣相乘之方程式

```
private float[][] BackwardSupstitution(float U[][],float Y[][],int n) {
    float [][]x=new float[n][1];
    float T2=0;
    for(int i=n-1;i>-1;i--) {
        T2=0;
        for(int j=i+1;j<n;j++) {
            T2=T2+U[i][j]*x[j][0];
        }
        x[i][0]=(Y[i][0]-T2)/U[i][i];
    }
    return x;
}
```

圖 59 反向迭帶(BackwardSupstitution)之程式碼

```
private float[][] ForwardSupstitution(float L[][],float B[][],int n){
    float [][]y=new float[n][1];
    float T1=0;
    for(int i=0;i<n;i++) {
        T1=0;
        for(int j=0;j<=i-1;j++) {
            T1=T1+L[i][j]*y[j][0];
        }
        y[i][0]=(B[i][0]-T1)/L[i][i];
    }
    return y;
}
```

圖 60 正向迭帶(FordwardSupstitution)之程式碼

圖 59 和圖 60 是分解完 LU 後所要用到的計算程式碼。接下來介紹 LU 的程式部分。一開始我們須接收要分解之 **A** 矩陣與 **B** 矩陣，之後就是使用邏輯判斷把 **A** 分解成 **L** 及 **U**。判斷程式如圖 61：

```
for (int i = 0; i < p; i++)//將A矩陣分解成LU矩陣
{
    for (int j = 0; j < q; j++) {
        float sum1 = 0; //暫存器
        float sum2 = 0;
        if (i == 0) { //算U[0][j]
            U[i][j] = A[i][j];
        } else {
            if (j == 0) { //算L[i][0]
                L[i][0] = A[i][0] / U[0][0];
            } else {
                if (i > j) {
                    for (int s = 0; s <= j - 1; s++) {
                        sum1 = sum1 + L[i][s] * U[s][j];
                    }
                    L[i][j] = (A[i][j] - sum1) / U[j][j];
                }
            }
            if (j >= 1) {
                for (int s = 0; s <= i - 1; s++) {
                    sum2 = sum2 + L[i][s] * U[s][j];
                }
                U[i][j] = A[i][j] - sum2;
            }
        }
        if (i == j) {
            L[i][j] = 1;
        }
    }
```

圖 61 將 A 矩陣分解之程式碼

在理論中我們已知如何求 L 及 U 的每個數值，在此我們只要將算出的數值填入正確的位置即可。此處 i 和 j 分別為 L 與 U 矩陣的行及列。若 i 為 0，則我們先算 U[0][j]，如此可求出第一行的 U。而當 i 不為 0，j 不為 0 且 i 大於 j 時，我們可以算出 L[i][j]的數值(下三角部分)。當 i 不是 0，且 j 大於等於 1 時，我們可以判斷出 U[i][j]的數值(上三角部分)。最後，當 i 不為 0 且 i 與 j 相等時，我們將 L[i][j]填 1。以上的先後順序在計算時是有先後順序的，因為後面之計算會用到前面所算出的數據，所以計算順序是 LU 程式很重要的一部份。

第五章 APP 使用說明

適用機型

Android 5.1 以上任意行動裝置

使用步驟

對 Truss 進行結構分析：

(1)進入介面：先想好欲建立的 F.B.D.圖，點擊 Truss 按鈕，進入 Truss 的繪圖視窗。

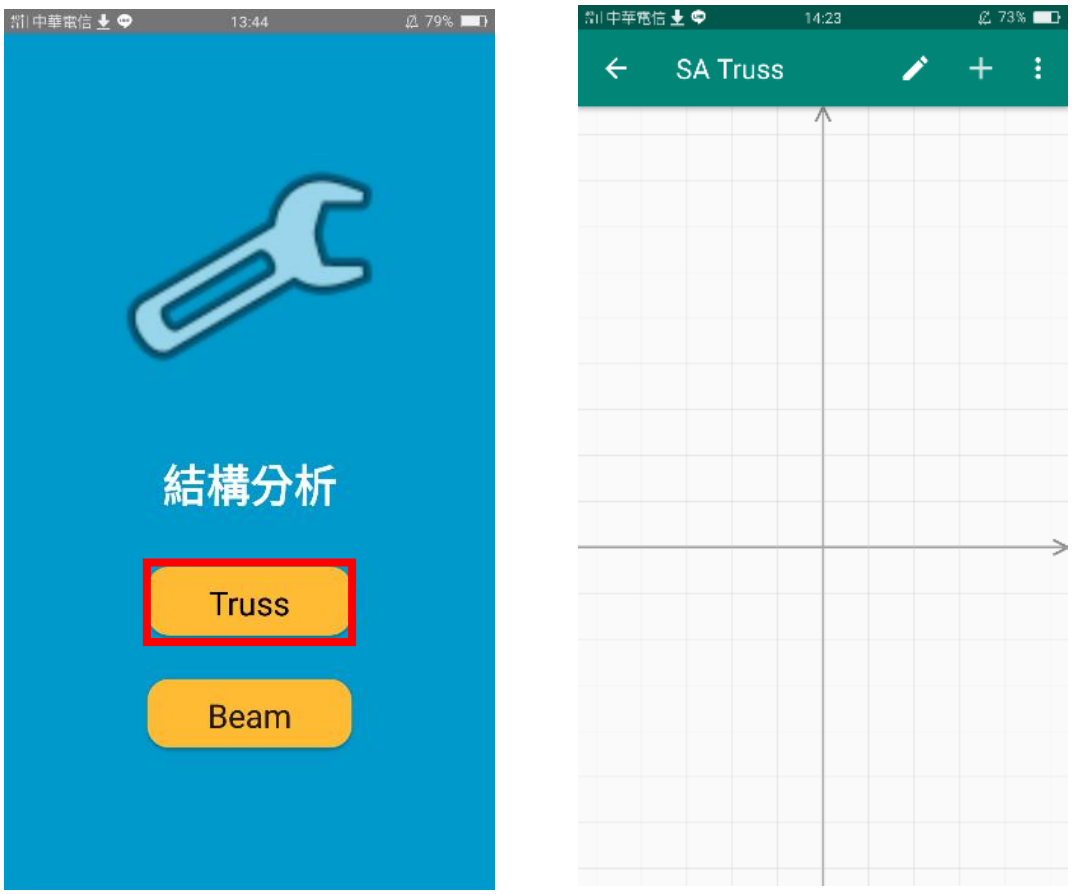


圖 62 演示進入繪製 Truss 介面

(2)建立節點：點選右上角的節點標示，點出各點大致的位置，網格背景中每一格代表一公尺

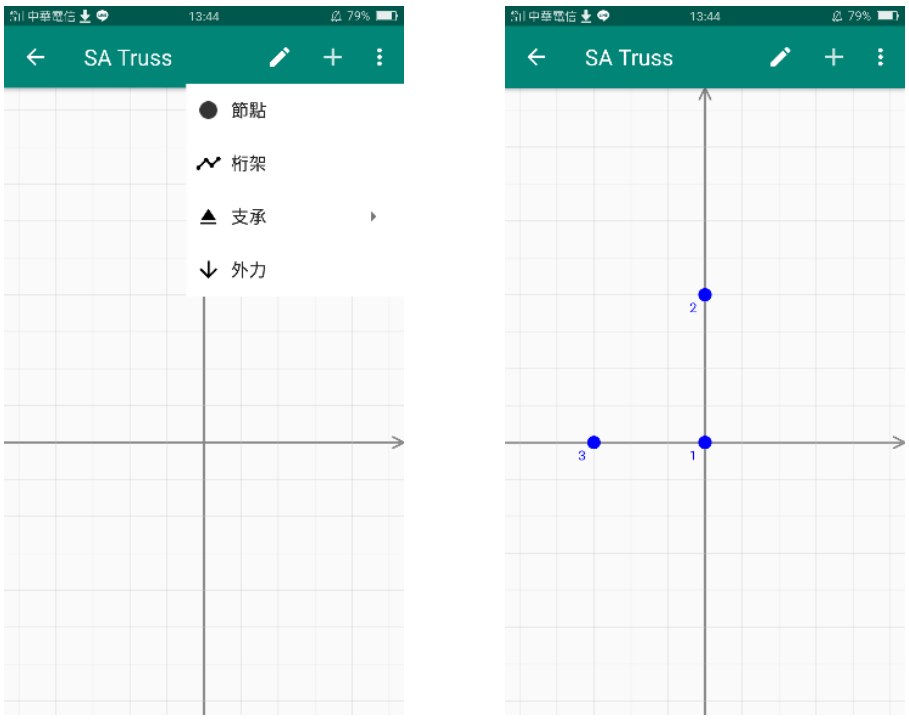


圖 63 演示建立節點

(3) 建立桿件: 點選選單中的桁架標示，接著點選需要連接的兩點，即可建立兩點間的桿件。

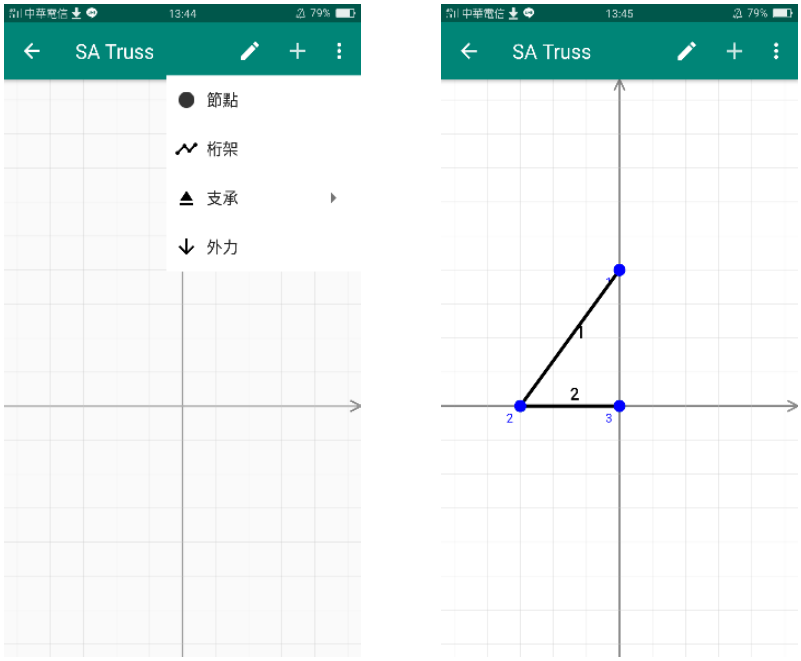


圖 64 演示建立桿件

(4) 輸入各個節點的接點種類以及外力：點選右上角支承圖示，根據目標節點的接點種類，選擇相對應的圖示後，再點擊目標節點，即可完成目標節點接點種類輸入。再點選右上角外力圖示，添加所需外力。

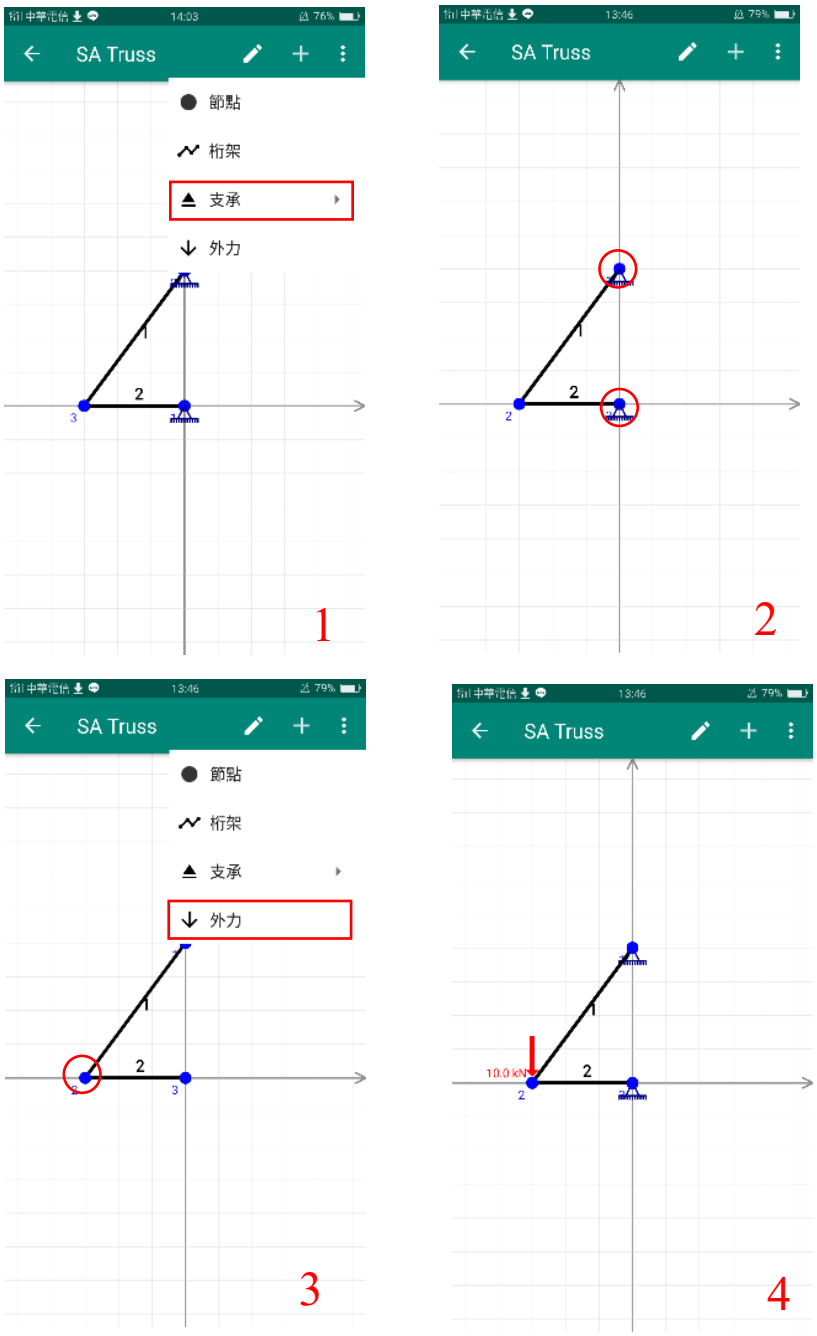


圖 65 演示建立支承和外力

(5) 修改節點桿件資訊: 先點選選單中的按鈕，之後選取欲修改屬性的物件，可跳出紀錄該物件相關數據的視窗。接下來輸入目標值後，點擊彈出視窗的 ADD 按鈕，即可完成物件屬性的修改，修改後的結果會動態顯示在繪圖視窗上。

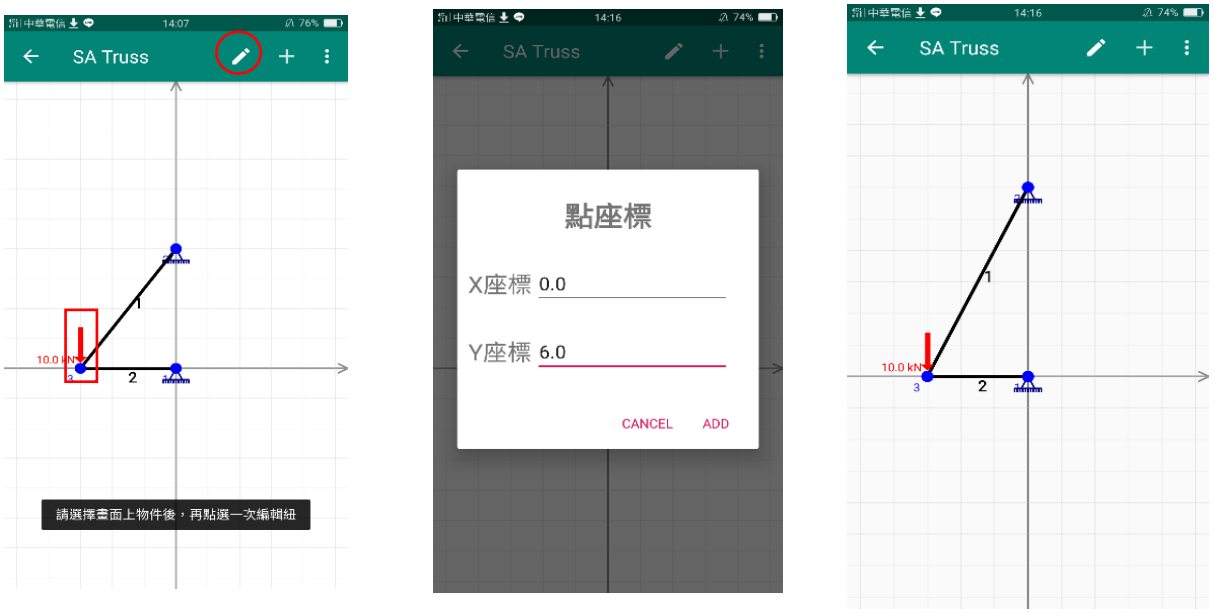


圖 66 演示更改外力屬性演示

(6) 資料輸入完成，開始計算：確認所需各項輸入資訊無誤後，點選右上角的  圖示，會跳出一個選單，選擇「開始計算」按鈕，程式就開始計算。計算完後，因施加外力而變形的圖形會以虛線表示，內力以藍色箭頭表示，內力的顯示為擠壓為負，拉伸為正。此外左上角還有一個可滾動的視窗，可以檢視所有節點受到的外力和所有桿件受到的內力。

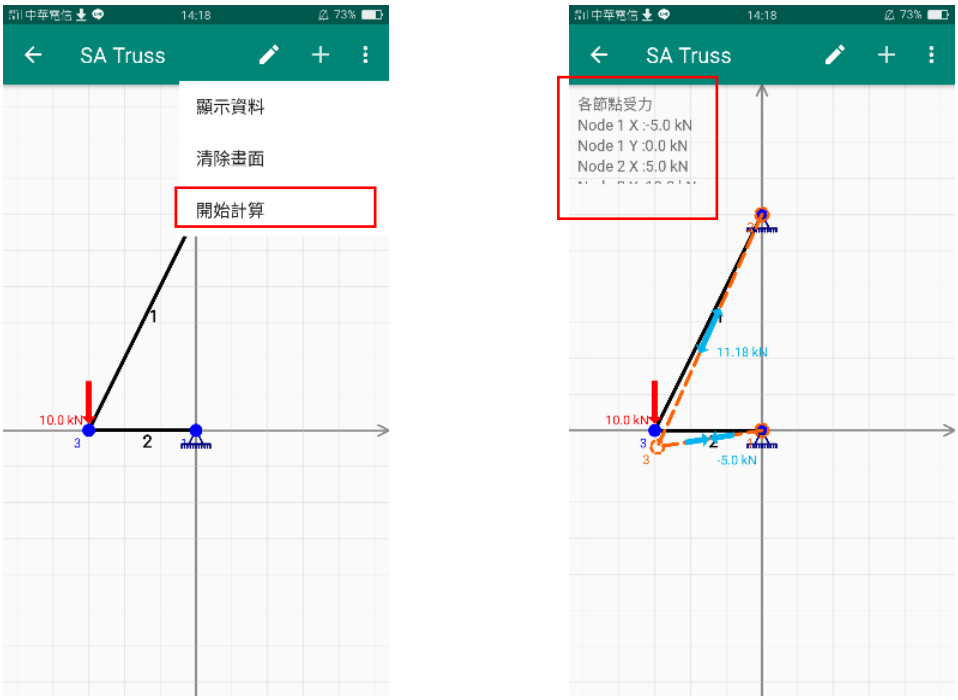


圖 67 演示 Truss 計算結果演示

對 Beam 進行結構分析：

(1) 進入介面：進入 APP 首頁，選擇 Beam，準備。

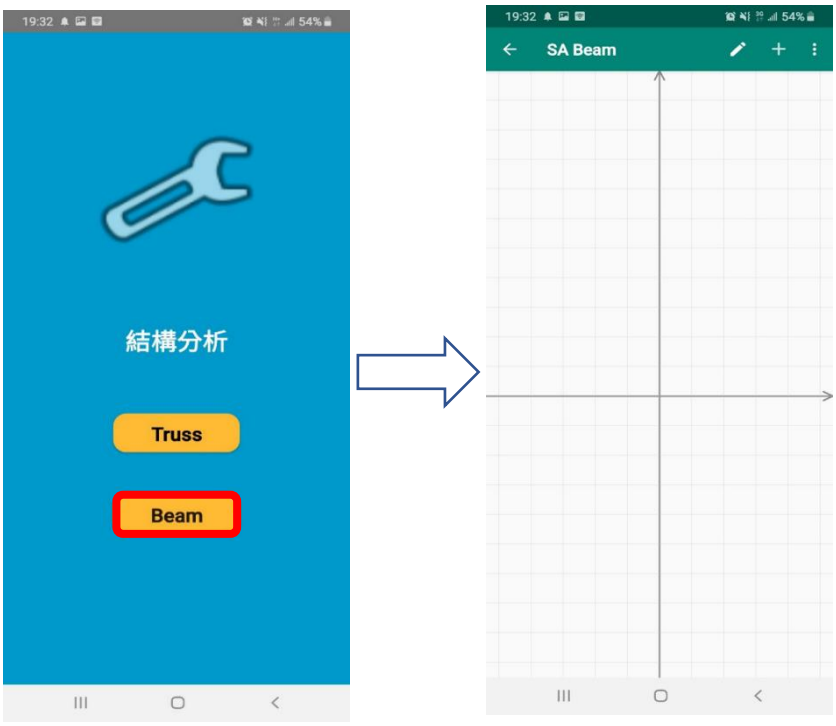


圖 68 演示進入繪製 Beam 介面

(2) 建立節點: 點擊+按鈕會跳出可建立物件之清單，選擇清單中的節點按鈕後，再碰觸螢幕即可建立節點

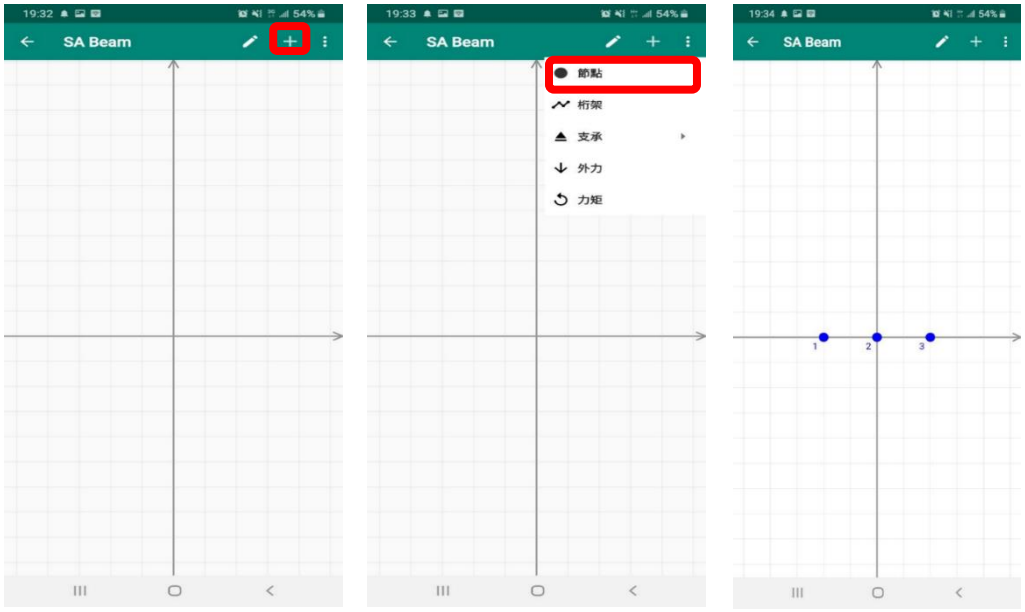


圖 69 演示在 Beam 繪圖介面建立節點

(3) 建立桿件: 接著點選需要連接的兩點，即可建立兩點間的桿件。

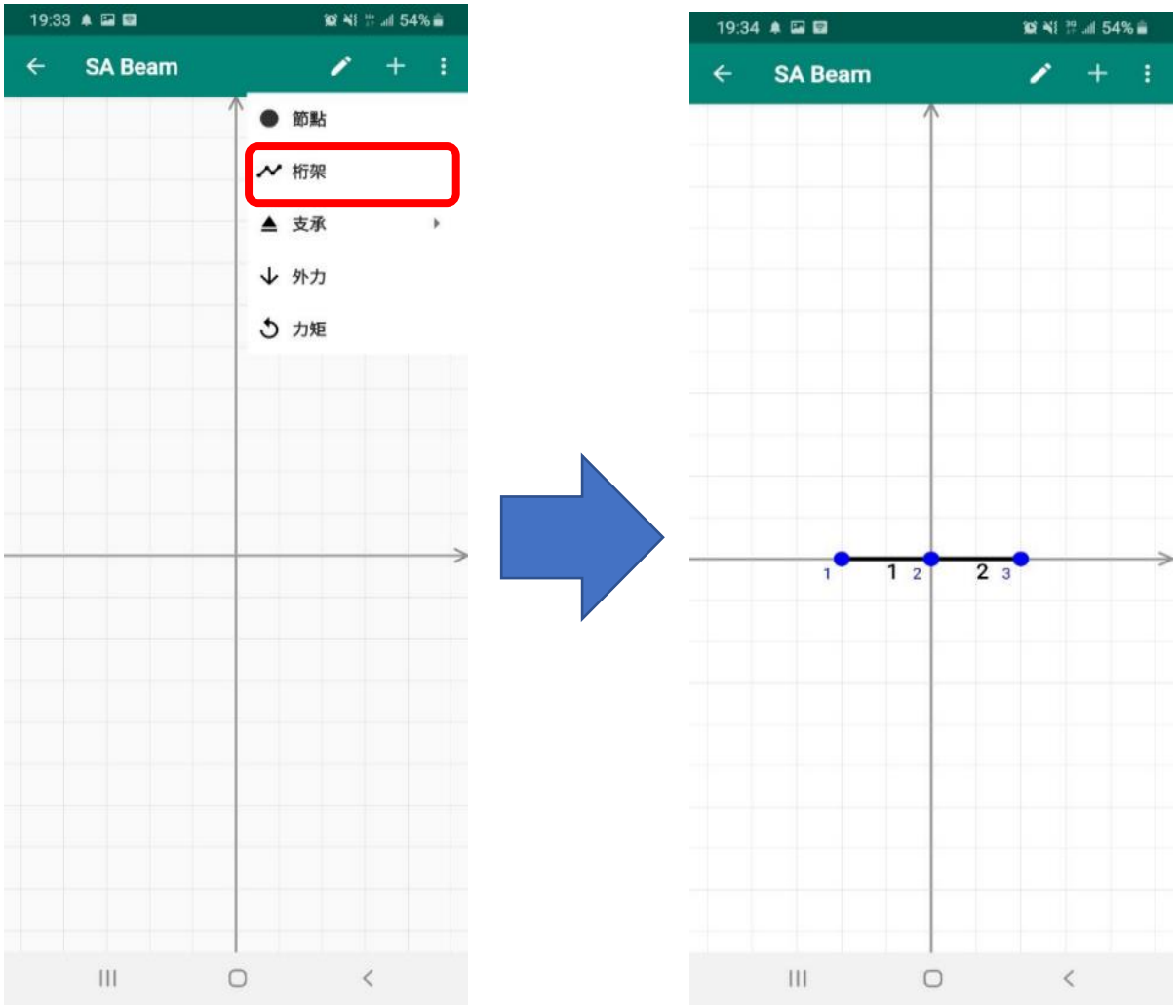


圖 70 演示在 Beam 介面建立桿件

(4) 輸入各個節點的接點種類:根據目標節點的接點種類，選擇相對應的圖示後，再點擊目標節點，即可完成目標節點接點種類輸入。

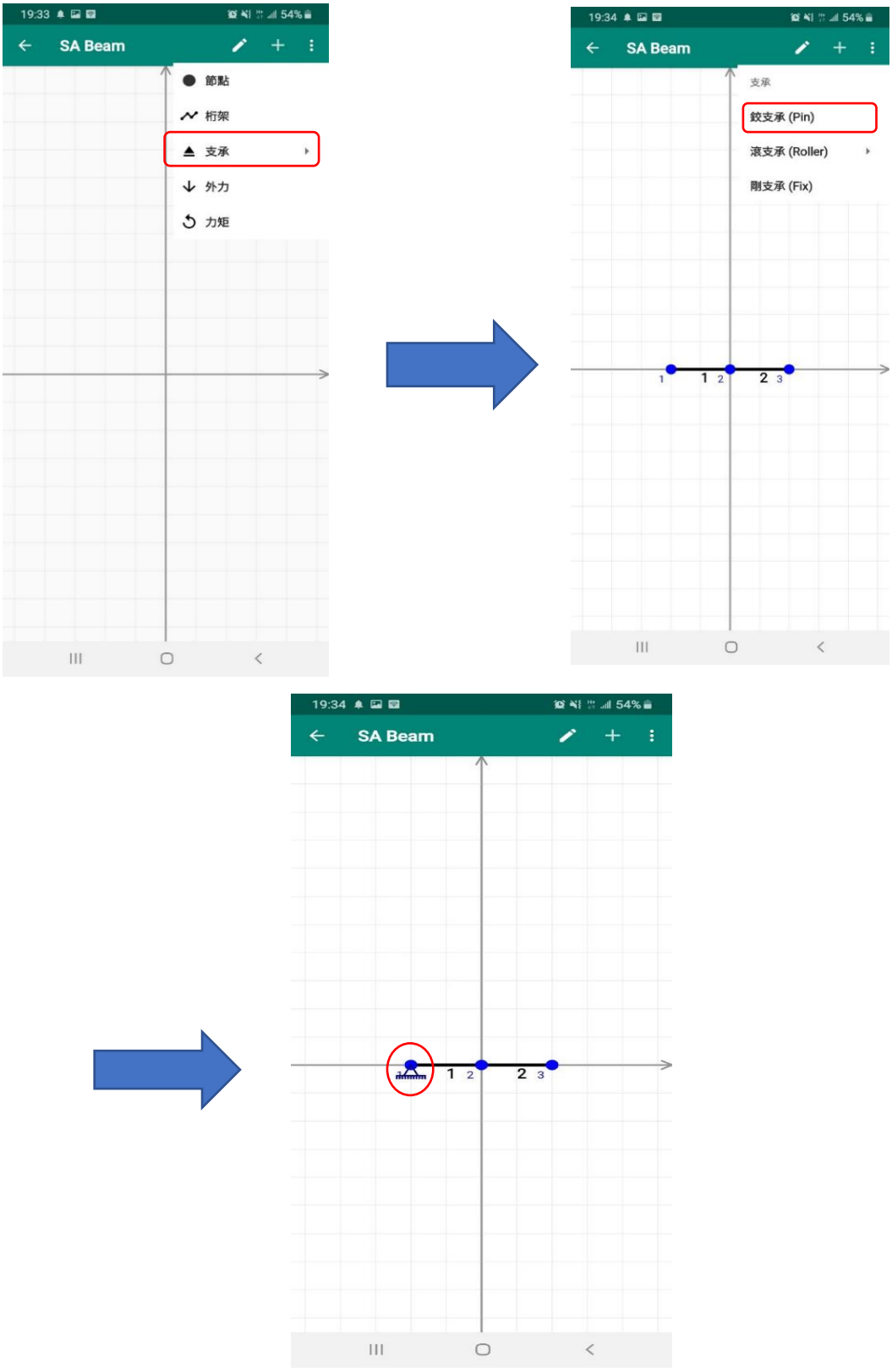


圖 71 演示建立 Support 中的 Pin

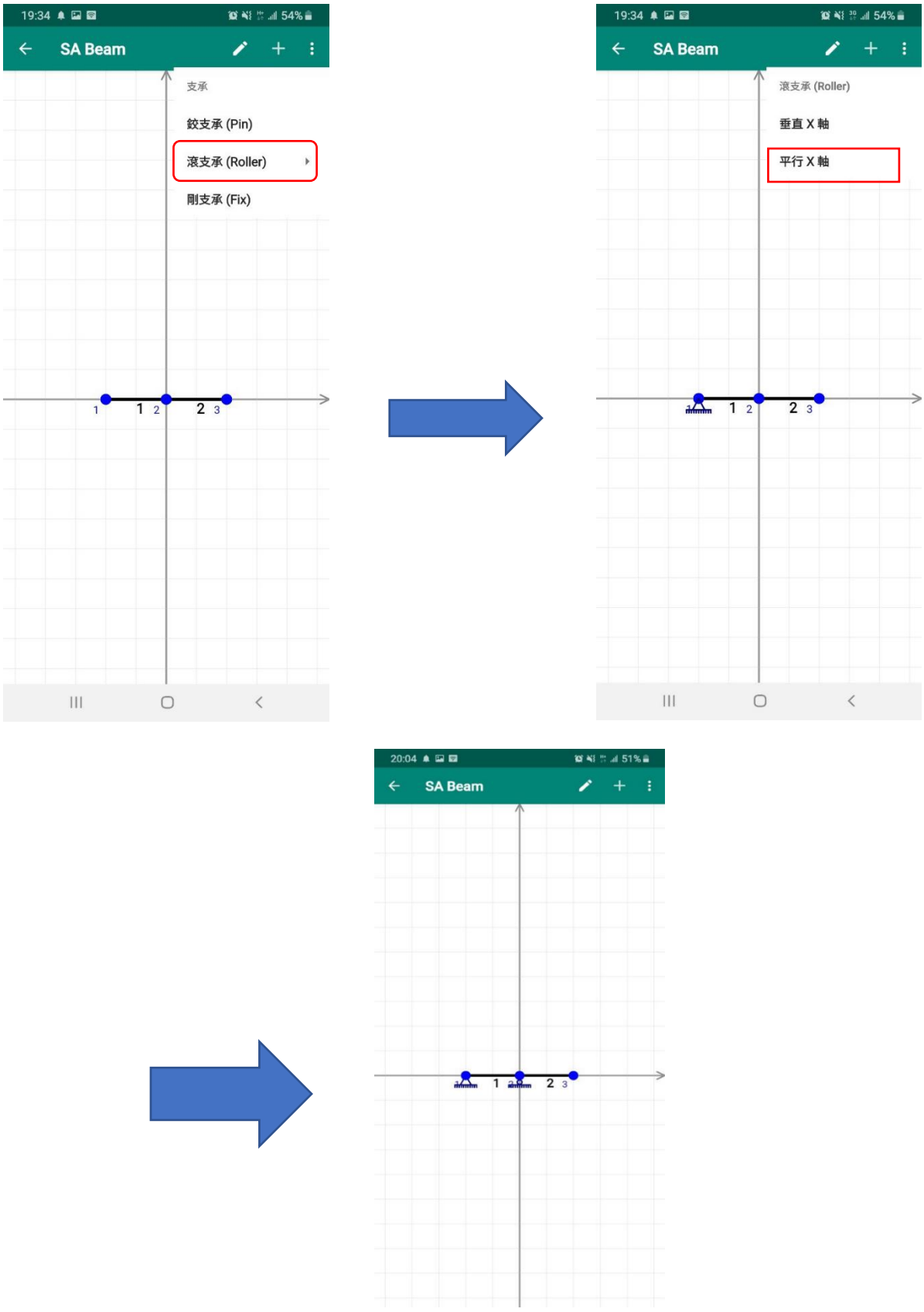


圖 72 演示建立 Support 中的 Roller

(5) 增加外力並修改各項資訊：選取畫面上的外力物件後，再點擊右上角之編輯按鈕，就會跳出彈出視窗，輸入欲更改之數值後點擊 ADD 按鈕即可。

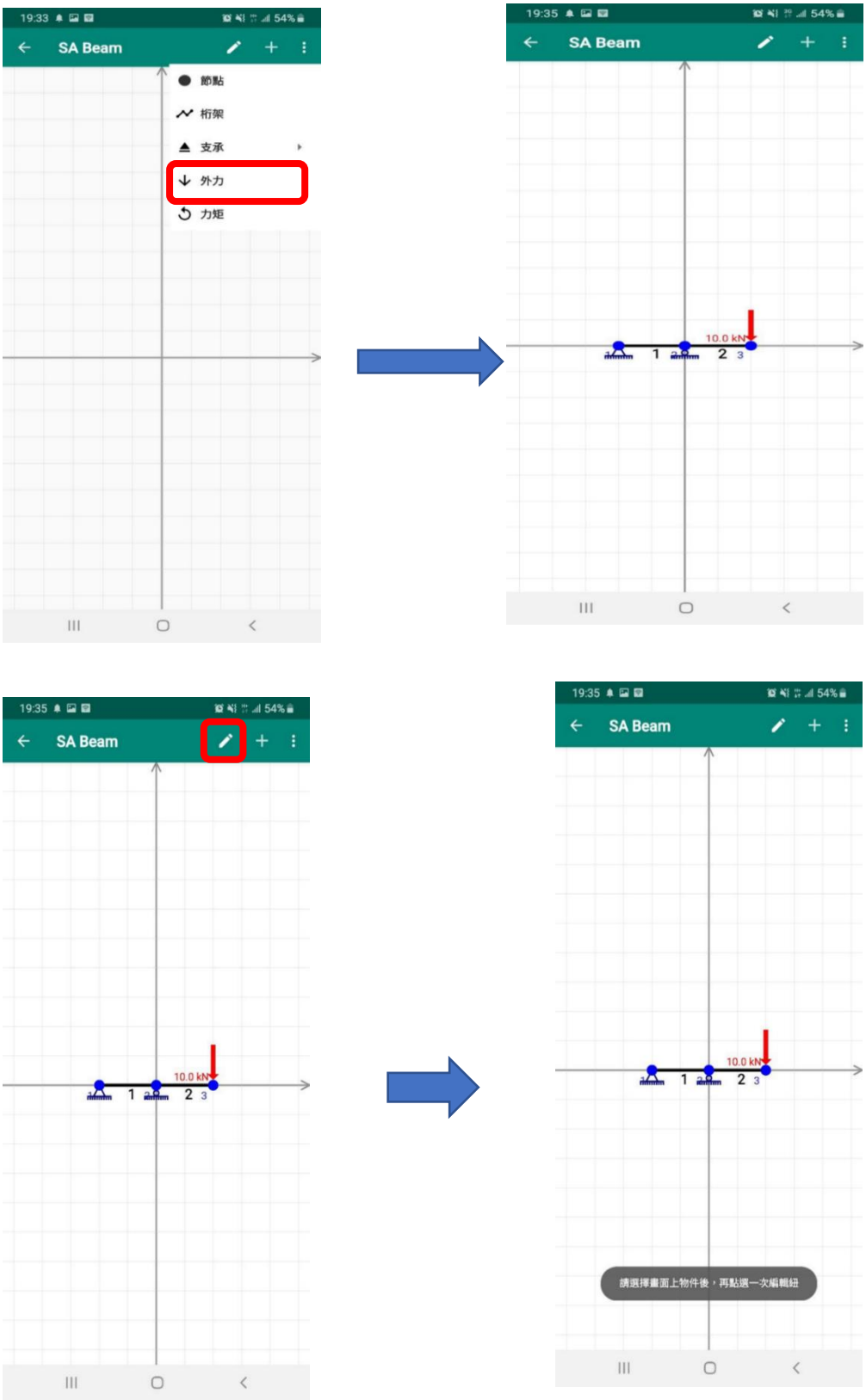


圖 73 演示編輯外力屬性(1)

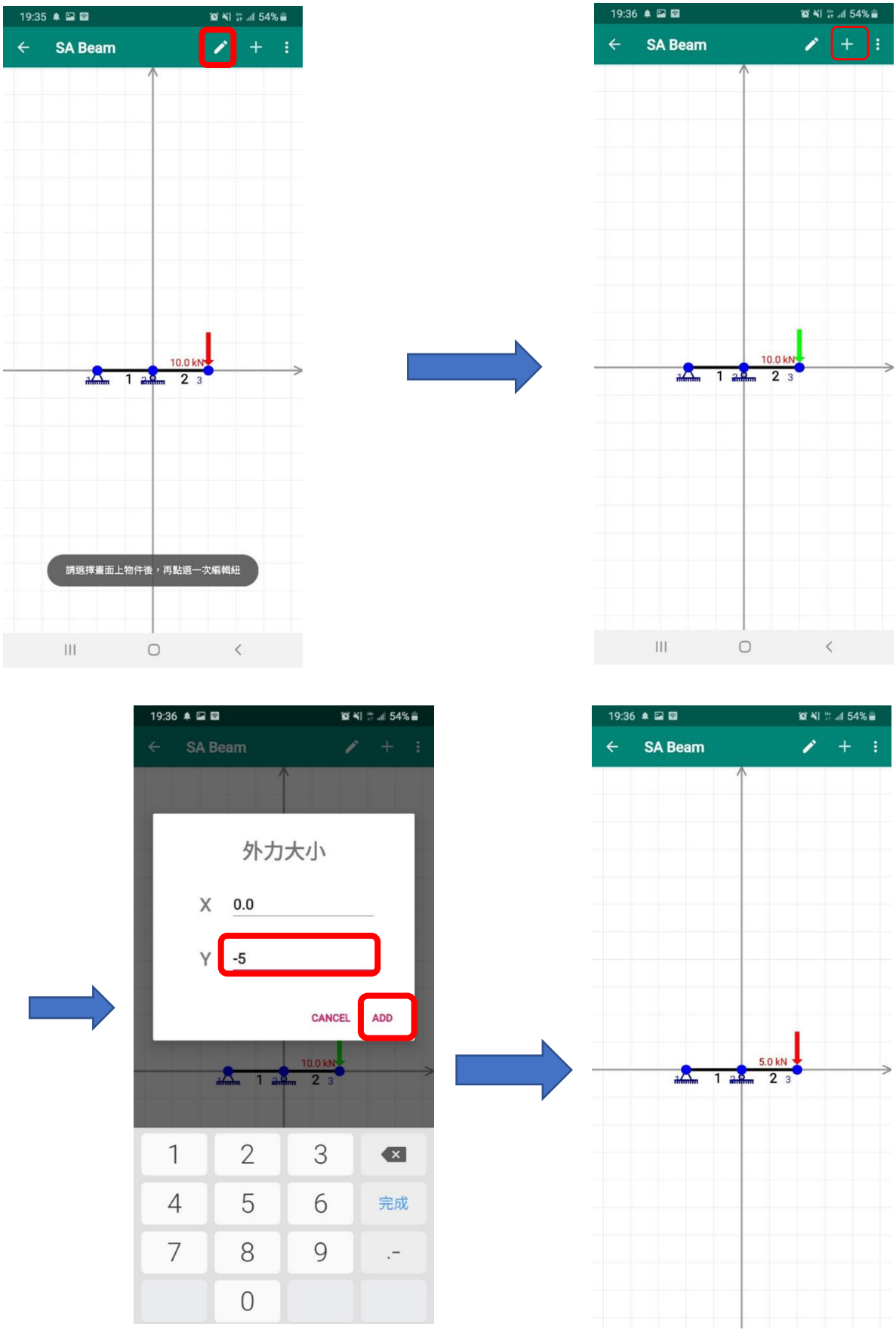


圖 74 演示編輯外力屬性(2)

(6) 資料輸入完成，開始計算： 確認所需各項輸入資訊無誤後，選擇「開始計算」按鈕，即可開始計算。

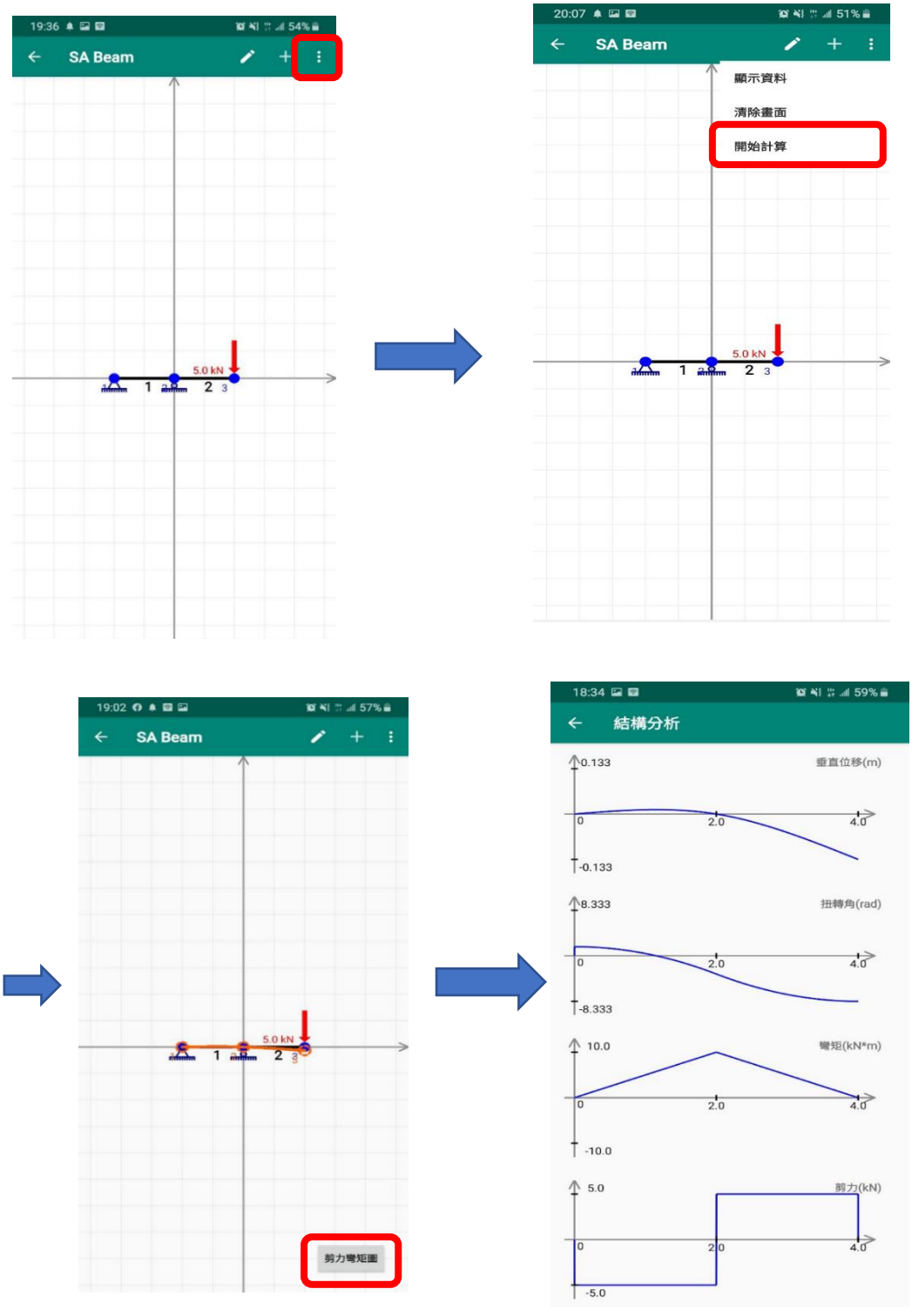
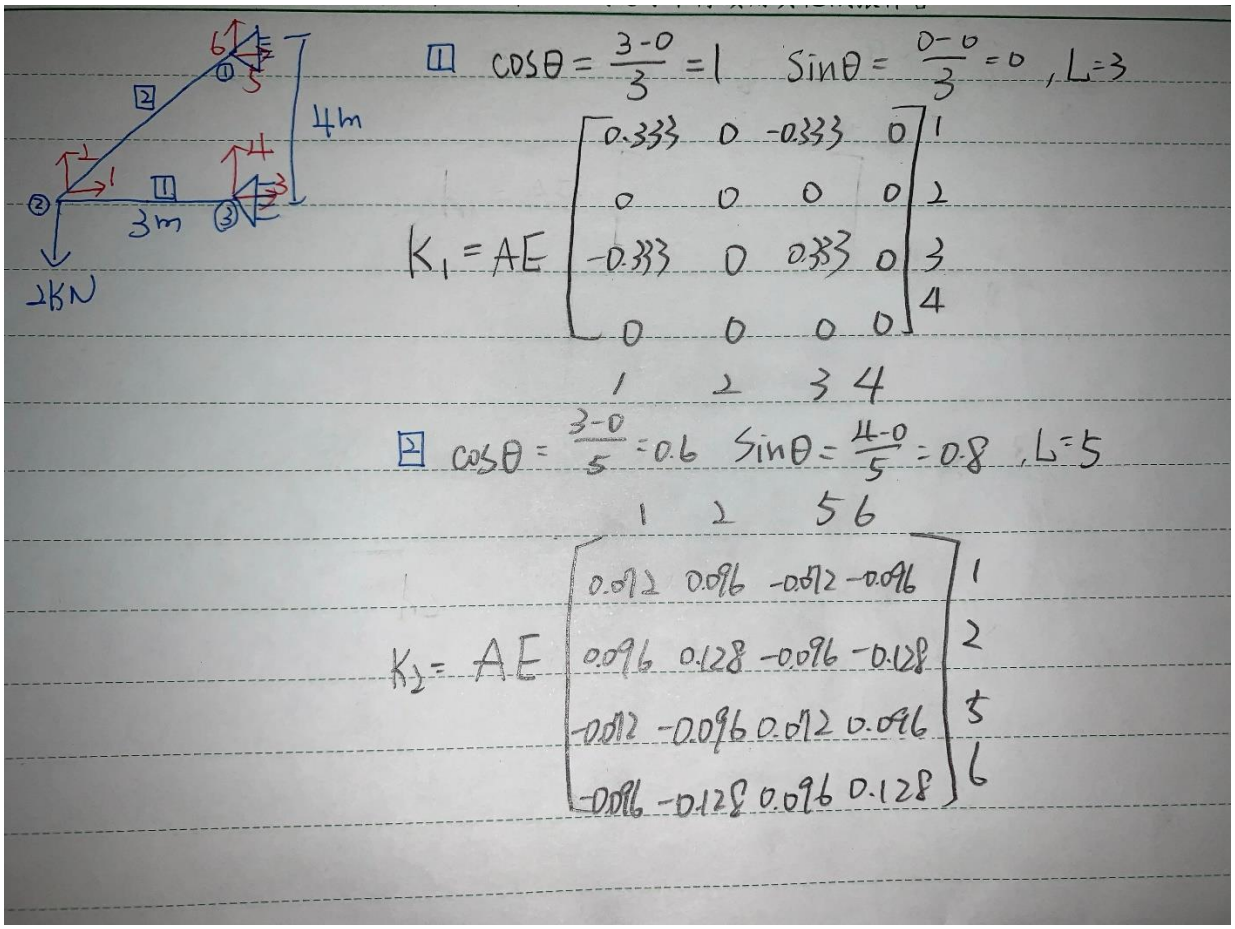


圖 75 演示 Beam 顯示答案

第六章 範例演示

Truss：使用 *Structural analysis* 第 8 版第 554 頁 Example 14.3 作為演示範例



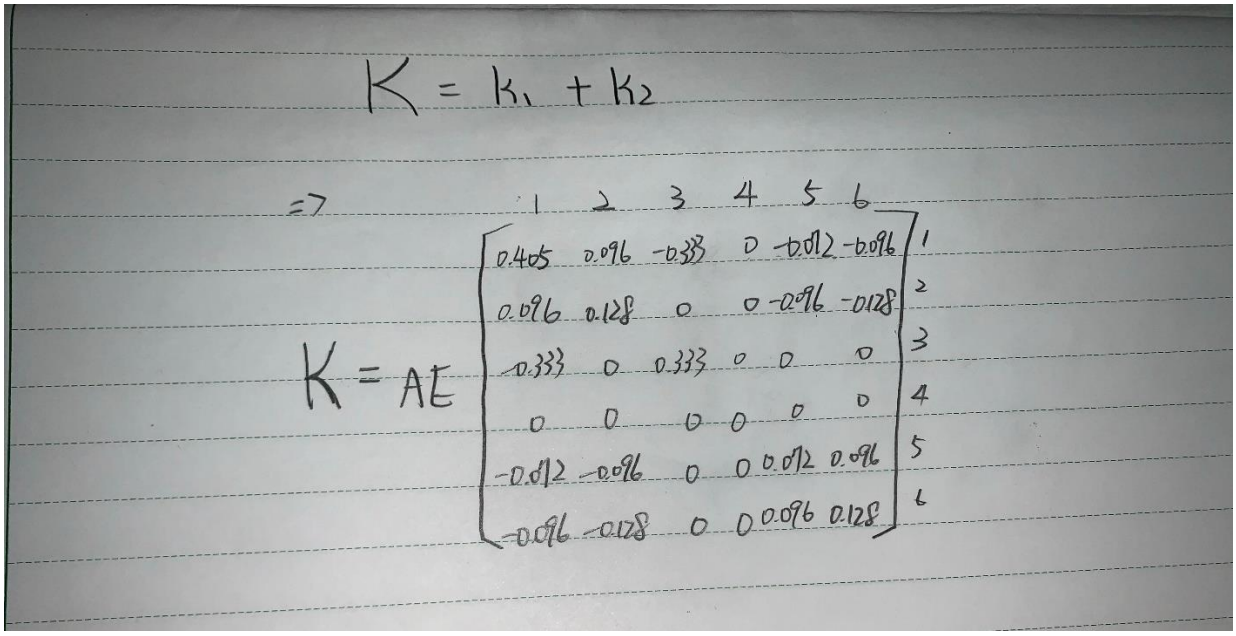
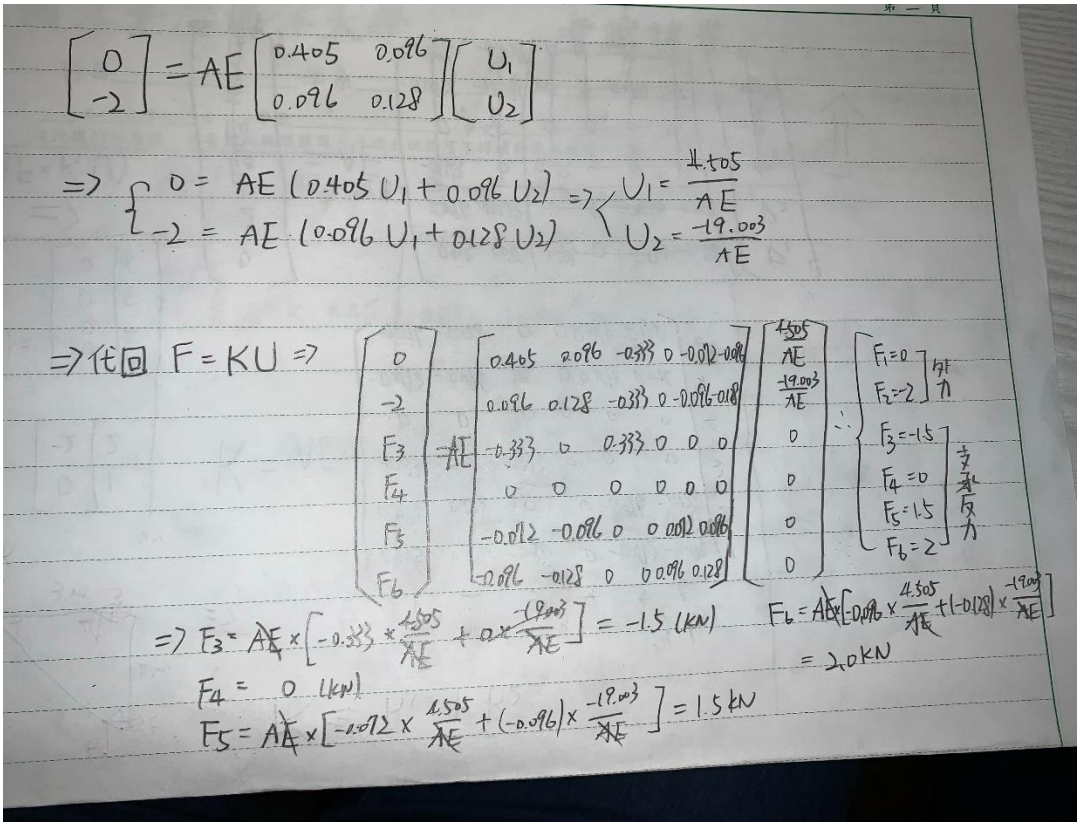
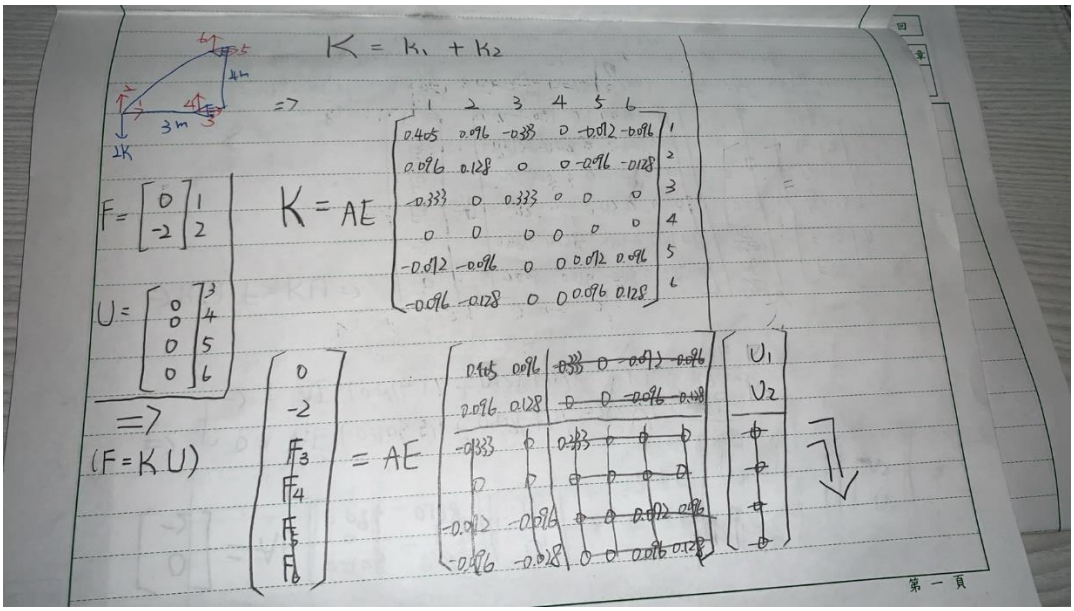
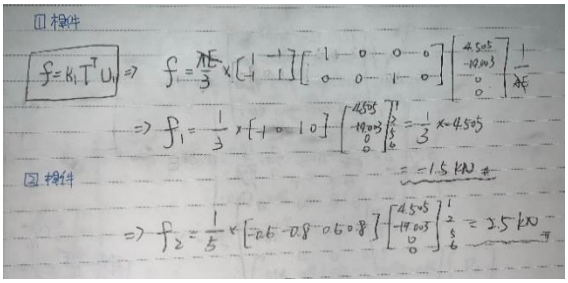


圖 76 Truss 範例題目(1)







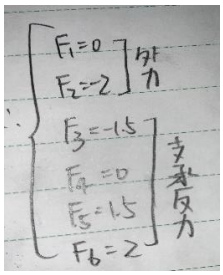


圖 77　Truss 範例題目(2)

依照題目，繪圖如左，按下計算後，得出結果如右：

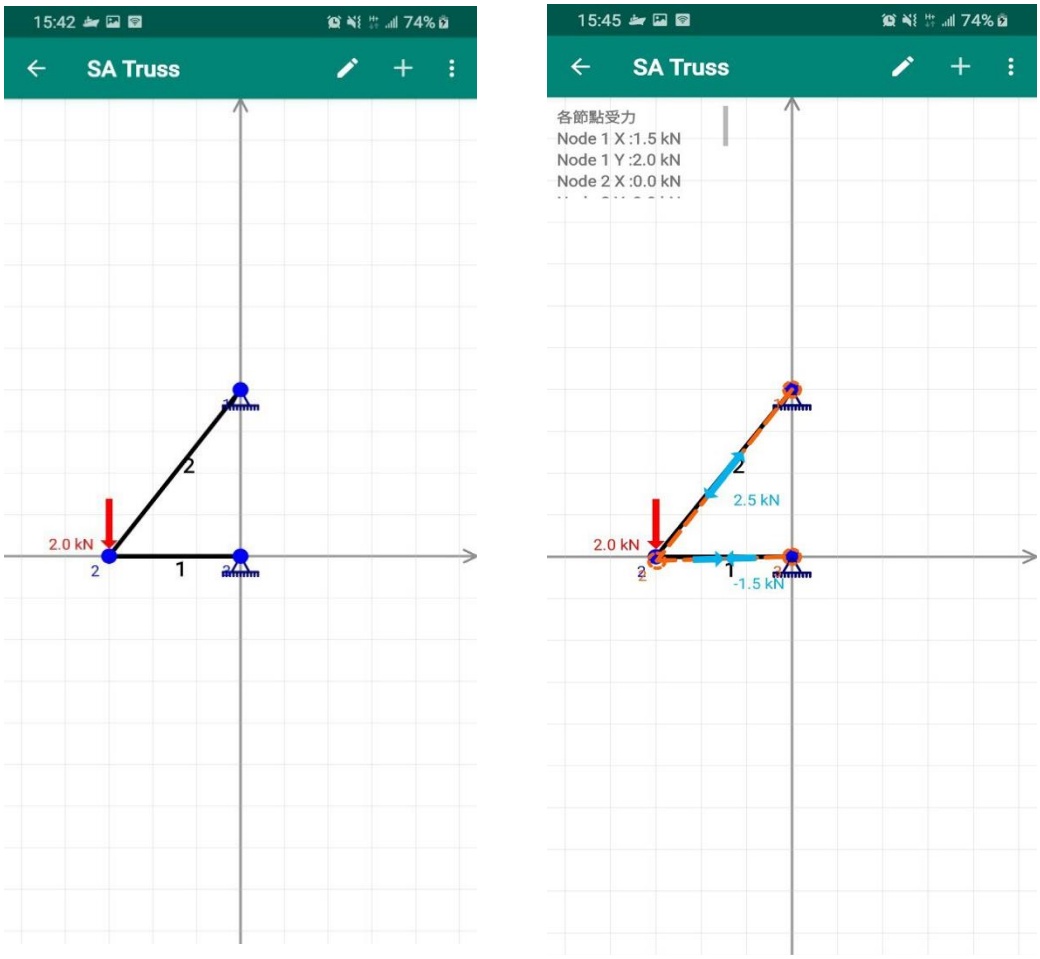


圖 78 程式計算之答案顯示

圖片中的左上角為答案之詳細結果，如圖 79，兩者比對的結果如表格 3：

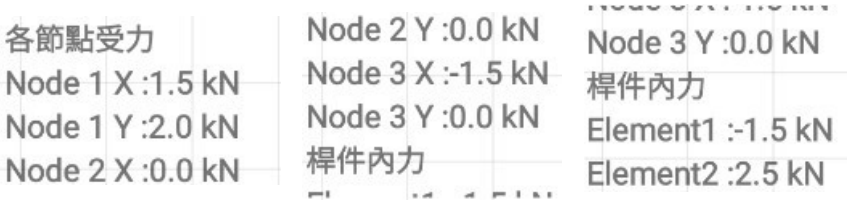
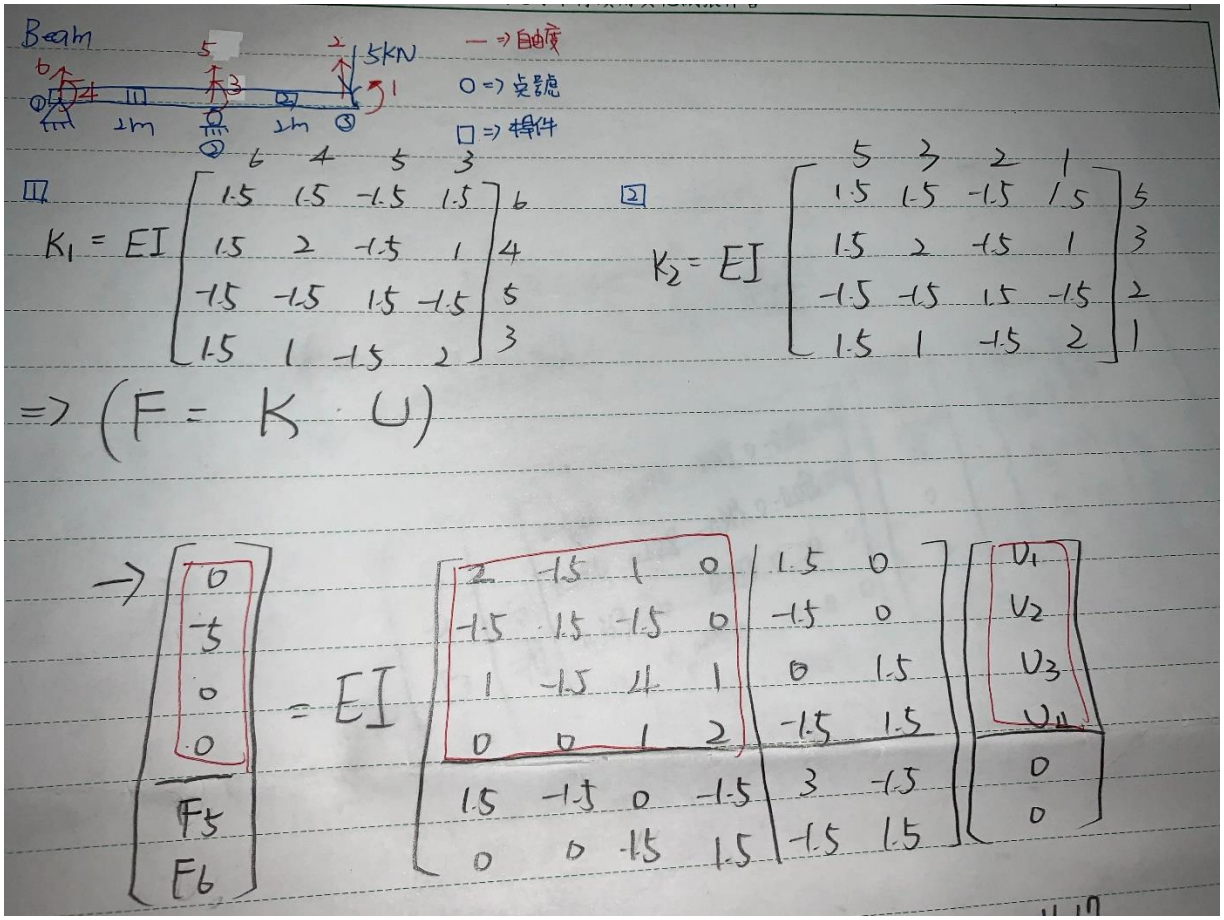


圖 79 滾動視窗中的答案數據

表格 3 Truss 例題和程式計算之答案比較

點號	方向	例題的答案(kN)	使用 APP 計算的答案(kN)
1	X	1.5	1.5
1	Y	2.0	2.0
2	X	-1.5(桿件內力)	-1.5(桿件內力)
2	Y	2.5(桿件內力)	2.5(桿件內力)
3	X	-1.5	-1.5
3	Y	0	0

Beam: 使用 *Structural analysis* 第 8 版第 582 頁 Example 15.1 作為演示範例



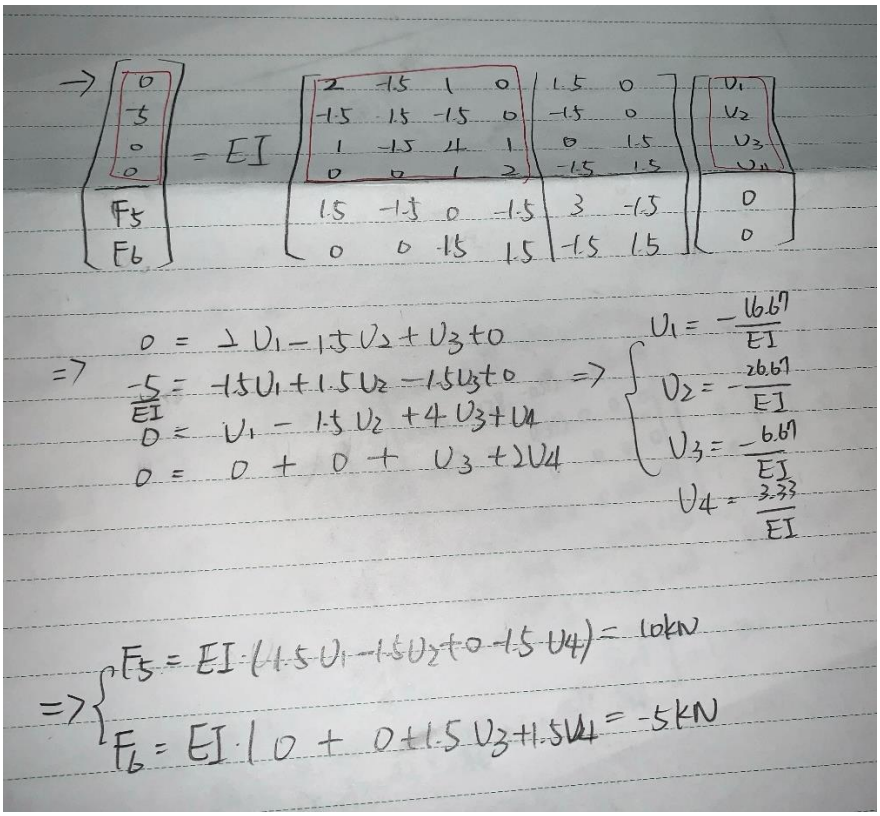


圖 80　Beam 範例題目(1)

APP 上繪圖結果如左，程式計算出的圖形如右

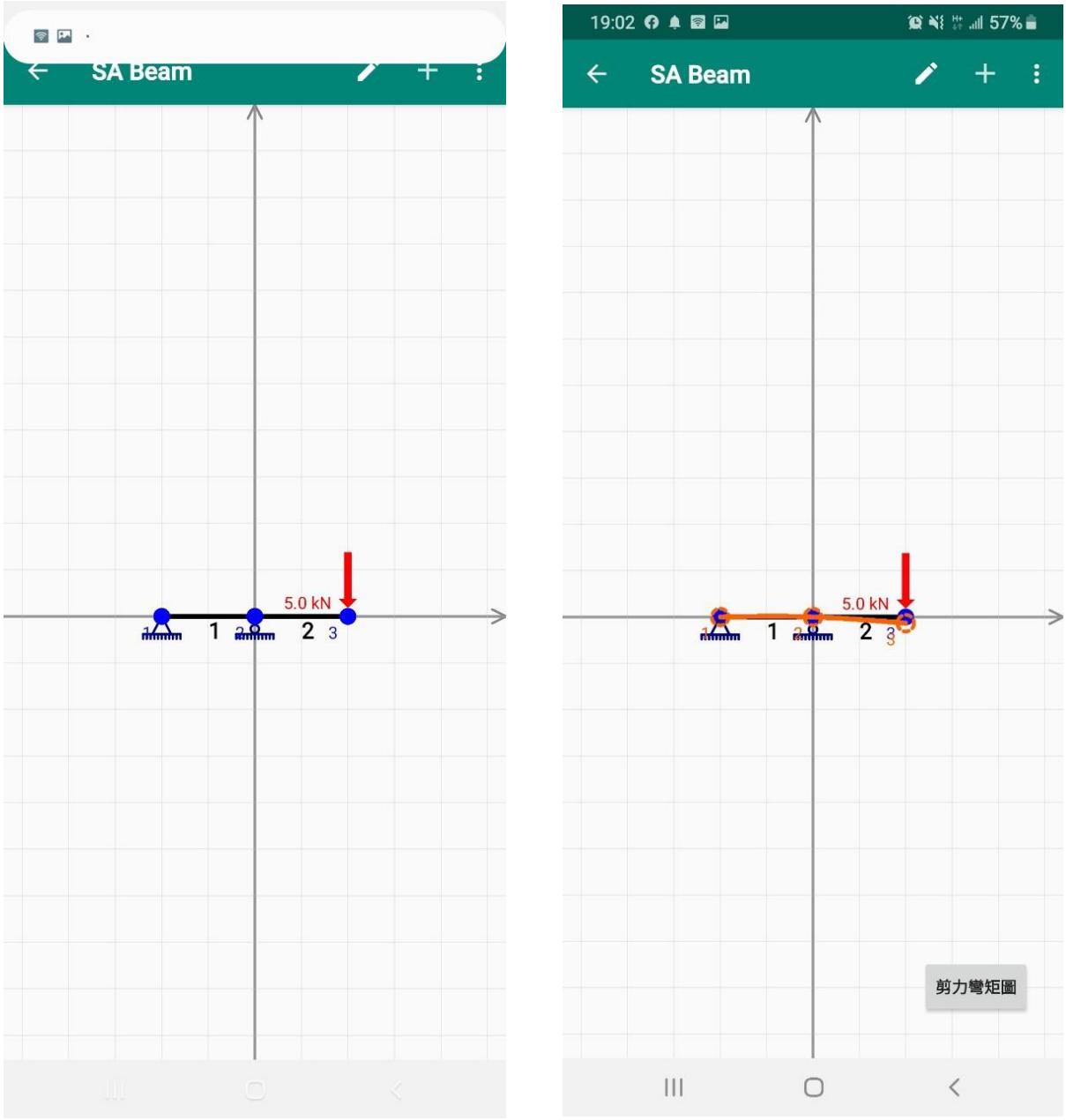


圖 81　Beam 程式計算後的答案介面(1)

計算出的垂直位移圖、扭轉角、剪力圖、彎矩圖如下，兩者比對的結果如表格 4(表中的點號為 APP 中表示的點號)

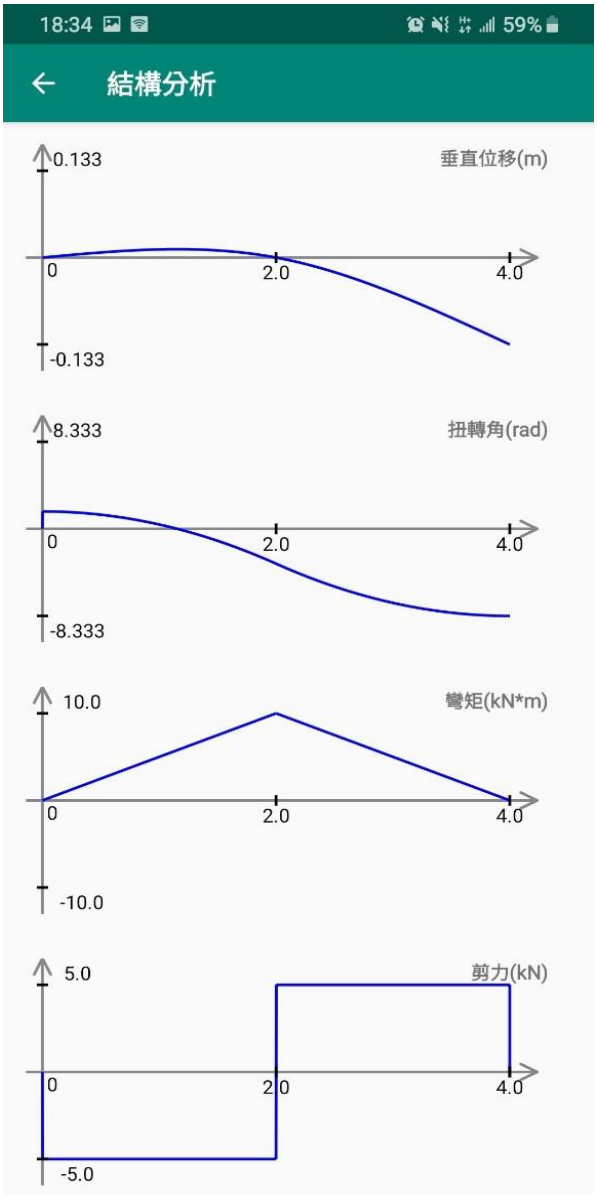


圖 82 程式計算後的答案介面(2)

表格 4 Beam 例題和程式計算答案之比較

點號	方向	例題的答案	使用 APP 計算的答案
1	Y	-5 (kN)	-5 (kN)
1	彎矩	0 (kN/m)	0 (kN/m)
2	Y	10 (kN)	10 (kN)
2	彎矩	0 (kN/m)	0 (kN/m)
3	Y	-5 (kN)	-5 (kN)
3	彎矩	0 (kN/m)	0 (kN/m)

第七章 結論

此計畫開發到計畫期限 8 月 31 日為止，該 APP 的功能有：

(1) Truss 介面:繪製節點、連接節點形成衍架(Truss)、在節點上施加外力、為節點設置支承、清除全部畫面、可點擊設置過的物件進行屬性設定、按下開始計算後輸出答案數值條列於左上角的 ScrollView 中、用箭頭顯示內力方向、施力後節點的位置。

(2) Beam 介面:繪製節點於橫向 X 軸上、連接節點形成樑(Beam)、在節點上施加外力和力矩、為節點設置支承、清除全部畫面、可點擊設置過的物件進行屬性設定、按下開始計算後顯示施力後樑的變形圖、出現按鈕前往輸出結果、在輸出結果視窗顯示隨著 Beam 起點至終點的剪力、彎矩、扭轉角以及垂直位移折線圖。

該成品尚未達成我們預想的完全體。因此，在計畫結束後，本小組還會持續擴充功能、改善使用者體驗以及優化程式執行效率，並在學期開始後提供給系內部分人員進行測試，收集使用者的使用心得以及錯誤報告，持續進行改善和修正，讓這個結構分析 APP 日臻完善。最終目標是讓此 APP 能夠發布到 Google Play 商店上，提供給更多的用戶使用。

經過這兩個月的開發，我們從甚麼都不了解，經歷了開發平台的選擇，再到深入了解介面程式和計算程式，我們了解到要開發出一款可以上架開放平台的 APP 是非常不容易的。Java 本身是一種複雜的語言，雖然功能完整齊全且能進行跨平台開發，但是寫起來是相當繁瑣。特別是物件導向的建立物件以及前後台的數據連接和交流相當的複雜較難以理解，而且在矩陣運算部分並沒有內建的封包可以直接使用，所以在有矩陣運算時我們需要自行設計邏輯和方法。

在介面設計時，由於 Android Studio 也有 Google 自行開發的內建封包，要全盤掌握它的代碼與設計邏輯也花了我們團隊非常久的時間。一開始的畫布座標問題就讓團隊中負責介面設計的人員苦惱許久，再到如何設計可選擇式的物件，還有跨類別的資料傳遞，甚至到如何在按下計算後，能控制我們所要顯示或隱藏的視窗...等等。雖然歷經了看不懂代碼、無限 debug、還有為了防止使用者進行的某些操作而苦思防呆邏輯的設置等等問題，我們最後還是完成了大部份的介面設計。而在計算程式的設計部分令我們卡關的部分是進入理論部分後的設計。雖然理論不難，但是要轉換成程式語言就是一項大挑戰。介面組受困的是 Android Studio 的代碼理解和平台使用機制，而程式組困難的地方則是將理論化成程式。我們團隊從這個研發企劃中學習到許多在程式課從未接觸過的工具套件和封包，還有理解到不同開發平台的使用方式和差異，以及理解到一個團隊中分工的分工需要動態調整，需要隨著計劃進度重新審視人力的分配。此次大暑計畫雖然讓我們確切的體會到自身的不足，但也對這兩個月的開發經驗帶來的收穫相當滿足。

程式設計的過程既痛苦然而卻又非常有趣，痛苦在常常找不到 BUG 從何而來，但有趣的是學到新東西的成就感。從無到有的 APP 開發不但增加我們程式設計能力，給了我們更加認識程式語言的機會，也帶我們體會到程式設計師的辛苦之處。

第八章 成果自評

預計目標

Beam 的計算、Truss 的計算、Beam 與 Truss 的複合結構計算、繪圖介面製作、計算結果整合(展示答案、標示各點位移、繪畫剪力彎矩圖)。

已達成目標

程式計算組:

1. Truss 的整體計算

2. Beam 的部分計算

3. 將計算結果提供給繪圖介面

介面組:

1. 繪圖介面製作

2. 列有功能之工具列

3. 自訂點座標

4. 能夠使用工具列功能畫出桿件結構

5. 顯示節點、桿件、外力以及支點資料

6. 計算結果整合(展示答案)。

待完成目標

程式計算組:

1. 分布力的添加

2. 預位移設計

3. 牛頓法計算極值

4. 支承種類擴充

5. 若使用者有點多餘點，其按下計算後的刪除功能(目前若有此情況會視為不可計算)

6. Truss 與 Beam 的複合結構程式

介面組:

1. 縮放與平移

2. 復原功能

3. 輸出極值至圖表介面

4. 將樑的截面顯示於彈出視窗

5. 一鍵更改所有桿件屬性

6. 編輯機制優化

計畫完成度評估

程式計算組：60%，介面組：80%，成品完成度評估：約 70%

我們這次主要有很大的部分都將時間花在介面設計上，由於要了解介面設計的程式邏輯需要花一些時間，因此到計畫後期，介面的設計進度有些許的落後。程式組在完成到一個段落後，也加入介面的設計。為了要設計出好的使用者體驗，我們不斷的吸收書籍和網路上的教學知識，充分理解代碼的組合邏輯，才有今日之結果。在未來，我們會利用課餘時間持續完成此作品，甚至能讓 IOS 系統之用戶也能使用我們的 APP。

結構分析 APP 下載點(限 Android 機種)：

如果無法進行安裝，至手機的設定→Google→安全性→Google Play 安全防護→將掃描裝置中的安全性威脅選項暫時關閉，下載完成後再開啟即可。

圖 83 結構分析 APP 載點

第九章 指導教授評語

本次計畫對學生的知識要求與工作量要求較高，應用 APP 主要分為兩個發展項目，一個是使用者介面的建立，第二個是基本理論轉換成程式的撰寫。由於內容涉及大三的結構學課程，參與同學在暑假期間必須自修將結構學的相關知識吸收並了解，就自我學習的這個項目上表現得非常優異。另外，使用者介面的開發也是仰賴大家蒐集資料、測試並與老師溝通，才能逐步將所需要的功能一一完成，所以本人對本屆大暑計畫中的幾位同學的評價是很高的，以下就各位學生的表現進行描述與建議：

王奕杰：身為團隊的隊長，奕杰負責通知、協調工作等聯絡事務，同時也是使用者介面的開發者，相對於其他的同學來說，他的貢獻最高，表現也最成熟，雖然對力學等數學的議題敏感度比較低，但是對程式開發方面有很好的天份與求知慾望。奕杰也是這次計畫中不可或缺的重要成員。我對他的執行能力、學習能力與自我要求都有很高的評價。(貢獻度 35%)

白謹瑜：小白的個性認真，對自己要求很高，也是整個團隊程度最好的一位。他在團隊中擔任的是技術領導的人，負責開發核心程式與結構分析的計算程式，基本上這個計畫也不能沒有他的貢獻。他的學習能力很強，是最常問問題、也都會問對的問題的那位隊員，他唯一需要加強的其實是台灣所有好學生的通病，就是什麼事情都想要自己解決，也什麼事情都想幫同學做，常常超過自己的負荷。因為小白對理論的理解能力很強，未來建議能夠走基礎研究的路線，他的天分也能夠充分的發揮。(貢獻度 35%)

林昆毅：昆毅在計畫中是協助的角色，比較沉默寡言，也比較難了解他的學習狀況，再加上他在計畫執行過程中有比較多的外務要處理，所以貢獻較少，不過他在計畫執行過程中仍然有全程參與，結構學的先修課程也能完整了解。希望這次計畫中所學到的知識，對他未來修習課程會有幫助。(貢獻度 15%)

李承曄:：承曄在計畫中也是輔助的角色，他跟昆毅都是幫助奕杰與小白開發各自的功能，也有充分參與到計畫的過程，只是後半段因為系上有其他活動，這個比較可惜一點，但是他也有完成結構學的先修課程與程式開發的過程，希望未來能夠好好學習努力。(貢獻度 15%)

參考文獻

[1] Hibbeler, Russell C. and Tan Kiang.，*Structural analysis*.，Pearson Prentice Hall，2015.

[2] Wang, An Pang.，*Android App 開發之鑰-使用 Java 及 Android Studio (6.X 以上終極適用版)*，R.O.C.深石數位科技，2018.09

[3] Wang, An Pang.，*Android 7.X App 開發之鑰： 使用 Java 及 Android Stuido*，R.O.C.深石數位科技，2017.08

[4] Shih, Wei Ming.，*Android App 程式設計腳本之無痛起步*，R.O.C.旗標科技股份有限公司，2018.05

[5] Google developrs，Android Studio user guide
https://developer.android.com/studio/intro?hl=zh-TW

[6] GcsSloop，*自定義 View 教學系列*，last updated：2019/04/29
https://github.com/GcsSloop/AndroidNote

[7] GcsSloop，*自定義 View 分類與流程*，last updated：2018/09/15
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B01%5DCustomViewProcess.md

[8] GcsSloop，*Paint 畫筆基礎*，last updated：2018/10/24
https://www.gcssloop.com/customview/paint-base

[9] GcsSloop，*Path 基本操作*，last updated：2018/10/05
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B05%5DPath_Basic.md

[10] GcsSloop，*Canvas 畫布操作*，last updated：2018/09/17
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B03%5DCanvas_Convert.md

[11] GcsSloop，*Matrix 原理*，last updated：2018/10/05
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B09%5DMatrix_Basic.md

[12] GcsSloop，*Matrix 詳解*，last updated：2017/07/09
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B10%5DMatrix_Method.md

[13] GcsSloop，*特殊控件的事件處理方案*，last updated：2016/11/22
https://github.com/GcsSloop/AndroidNote/blob/master/CustomView/Advance/%5B17%5Dtouch-matrix-region.md

[14] GcsSloop，*繪製座標系輔助工具-CanvasAidUtils*，last updated：2016/12/06
https://github.com/GcsSloop/ViewSupport/blob/master/Library/src/main/java/com/gcssloop/view/utils/CanvasAidUtils.java

[15] Shih, Wei Ming.，*最新 Java8 程式語言*，R.O.C.旗標科技股份有限公司，2017.10

[16] Tsai,Wen Lung.，*Java SE10 基礎必修課*，R.O.C.碁峰資訊股份有限公司，2018.07
[17] taurus770423，*[JAVA]Arraylist*，2015/02/06
https://taurus770423.pixnet.net/blog/post/281155324-%5Bjava%5Darraylist
[18]維基百科，*矩陣乘法*
https://zh.wikipedia.org/wiki/%E7%9F%A9%E9%99%A3%E4%B9%98%E6%B3%95